© 2026 Shuai Wang

Unified Automated Reliability Testing for Configurable Cloud Systems

by

Shuai Wang

Dissertation

Submitted in partial fulfillment of the requirements
for the degree of Doctor of Philosophy in Computer Science
in the Graduate College of the
University of Illinois Urbana-Champaign, 2026

Urbana, Illinois

Doctoral Committee:

Professor Darko Marinov, Co-Chair
Associate Professor Tianyin Xu, Co-Chair
Assistant Professor Reyhaneh Jabbarvand
Assistant Professor Yongle Zhang, Purdue University

Abstract

Modern distributed systems, such as cloud storage, data processing engines, and coordination services, are highly configurable and run for long periods under changing conditions. Their reliability depends not only on correct code, but also on how they behave under configuration changes and lifecycle operations such as restart and upgrade. Yet configuration changes and lifecycle operations are under-tested: regression testing exercises code mostly under a single default configuration and a steady-state lifecycle, although misconfigurations and restart faults are leading causes of production incidents with severe consequences such as outages and data losses.

This dissertation improves the reliability of configurable and distributed systems by *reusing existing tests*, the tests developers already write and maintain, to test code, configuration, and lifecycle changes, using runtime signals to make that reuse precise and efficient. It does so through four lines of work: the first three target configuration changes, and the fourth targets a lifecycle operation, graceful restart.

The dissertation first presents *Ctest4J*, a practical configuration testing framework for Java. Configuration testing checks a system's code together with the configuration values it will actually run on, catching both misconfigurations and code bugs that only a particular configuration exposes. Its key idea is the configuration test (Ctest): a test that takes a configuration as input and checks the code under it. Ctest4J lets developers write, generate, and run Ctests like ordinary unit tests, integrating with standard Java test frameworks so that configuration testing fits into existing development workflows. Evaluated on 12 open-source Java projects, Ctest4J runs Ctests 3.4× faster on average than prior Ctest tooling.

The dissertation next presents *uRTS*, which addresses a gap left by regression testing: code changes that pass under the default configuration can still fail under the configurations

used in production, and configuration changes are rarely tested against the latest code at all. uRTS is the first unified regression test selection technique that tests both kinds of change together, code under both default and production configurations, and configuration changes against the current code. Its key idea is to drive selection from the runtime dependencies among tests, code, and configuration parameters, and to compare a change against tests already run so that no test is rerun redundantly across many production configurations. On five large systems, uRTS reduces end-to-end testing time by 3.64× over running all tests.

The dissertation then presents *ConfFuzz*, which targets a blind spot of configuration testing: developers write Ctests for the configuration values they anticipate, but systems fail on values they did not anticipate. ConfFuzz reuses existing unit tests as fuzz drivers, mutating the configuration each test reads to search for values the system mishandles. Its key idea is to be parameter-aware: it tracks at runtime which configuration parameters a test actually exercises, and concentrates mutation and fuzzing effort on those parameters. Using 4,838 unit tests as drivers across nine cloud systems, ConfFuzz and its variants found 125 unique configuration-related bugs, of which 14 have been fixed and 12 further confirmed, and ConfFuzz covered 13.3%–14.2% more branches than its parameter-aware ablations.

Finally, the dissertation presents *Palingenesis*, which tests the restart lifecycle operation. Restarting a node is a routine operation, yet restart-specific bugs survive extensive testing because existing tests mostly run a cluster in steady state and never restart it. Palingenesis reuses these existing cluster tests and transforms them into restart tests. Its key idea is to inject a graceful restart at a meaningful point in a test while preserving the test's original workload and correctness oracle, using a large language model, guided by project context, to place the restart, and using runtime signals to refresh references that the restart invalidates so the test still runs afterward. Across seven distributed systems, Palingenesis transformed 4,459 cluster tests and found 17 restart bugs, of which 6 have

been fixed and 5 further confirmed.

Together, these lines of work show that the tests developers already have, when reused with runtime signals, can systematically expose reliability problems that arise from configuration and lifecycle changes.

Acknowledgements

Looking back on the years that led to this dissertation, what I remember most is not the papers or the deadlines, but the people who carried me through them. I did not get here on my own, and I am not sure I would have gotten here at all without them. Before I say anything else, I want to thank everyone who walked some part of this road with me.

I have been extraordinarily fortunate to be advised by Darko Marinov. Darko shaped how I think about research. From him I learned what it means to have taste, to recognize which problems are worth working on and which are not, and to hold myself to a standard of rigor and truthfulness that I had not known before. These are not lessons I will leave behind with this dissertation; they are how I now approach work of any kind. Darko also taught me to be deliberate in how I spend my effort. He repeatedly reminded me to validate whether an idea was feasible in the least time possible, rather than sink weeks into building something only to discover at the very end that it could never have worked. And he taught me to communicate. Whatever ability I now have to write a paper that someone wants to read, to frame an idea so that it lands, or to give a talk that holds an audience, I owe in large part to his patient, unsparing feedback, draft after draft. What I am most grateful for, though, is harder to put into a sentence. Darko cared about me as a person, not only as a student producing papers. Whenever I was sick he checked in with genuine kindness, and he looked out for my life beyond research. He kept his faith in me through the highs and the lows of this journey, and his steadiness in the lowest moments did more to keep me going than he probably knows. I could not have asked for a better example of what a researcher, and a mentor, should be.

I have been just as fortunate to be advised by Tianyin Xu, who gave me my start and in a real sense made this entire journey possible. It was Tianyin who offered me a summer research internship in 2020, in the middle of the pandemic, and who then opened the door for me to come to UIUC for my Ph.D. Without that first chance, none of what follows

would exist. Throughout the years since, Tianyin pushed me toward harder problems and more ambitious work, and held me to a high standard even when I would have settled for less. The decision I am most grateful to him for was one I resisted at the time. At a point when I wanted to switch directions, Tianyin urged me instead to dig deep into a single area rather than scatter my effort. It is that persistence, his insistence on it, that ultimately carried me through to the finish, and I understand now what I could not see then. Above all, Tianyin taught me how to think. He would not let me stop at the surface of a problem; he made me ask what the problem really was, again and again, until I understood it. That habit of mind is the most valuable thing I take from my Ph.D., and it is his.

I am grateful to the other members of my committee, Reyhaneh Jabbarvand and Yongle Zhang, for their time and guidance. I had the privilege of working with Reyhaneh on a project led by Runxiang Cheng, where she offered a perspective different from my own; she also gave me valuable advice on my presentation during my preliminary exam, including how to motivate the background more thoroughly. Yongle gave me far more than I could ever expect of a committee member. He was, in effect, a third advisor to me. Without him, ConfFuzz and Palingenesis would have been a far harder road. Yongle brought a systems mindset and a different way of approaching problems; he would often ask questions I could not answer, forcing me to confront what a problem or a difficulty was really about, which did more for my growth as a researcher than almost anything else. He also gave tirelessly of his time on our papers, rebuttals, and major revisions. He was an indispensable part of my path through this Ph.D.

Before UIUC, it was at the University of California, Riverside that I first found my way into research. I am grateful to Daniel Wong and Heng Yin, who introduced me to what research is and gave me my earliest taste of it. They encouraged me to pursue a Ph.D. and helped me along the way to applying. Without that early push and guidance, I would never have set out on this path at all.

I owe a great deal to the collaborators who made the work in this dissertation possible.

Runxiang (Sam) Cheng was a senior student and mentor to me from early on. We worked together on LongRTS, and through that collaboration I saw up close how he does research, the care he takes with details, and a breadth I still find remarkable: he moved into a very different area for a stretch of his Ph.D. and thrived there all the same. Sam set an example of what it looks like to be both meticulous and versatile, and I learned a great deal simply by working alongside him. Yinfang Chen has been one of the closest friends of my entire Ph.D. He was among the first people I met at UIUC, and from there we became collaborators, roommates, and confidants. We started this journey in the same cohort and walked through all of it together. We hauled U-Hauls across town on moving days, traveled together, and spent countless nights talking, drinking, and playing cards, all the things brothers are supposed to do for one another. A friendship like this is rare to find during a Ph.D., and I treasure it. In the hardest moments, we leaned on each other to keep going, and I cannot imagine these years without him. I was also fortunate to mentor two talented students who became real collaborators. Hao (Tony) Wang was my co-author on ConfFuzz; the project came together through his hard work as much as mine, and although I was nominally mentoring him, I learned plenty from him in return. His diligence and reliability made him a pleasure to build something with. Xinyu Lian was my co-author on Ctest4J and uRTS, two works central to this dissertation. His careful, dependable work was essential to both, and I am grateful for everything he contributed.

I was lucky to share these years with a wonderful group of labmates. From my own cohort, I thank Siyuan Chai and Tyler Gu for going through it all alongside me. I am grateful as well to the many senior and junior members of the group whose company, discussions, and camaraderie made the lab a place I was glad to come to: Xudong Sun, Jinghao Jia, Xuhao Luo, Yang Chen, Yicheng Ouyang, Sam Grayson, Lilia Tang, Jiawei Liu, Steven Xia, Shizhuo Zhang, Zijie Zhao, Jiyuan Zhang, Wentao Zhang, Cathy Cai, Yiming Su, Anna Mazhar, Wenqing Luo, Parth Thakkar, Saad Sher Alam, Xuecheng Liu, Xinyu Wu, Yuxuan Jiang, Zicheng Ma, Yicheng Lu, Qingyu Li, and Chaitanya B.

Bhandari. Thank you all.

Outside of research, it was music that carried me through the hardest stretches of this Ph.D. There were times when nothing about the work seemed to be going right, and it was picking up an instrument, and the people I played it with, that pulled me back out and gave me the energy and the hope to keep going. I owe more than I can say to the two bands that shaped my years here. To my bandmates in Sadless and RammScorn, Xuanzi (Tony) An, Yuying (Skylar) Song, Hao Kang, Shiqi (Katharina) Cheng, Yixin (Leo) Luo, Yuliang (Jason) Cai, Qianfu (Edward) Tang, Lili Sheehan, Anya Luo, and Dingming (Adam) Yang, thank you for the music, and for everything those years meant to me. Tony deserves a special word. Beyond the band, he was my roommate for the last two years of my Ph.D., and I came to think of him not just as a roommate but as a younger brother. I am grateful for the time we shared during these years, living together, studying together, and sharing the joy of music, and for the many memories we made along the way. I am grateful, too, to my friends in the Jasmine Field Orchestra, Huanyi Jiang, Jacky Xiang, Ziwei (Wills) Liu, Yile Huang, Zehua (Neo) Yuan, Xinyang Yang, Yiming Niu, Shirley Zhuang, Runhan Yang, Enming Liang, Haojiang Shi, Zhenhai Tu, Feifan Chen, Weitong Chen, Wenxuan Bao, and many others I cannot all name here, for the joy of making music together. Beyond music, I want to thank Yulun Wu, Zeyuan Yu, Yubo Xia, Yibin (Tom) Chen, Yihan Ni, Wenzhe Mi, Yichen Cao, Zhilin Chen, and Jie Li, with whom I shared time around cars, for the fun and the welcome escape.

I have not forgotten the friends I made when I first arrived in the United States, at UC Riverside, who were my earliest home away from home: Xinyao Wang, Boxiang (Box) Zhang, Wenjing Huang, Ziying Deng, Tong Jia, Junze Liu, Tiancheng Yang, Runyao (Rosa) Tang, Luyang Yu, Ziqing Yu, Yizhi Zhou, Yaming Zhang, Jiahe Cao, and many more I can no longer name one by one.

Finally, and most of all, I want to thank my family. To my parents: everything I am, I owe to you. You held me up so that I could chase the life I wanted, and you supported

every path I chose, unconditionally and without a single doubt, even when it took me far from home. I know the distance was not easy on you. There was a stretch of nearly two years when I could not come home at all, and over video calls my mother would cry and tell me how much she missed me, just as she has cried every time I leave home and every time I return. My father is a man of few words, but in every one of those calls he sat quietly beside her, listening with care to every small thing I had to share. I carried that image of the two of you with me through the hardest days of this Ph.D. Thank you for your endless love. I hope I have made you proud. I want to thank my dear friend Guangyu Li, who has been like a brother to me, for standing by me through the years. Last, I want to thank my beloved, Ziying Yang. You have made my life full, happy, complete, and whole in a way it simply was not before. Thank you for the joy you bring to my every day, for taking me to see a wider world and to taste all the things I would never have discovered on my own, and for your patience and your love through all of it. Whenever the pressure grew too heavy, a single hug from you was enough to fill me with energy again and send me back to face the challenges of work and life. When I could not return to China, you went in my place to visit my parents and care for my family, a kindness I will never forget. I am grateful, too, to your parents for their warmth and their unwavering support of us, and especially to your mother, Lijuan Shao, for treating me as her own. There is no way to repay all that you have given me, and I will not try; I will only carry this love forward with you, hand in hand, toward the brighter and happier future we are building together.

I would not be who I am, or where I am, without every one of you. Thank you, from the bottom of my heart.

Contents

Chapter 1 Introduction . . . 13
1.1 Problems and Challenges . . . 14
1.2 Thesis Statement . . . 16
1.3 Contributions . . . 16
1.4 Dissertation Organization . . . 19
Chapter 2 Ctest4J: A Practical Configuration Testing Framework for Java . . . 20
2.1 Overview . . . 20
2.2 Background: Configuration Testing . . . 21
2.3 Usage . . . 23
2.4 Implementation . . . 27
2.5 Evaluation . . . 30
2.6 Discussion . . . 31
2.7 Summary . . . 32
Chapter 3 Unified Regression Test Selection for Code and Configuration Changes 33
3.1 Overview . . . 33
3.2 Background . . . 35
3.3 Unified Regression Testing . . . 38
3.4 The uRTS Algorithm . . . 40
3.5 Implementation . . . 43
3.6 Evaluation . . . 47
3.7 Discussion . . . 55
3.8 Summary . . . 56
Chapter 4 ConfFuzz: Parameter-Aware Greybox Configuration Fuzzing for Cloud Systems . . . 57
4.1 Overview . . . 57
4.2 Background and Motivation . . . 59
4.3 Design . . . 62
4.4 Implementation . . . 69
4.5 Evaluation . . . 70
4.6 Discussion . . . 81
4.7 Summary . . . 82

Chapter 5 Palingenesis: Testing Graceful Restart in Distributed Systems 84
5.1 Overview 84
5.2 Background and Motivation 86
5.3 Design 90
5.4 Implementation 96
5.5 Evaluation 97
5.6 Discussion 106
5.7 Summary 107
Chapter 6 Related Work 108
6.1 Regression Test Selection 108
6.2 Configuration Testing and Fuzzing 108
6.3 Lifecycle and Reliability Testing 110
Chapter 7 Conclusions and Future Work 112
7.1 Conclusions 112
7.2 Future Work 113
7.3 Closing Remarks 114
References 116

Chapter 1 Introduction

Modern software runs on large-scale distributed systems, such as cloud storage, data-processing engines, and coordination services [1], [2], [3], [4]. These systems are highly configurable, exposing hundreds to thousands of configuration parameters that span diverse types, semantics, and constraints [5], [6], [7], [8], [9], [10]. They also run continuously and pass through routine lifecycle operations, such as restarts and rolling upgrades, that take nodes down and bring them back while the system keeps serving [1], [2], [11], [12]. Their reliability is paramount: because applications and data sit atop them, a single fault can affect every upper-layer service [13], [14].

Failures of these systems have two long-standing root causes, faults in program code and errors in configuration [13], [15], [16], [17], [18], [19], [20]. Misconfigurations are a dominant cause in practice. They are widespread in deployed software and can lead to the same problems as code bugs: crashes, hangs, and silent failures [21], [22], [23], [24]. Faulty configurations were the second-largest cause of service disruptions in a major Google production service [25], and at Meta, 16% of service-level incidents, including significant outages, were introduced by configuration changes [5], [16]. Lifecycle operations are a third source: restarting a node exercises a shutdown and rejoin protocol whose bugs cause correctness violations, rare races, and hangs that are hard to reproduce and often invisible under steady-state testing [26], [27], [28].

Testing is the standard way developers guard reliability, and decades of work on regression testing and test selection have made it efficient to rerun the right tests after a code change [29], [30], [31], [32], [33]. Yet current practice under-tests both configuration and lifecycle changes. Regression testing exercises code mostly under a single *default* configuration, while deployed software runs on *production* configurations that differ from the default [29], [32], [33]. Code changes are therefore not tested under the configurations they will actually run on, configuration changes are rarely tested against the latest code, and the values a system will encounter in the field are not explored at all. Lifecycle operations

are similarly neglected: most existing tests run a system in steady state and never restart it, so restart-specific bugs survive even extensive crash and recovery testing [20], [27], [28].

The gap has a common cause: writing tests for configuration and lifecycle changes from scratch is expensive, and the tests developers already have are tied to a fixed configuration and a steady-state lifecycle. Those existing tests are a resource. With the right runtime signals, they can be reused to test code, configuration, and lifecycle changes rather than written anew. This dissertation develops that idea through four lines of work: the first three target configuration changes, and the fourth targets a lifecycle operation, graceful restart.

1.1 Problems and Challenges

1.1.1 How to Test Code Together with the Configurations It Runs On?

Regression testing checks code under the default configuration, but the same code runs in production under many different configurations. A code change can pass every regression test under the default configuration and still fail in production, and a configuration change is rarely tested against the latest code at all [29], [32], [33]. Prior work on configuration reliability largely validates configuration values against inferred rules or diagnoses errors after they manifest [34], [35], [36], [37], [38], [39], [40], but checking values alone misses code bugs that only a particular configuration exposes. Configuration testing [41], [42] closes this gap by checking the code together with the configuration values it will run on. Making configuration testing usable, however, raises its own challenges. Tests that take a configuration as input must integrate with the test frameworks developers already use, run with low overhead, and be selected efficiently. Rerunning every test for every code and configuration change across many production configurations is otherwise prohibitively expensive. This dissertation asks: how can developers write, run, and select configuration tests as a practical part of regression testing?

1.1.2 How to Find the Configuration Values a System Mishandles?

Configuration tests check the configuration values a developer anticipates, but systems fail on the values they do not anticipate. Highly configurable systems accept a vast space of parameter values with intricate constraints [5], [6], [7], [8], and developers cannot enumerate, at test-writing time, every value that the code might mishandle. Greybox fuzzing explores large input spaces by coverage-guided mutation [43], [44], [45], [46], and recent work fuzzes program options and configurations directly [47], [48], [49], [50]. Applying it to large configuration spaces is hard, however: the relevant parameters differ from test to test and are only revealed when the code runs, so a blind search over all parameters wastes most of its effort on values no test exercises. This dissertation asks: how can the configuration-value space be explored automatically to expose configuration-related bugs, reusing the tests developers already have?

1.1.3 How to Test Lifecycle Operations Such as Restart?

Restarting a node is a routine operation in production distributed systems, performed for rolling upgrades, configuration changes, maintenance, and performance tuning [11], [12], [51], [52]. Distributed-systems reliability testing has been studied extensively, through fault injection, crash and recovery testing, chaos engineering, and model checking [20], [27], [28], [53], [54], [55], [56]. Unlike a crash, however, a planned restart must validate the shutdown and rejoin protocol itself, so restart-related bugs can remain undetected even after extensive crash testing [26]. Yet most existing tests run a system in steady state and never restart it. Reusing these tests to test restart is challenging: a restart must be injected at a point where it is meaningful without breaking the test's original workload and correctness oracle, and a restart invalidates the references a test holds (client handles, service stubs), so the test can no longer run to completion afterward. This dissertation asks: how can existing tests be transformed to test the graceful-restart lifecycle operation while preserving what they check?

1.2 Thesis Statement

We can improve distributed system reliability by reusing existing tests against code, configuration, and lifecycle changes using runtime signals.

This statement has three aspects. Existing tests, the tests developers already write and maintain, can be reused rather than written anew. By runtime signals we mean the dependencies and parameters a test exercises as it runs, which make that reuse precise and efficient. The same idea spans code, configuration, and lifecycle changes, three sources of failure that current testing under-covers.

1.3 Contributions

To support this thesis statement, this dissertation makes four contributions. The first three reuse existing tests to test configuration changes: Ctest4J makes configuration testing practical (Chapter 2), uRTS selects which tests to rerun for code and configuration changes (Chapter 3), and ConfFuzz reuses tests to search the configuration-value space (Chapter 4). The fourth, Palingenesis, reuses existing tests to test a lifecycle operation, graceful restart (Chapter 5).

1.3.1 Ctest4J: A Practical Configuration Testing Framework (Chapter 2)

Configuration testing [41], [42] checks a system's code together with the configuration values it will run on, catching both misconfigurations and code bugs that a particular configuration exposes. Its core primitive is the *configuration test* (Ctest): a test that takes a configuration as input and checks the code under it. Prior configuration-testing work focused on generating Ctests and prioritizing them [41], [57], [58], but for configuration testing to be adopted, developers must also be able to write, generate, and run Ctests as easily as ordinary unit tests.

Contributions: This dissertation presents *Ctest4J*, a practical configuration testing framework for Java. Ctest4J integrates with standard Java test frameworks so devel-

opers can write, generate, and run Ctests like ordinary unit tests, requiring only about 20 lines of configuration-API instrumentation per project. Evaluated on 12 open-source Java projects, Ctest4J runs Ctests 3.4× faster on average, and up to 41.3× faster, than prior Ctest tooling, with at most 1.27× runtime overhead over a vanilla test runner.

1.3.2 uRTS: Unified Regression Test Selection (Chapter 3)

Regression test selection reruns only the tests a change can affect [29], [30], [31], and has been made practical at file, class, and hybrid granularity and deployed in industrial CI [32], [59], [60], [61], [62], [63], [64]. Traditional selection, however, considers only code changes under the default configuration [29], [32], [33]. It neither tests code under production configurations nor tests configuration changes against the latest code, and rerunning tests for every production configuration is expensive.

Contributions: This dissertation presents *uRTS*, the first unified regression test selection technique that tests code changes under both default and production configurations and tests configuration changes against the latest code. uRTS drives selection from the runtime dependencies among tests, code, and configuration parameters, and uses a two-dimensional comparison to avoid rerunning a test redundantly across many production configurations. On five large systems, uRTS reduces end-to-end testing time by 3.64× over running all tests and by 1.87× over safe file-level regression test selection.

1.3.3 ConfFuzz: Configuration Fuzzing (Chapter 4)

Ctests check the values a developer anticipates, but highly configurable systems fail on values they do not anticipate [5], [6], [7], [8]. The space of parameter values is too large to enumerate by hand, and the parameters that matter differ from test to test and are revealed only at runtime. Greybox fuzzing [43], [44], [45] can explore such spaces, and fuzz drivers can even be derived from existing unit tests [65], but prior configuration and option fuzzers [47], [48], [50] do not target the parameters each test actually exercises.

Contributions: This dissertation presents *ConfFuzz*, which reuses existing unit tests as fuzz drivers and mutates the configuration each test reads to search for values the system mishandles. ConfFuzz is parameter-aware: it tracks at runtime which configuration parameters a test actually exercises and concentrates mutation and fuzzing effort on those parameters. Unit tests are abundant in mature projects and execute far faster than the system tests used by prior cloud-system configuration fuzzers, giving the test execution speed greybox fuzzing demands. Using 4,838 unit tests as drivers across nine cloud systems, ConfFuzz and its variants found 125 unique configuration-related bugs, of which 14 have been fixed and 12 further confirmed. ConfFuzz covered roughly 3.2 times more branches than the ConfigFuzz-style greybox baseline and 13.3%–14.2% more than its two parameter-aware ablations.

1.3.4 Palingenesis: Graceful-Restart Testing (Chapter 5)

Restarting a node is a routine operation, yet restart-specific bugs survive testing because most existing tests run a system in steady state and never restart it [26], [27]. Large language models can transform and generate code and tests [66], [67], [68], and benchmarks now measure this capability on real software engineering tasks [69], which opens a path to reusing existing tests for restart. Doing so requires injecting a restart at a meaningful point without breaking the test's workload or oracle, and keeping the test runnable after the restart invalidates the references it holds.

Contributions: This dissertation presents *Palingenesis*, which reuses existing cluster tests and transforms them into restart tests. It injects a graceful restart at a meaningful point in a test while preserving the original workload and correctness oracle, using a large language model, guided by project context, to place the restart, and uses runtime signals to refresh the references a restart invalidates so the test still runs afterward. Across seven distributed systems, Palingenesis transformed 4,459 cluster tests and found 17 restart bugs, of which 6 have been fixed and 5 further confirmed.

1.4 Dissertation Organization

The rest of this dissertation is organized as follows. Chapter 2 presents Ctest4J, the configuration testing framework and the Ctest primitive the later configuration-testing works build on. Chapter 3 presents uRTS, which selects tests for code and configuration changes. Chapter 4 presents ConfFuzz, which reuses tests to fuzz the configuration-value space. Chapter 5 presents Palingenesis, which transforms existing tests to test the graceful-restart lifecycle operation. Chapter 6 discusses related work on configuration testing, regression test selection, fuzzing, and distributed system reliability testing. Chapter 7 concludes and outlines future work.

Chapters 2 through 5 are based on published papers. Chapter 2 (Ctest4J) was published in the Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion 2024) [70]. Chapter 3 (uRTS) was published at the 45th International Conference on Software Engineering (ICSE 2023) [71]. Chapter 4 (ConfFuzz) was published at the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE 2026) [72]. Chapter 5 (Palingenesis) was published at the 37th IEEE International Symposium on Software Reliability Engineering (ISSRE 2026) [73].

Chapter 2 Ctest4J: A Practical Configuration Testing Framework for Java

2.1 Overview

Configuration testing [41], [42] is a recently proposed approach for rigorously testing software configurations, similar to how software code is tested today. The key motivation is to test production system configurations before deploying them: configuration testing connects configurations to software tests so that configuration changes can be tested in the context of code affected by the changes. A configuration test (Ctest) is a test that takes a system configuration as input and checks the configuration together with the code. Prior work [41] has shown that configuration testing outperforms previous approaches [5], [6], [19], [35], [74] for detecting failure-inducing configuration changes, including sophisticated misconfigurations and valid configuration changes that trigger dormant software bugs. Regression test selection [41], [71] and test-case prioritization [57], [58] have been developed to make configuration testing more efficient for continuous integration and deployment.

However, despite active research on configuration testing from several groups [41], [42], [48], [57], [71], [75], including Ctests for fuzzing [48] and unsafe parameter detection [75], there has been no practical, systematic framework for configuration testing. Prior research used *ad hoc* scripts [76] that are deficient for practical use: they do not support parallel test execution, have poor maintainability due to external dependencies (requiring a file that maps configuration parameters to the tests that use them), work only for JUnit4 and Maven, and require manual instrumentation of the configuration API. This deficient support makes configuration testing harder to adopt and even hampers research: all prior papers [41], [42], [48], [57], [71] evaluated configuration testing on a fixed set of five or six open-source projects.

We present Ctest4J, a practical configuration testing framework for Java projects. Ctest4J provides new source-code annotations so that developers can write and maintain Ctests similarly to regular unit tests, connecting the configuration values under test

with the corresponding Ctests, and provides automated code instrumentation for common configuration APIs. Ctest4J supports the most popular Java-based test frameworks (JUnit4, JUnit5, and TestNG) and build systems (Maven and Gradle). In sum, Ctest4J provides the following features:

- **Parallel test execution.** Ctest4J supports running Ctests with different configurations in parallel, addressing the sequential-only execution of prior Ctest scripts (caused by sharing static configuration objects). Ctest4J also incurs a low runtime overhead; a Ctest runs only ∼3% slower on average than a regular unit test.

- **Maintainability.** Ctest4J provides source-code annotations to make Ctests easy to maintain. Developers specify the mapping between a Ctest and its configuration parameters as annotations inside the test code, without a dependency on separate external files. For backward compatibility, Ctest4J also supports the original mapping files used by prior scripts.

- **Automation.** Configuration API instrumentation is a necessary step to enable Ctests. Ctest4J uses AspectJ to automate the instrumentation, easing the adoption of configuration testing for existing projects. With our automation, enabling Ctests takes only 20 lines of code on average across 12 Java projects.

Ctest4J is available at `https://github.com/xlab-uiuc/ctest4j` and released in the Maven Central Repository.

2.2 Background: Configuration Testing

This section introduces the configuration-test (Ctest) primitive that this chapter, Chapter 3, and Chapter 4 build on. Later chapters reuse Ctests as the unit of configuration testing; the uRTS (Chapter 3) and ConfFuzz (Chapter 4) chapters refer back to this section rather than redefining Ctests.

2.2.1 Definition

A Ctest, $\hat{t}(\hat{P})$, is a test $\hat{t}$ parameterized by a set of system configuration parameters $\hat{P}$ [41]. Running a Ctest instantiates each parameter $p \in \hat{P}$ with a concrete value, either from a production system configuration or from a configuration change (a configuration file "diff") to be deployed. $\hat{P}$ is typically a very small subset of all system configuration parameters, denoted $\mathbb{P}$; that is, $|\hat{P}| \ll |\mathbb{P}|$. Like regular software tests, a Ctest exercises the program and asserts on program properties (correctness, performance, security); Ctests can be unit, integration, or system tests. A key property is that when a Ctest is instantiated with the default configuration, it behaves as a regular unit test. This equivalence is what lets a single framework run both Ctests and regular tests, and what lets uRTS treat the two uniformly (Chapter 3).

Configuration testing detects *both* misconfigurations (invalid values assigned to configuration parameters) and bugs in the code that are exposed by changing configuration parameters to new *valid* values [41]. The latter is significant: existing configuration validation and misconfiguration detection techniques only check configuration *values* and cannot detect configuration changes whose root cause is in the code, e.g., a valid change that exposes a dormant bug. By connecting tests to production configurations, Ctests can reason about program behavior under those configurations and detect failures that value-only validation misses.

2.2.2 Generating Ctests

Ctests can be written from scratch, but in mature projects they are typically *generated* by transforming existing tests [41], reusing the well-engineered test logic and assertions already present. Generation proceeds in two steps: an existing test t is first parameterized by the configuration parameters of the target system, then transformed into a Ctest. The main challenge is identifying which parameters can be in $\hat{P}$: a test's logic or oracles may assume specific configuration values (implicitly, by assuming defaults, or explicitly,

by hardcoding them), so naïve parameterization does not always yield a valid Ctest. If $\hat{t}$ assumes a specific value for a parameter p, then $\hat{t}$ cannot be a Ctest for p (it would fail incorrectly on other valid values), though it may still be a valid Ctest for the other parameters it exercises. The original Ctest work generated 7,974 Ctests by transforming the existing test suites of five cloud systems and showed that the generated Ctests detected the failure-inducing configurations in 96.9% of 64 real-world configuration-induced failures, detected 10 misconfigurations in 7 deployed configuration files, and exposed 14 previously unknown bugs [41].

2.2.3 Using and Selecting Ctests

A Ctest can check an entire system configuration or a configuration change. To check a configuration change (a diff D that updates a small set of parameters), only the Ctests parameterized by at least one changed parameter need to run; this *test selection criterion* avoids rerunning all Ctests after every configuration change [41]. Selecting and prioritizing Ctests is central to using configuration testing in continuous integration and deployment, and is the subject of later work [57], [71] that this dissertation extends in Chapter 3. The remainder of this chapter presents Ctest4J, a framework that makes writing, maintaining, running, and selecting Ctests practical for real Java projects.

2.3 Usage

We present a high-level overview of configuration testing with Ctest4J.

2.3.1 Writing Ctests

Ctest4J provides two source-code annotations for developers to mark Ctest classes and methods:

- `@CtestClass` marks that a class is a Ctest class;
- `@Ctest` marks that a method is a Ctest method.

These annotations specify the configuration parameters of the corresponding Ctest(s) in a class or a method. Making the configuration parameter usage explicit (1) substantially aids debugging (developers gain insight into which parameters are used by a Ctest), (2) enhances test selection [41], [71] and prioritization [57], and (3) enables more efficient Ctest fuzzing [48]. Both annotations can specify configuration parameter names used by the Ctest(s) as a list of parameters and as a regular expression to match parameters. For backward compatibility with the original Ctest scripts [76], `@CtestClass` can also specify a file containing the mapping between configuration parameters and Ctest methods. Lists, regexes, and files can be provided together, and Ctest4J unions them into one final mapping. Ctest4J's Ctest runner is specified within the Ctest class; for example, Ctest4J's `CtestJUnitRunner` is added through the usual `@RunWith` annotation in JUnit4.

Figure 1 is a simplified Ctest example from Hadoop Yarn with Ctest4J. We annotate the existing class `TestFSDownload` with Ctest4J's runner `CtestJUnitRunner` (line 1) and `@CtestClass` (line 2), and the test method `testDownload` with `@Ctest` (lines 8–10). `@CtestClass` specifies the class-level configuration parameters used by all the Ctest methods in the class; these mostly come from test setup and teardown. In this example, the method `getRecordFactory()` (line 6) uses the parameter `yarn.ipc.record.factory.class`, making it a class-level parameter. `@Ctest` specifies the method-level configuration parameters used in the body of the Ctest method; here `testDownload` uses four method-level parameters, specified through a regular expression. These parameters are used by various methods invoked during the test, e.g., `createFileSystem()` (invoked by `getLocalFSFileContext()`) uses `local.block.size`. During Ctest execution, Ctest4J instantiates every configuration parameter used by each Ctest method with the configuration under test, e.g., a production configuration.

```java
@RunWith(CtestJUnitRunner.class)
@CtestClass({"yarn.ipc.record.factory.class"})
public class TestFSDownload {
  private Configuration conf = new Configuration();
  static final RecordFactory recordFactory =
    RecordFactoryProvider.getRecordFactory(null);

  @Ctest(regex="fs.(client.resolve.remote.symlinks|" +
          "permissions.umask-mode|local.block.size|" +
          "AbstractFileSystem.file.impl)")
  public void testDownload() {
    // Create FileContext with parameters in @Ctest
    FileContext files =
      FileContext.getLocalFSFileContext(conf);
    ...
    // Start downloading
    FSDownload fsd = new FSDownload(files, ...)
    Path path = fsd.download(...);
    ...
    // Check whether the download is done
    assertTrue(path.isDone())
    ... // Check other properties of the downloaded file
  }
}
```

Figure 1: A Ctest in Yarn with Ctest4J. The configuration parameters in the code snippet are instantiated by values from the configuration under test (not shown in the figure).

2.3.2 Configuration API Instrumentation

To enable Ctests in a Java project, developers instrument the configuration API with Ctest4J so that Ctest4J can instantiate Ctests with the configuration under test at runtime. Ctest4J also provides APIs to track the usage of configuration parameter values during Ctest execution; tracking is important for debugging, for maintaining the input parameter set of each Ctest, and for adequacy measurement (e.g., coverage of configuration parameters of a Ctest suite). Ctest4J focuses on common configuration API patterns in Java projects, many of which use a unified configuration class with two basic API abstractions, GET and SET [9], [22], [40], [41], [77], [78]. GET APIs of the form "`<T> get(String parameter)`" take a parameter name and return a value; SET APIs of the form "`void set(String parameter, <T> value)`" set the value of the given parameter. Configuration APIs built on `java.util.Properties` and `org.apache.commons.configuration` follow this

pattern.

Instrumenting the configuration API. Without Ctest4J, all test executions would use only the default configuration. Ctest4J modifies the execution so that tests run with the configuration under test. Ctest4J instruments the configuration API with `connectProdConfig`, a static method that takes the configuration SET API as input and connects the configuration under test to the configuration objects used by the Ctests; typically it is added to the constructor of the configuration class. The following snippet shows how to instrument the configuration API of Apache Hadoop Yarn:

```
public Configuration() {
    this(true);
+   Ctest4J.connectProdConfig(
+       (name, value) -> set(name, (String) value));
}
```

For projects without a unified configuration class where Ctest4J can modify the SET API, Ctest4J supports a lazy instrumentation mode that modifies the GET API to use the configuration under test.

Tracking configuration parameters. Ctest4J provides a static `markParamAsUsed` method to track the usage of configuration parameter values during Ctest execution. Ctest4J calls `markParamAsUsed` upon the invocation of a configuration GET API, as shown in the following snippet from Hadoop:

```
public String get(String name) {
+   Ctest4J.markParamAsUsed(name);
    String[] names = handleDeprecation(deprecationContext.get(), name);
    String result = null;
    for(String n : names) {
+       Ctest4J.markParamAsUsed(n);
        result = substituteVars(getProps().getProperty(n));
    }
    return result;
}
```

For projects with multiple GET APIs, `markParamAsUsed` is placed at every API to ensure complete tracking. Ctest4J provides an AspectJ [79]-based utility to instrument the configuration API automatically (by specifying the fully qualified API names), if source-code changes are not preferred.

2.3.3 Running Ctests

Running a Ctest is similar to running a regular unit test; with Maven, Ctests run with Maven Surefire (`mvn test`). Ctest4J provides three modes:

- `debug`: run the Ctest with the default configuration and check whether all required parameters are used during execution, useful for developing and debugging Ctests;
- `prod`: run the Ctest with the configuration under test;
- `default`: run the Ctest with the configuration under test and check whether all required parameters are used.

Ctest4J supports running Ctests in parallel with different configuration files, and also supports input configuration through command-line arguments. If no configuration file or command-line argument is specified, Ctest4J runs the Ctest with the default configuration; in this case, a Ctest falls back to a regular unit test. Ctest4J implements parameter-aware Ctest selection [41]; we plan to integrate advanced test selection such as uRTS [71] (the subject of Chapter 3) and prioritization [57].

2.4 Implementation

The current Ctest4J implementation has ∼5,000 lines of Java code. Ctest4J takes three inputs: (1) the Ctest code, (2) the configuration under test (ConfUT), and (3) optionally a mapping file between parameters and the Ctests that use them (required by the original Ctest scripts [76]). Ctest4J processes the test annotations and instantiates the execution of Ctests. Figure 2 shows the four main components.

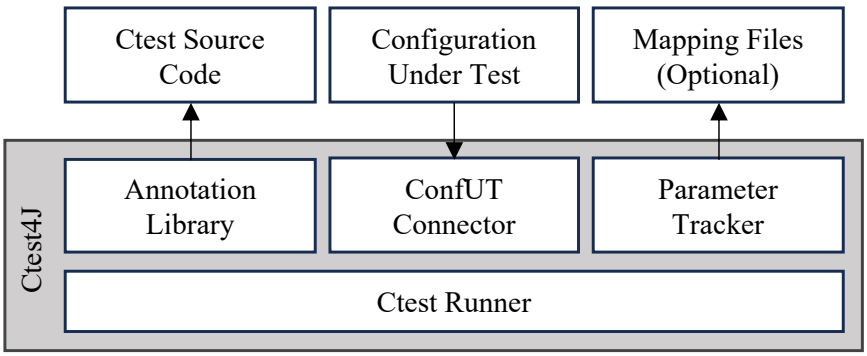


Figure 2: Overview of Ctest4J.

2.4.1 Annotation Library

The annotation library processes the annotations in the test code; the annotations are conceptually similar to the `@Test` annotation for regular unit tests. `@CtestClass` and `@Ctest` mark that the annotated class or method, respectively, is a Ctest, and specify its configuration parameters using a list or a regular expression. To get a precise parameter set from a regular expression, Ctest4J does not accept match-any operators (`".*"` or `".+"`), which could match too many parameters. `@CtestClass` can also specify parameters from a JSON mapping file with two fields: class-level parameters (required by all Ctests in the class) and method-level parameters (a map between Ctest method names and their parameters).

2.4.2 Configuration Connector

The configuration connector connects the ConfUT with Ctests, so that each test runs with the configuration under test rather than the default. The original Ctest scripts [76] implemented this by writing the ConfUT to a dedicated configuration file and changing the configuration object's initialization to read that file. However, the dedicated file was shared among all Ctests, so Ctests could not run concurrently with different configurations. This matters in practice because different Ctests may test different scenarios: some projects (e.g., HDFS) keep various configuration files (e.g., under `test/resources`) for different tests. Ctest4J's connector instead writes the ConfUT directly into the configuration object via

the configuration SET API, with no dedicated file: for each Ctest run it builds a map of each parameter and its value from the ConfUT, and the instrumented configuration API invokes Ctest4J's `connectProdConfig` to instantiate each parameter through the SET API. This design enables Ctest4J to support parallel execution of Ctests with distinct configurations.

2.4.3 Parameter Tracker

The parameter tracker monitors the usage of configuration parameters during Ctest runs. For a given Ctest class, the tracker maintains two levels of parameter usage list: (1) a class-level list of parameters used by all Ctest methods in the class (including `Before` and `After` methods), and (2) a method-level list of parameters used by each Ctest method. Invocations of the instrumented GET API call the tracker, which distinguishes whether execution is in the shared class-level setup/teardown (`BeforeClass`, `Before`, `After`, `AfterClass`) or in the method body, to record the parameter in the appropriate list.

2.4.4 Ctest Runner

The Ctest4J runner launches the Ctests using the ConfUT connector and the parameter tracker. For each test class, the runner first checks whether the class is annotated with `@CtestClass`; if not, it executes the test as a regular unit test but warns about the missing annotation. Otherwise, the runner executes the class as a Ctest and extracts the required configurations for the class and its methods from the annotations. For each Ctest class, the runner creates a new connector and tracker; this isolation ensures that the execution of one Ctest class does not interfere with others, which is essential for parallel execution. The runner fails a Ctest under any of the following conditions:

- *missing parameter usage*: the tracker identifies a required parameter that was not used during the run;

- *exceptions or errors*: the Ctest encounters an exception or error during execution (as for a regular unit test);
- *timeout*: the Ctest fails to finish within a specified timeout (as for a regular unit test).

2.4.5 Integration with Testing Frameworks

We integrated Ctest4J with JUnit4, JUnit5, and TestNG. For JUnit4, the Ctest runner is a custom JUnit4 runner that extends `BlockJUnit4ClassRunner` and implements the `CtestRunner` interface. Ctest4J integrates with JUnit5 as a JUnit5 extension and with TestNG as a TestNG listener. To integrate a new testing framework, one implements the `CtestRunner` interface, invoking the connector and tracker methods in the target framework's runner or listener.

2.5 Evaluation

We evaluate Ctest4J using 12 open-source Java projects, including all five projects used in prior Ctest work [41], [48], [57], [71] (Alluxio, HBase, HCommon, HDFS, ZooKeeper) and seven new projects (Figure 3). These projects use different testing frameworks (eight use JUnit4, three use JUnit5, and one uses TestNG) and different build systems (ten use Maven and two use Gradle).

2.5.1 Enabling Ctests

We enabled Ctests for the 12 mature, widely used Java projects using Ctest4J. The main effort is to understand each project's configuration API in order to instrument the GET and SET API (§2.3.2); this took on average about one hour per project, and we used Ctest4J's AspectJ utility to instrument the configuration API for all projects. No paper author is a developer on any evaluated project, so we expect developers familiar with their own project's configuration API to add the instrumentation even faster. With instrumentation

in place, we transform existing tests that use configuration parameters into Ctests following the original approach [41], using scripts that automatically add the Ctest4J annotations to the Ctest code.

2.5.2 Performance

We measure the Ctest running time using Ctest4J and compare it with the original Ctest scripts [76], and we measure Ctest4J's overhead by comparing the running time with and without Ctest4J (using the default configuration). We use the original scripts on only ten projects, because the scripts do not support Gradle, and we use the default parallelism configured in each project. Figure 3 shows that Ctest4J speeds up Ctest execution by up to 41.3× (3.4× on average) compared to the original Ctest scripts. The speedup comes mostly from parallel execution, which the original scripts do not support; for projects that configure high parallelism (e.g., Alluxio, Camel, and Flink) the speedup is large, while for projects that run tests sequentially the difference is smaller.

The overhead of Ctest4J is negligible. It comes mainly from configuration usage tracking and the additional checking logic that verifies whether all required parameters are used during execution. In Figure 3, the execution time of Ctest4J compared with the vanilla runner is up to 1.27× and 1.03× on average.

2.6 Discussion

The limitations of the current Ctest4J implementation come mainly from two assumptions about the configuration API, made by the ConfUT connector and the parameter tracker: (1) the configuration API can be abstracted as SET and GET APIs, and (2) the configuration parameters and their values can be represented in a key-value format. It can be difficult to use Ctest4J for projects whose configuration API does not meet these assumptions. Future work will support more configuration API designs found in real-world Java projects, extending the connector and tracker with more expressive abstractions.

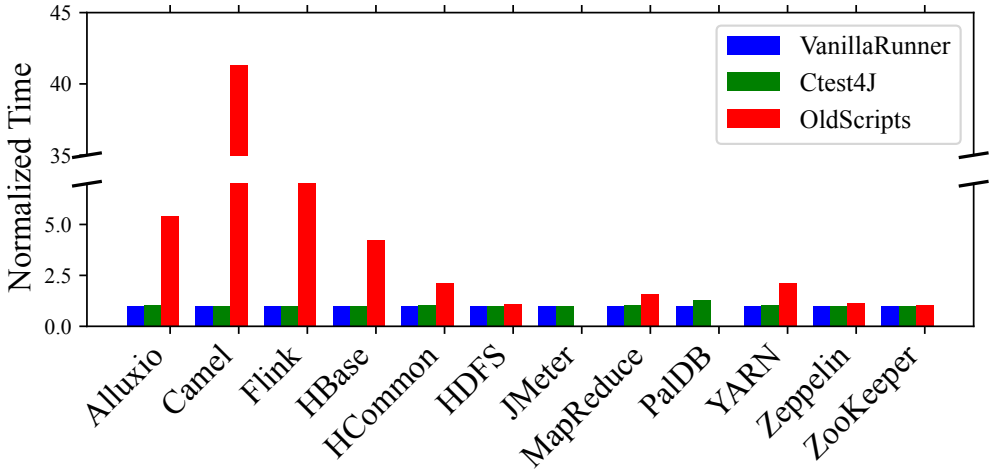


Figure 3: Execution time of running Ctests with Ctest4J, the Ctest scripts [76] (OldScripts), and the vanilla runner (VanillaRunner), normalized by VanillaRunner.

2.7 Summary

We presented Ctest4J, a practical configuration testing framework for Java projects. Ctest4J helps Java projects enable configuration testing with modest manual effort and low runtime overhead, and provides direct support for writing and maintaining Ctests. By lowering the barrier to adopting configuration testing in practice and broadening it beyond the handful of projects used in prior work, Ctest4J provides the foundation that Chapter 3 and Chapter 4 build on: Chapter 3 selects Ctests across code and configuration changes, and Chapter 4 transforms Ctests into fuzz drivers that search the configuration space for bugs. Ctest4J is available at `https://github.com/xlab-uiuc/ctest4j`.

Chapter 3 Unified Regression Test Selection for Code and Configuration Changes

3.1 Overview

Today's software failures have two dominating root causes: faults in program code (i.e., bugs) and errors in configuration files (i.e., misconfigurations) [13], [15], [16], [17], [18], [19]. Many software projects include some default configuration together with the code in the project repository, and modern continuous integration and deployment (CI/CD) [80], [81], [82], [83] widely use regression testing to check that project changes do not break previously working functionality. Traditional regression testing is mainly applied to (potentially changed) code under the (potentially changed) *default* configuration [29], [32], [33]. However, the deployed software uses *production* configurations that typically differ from the default configuration. Code changes are not tested under the production configurations, and changes in production configurations are not tested with the code, so many code changes pass the regression tests under the default configuration but lead to failures in production.

Recent research proposed *configuration testing* to detect erroneous changes to production configurations [41], [42], [57], [75]. A configuration test (Ctest) is a *parameterized* test [84], [85] whose input parameters are configuration parameters; it runs by instantiating the parameters with the values from production configurations (§3.2). A major limitation of prior work is that configuration testing addresses only configuration changes of the *same* code version: it assumes that the code does *not* change. Synergistically testing code and configuration changes requires *unified regression testing* (URT): URT tests code changes both under the default configuration and under production configurations, and tests configuration changes against the latest code version. In large software organizations, code-configuration co-changes are common, partially due to monolithic repositories that maintain both code and production configurations [10], [18], [86], [87].

URT could be very costly because it has to run a large number of tests under multiple configurations, including the default and several production configurations, and the cost of regression testing under only the default configuration is already considered high [62], [63], [64], [88], [89]. Regression test selection (RTS) [29], [30], [31], [90], [91] reduces this cost by running only the subset of tests *affected* by the project changes. Unfortunately, no existing RTS technique is tailored for URT: existing techniques are designed either for code changes only [29] or for configuration changes only [41], and none reasons about code and configuration changes collectively.

We introduce *unified regression test selection* (uRTS), the first RTS technique for URT. uRTS works with project changes on 1) code only, 2) configurations only, and 3) both code and configurations. It selects regular tests and Ctests with a unified algorithm that analyzes the code and configuration dependencies of each test across runs and across configurations, using a *two-dimensional comparison* to select fewer Ctests. uRTS provides the same safety guarantee as state-of-the-art RTS for code-only (Ekstazi [32]) and configuration-only changes (Ctest RTS [41]), and additionally guarantees safety for code-configuration co-changes. We implemented uRTS for Java and JUnit on top of Ekstazi [59] and Ctest [76], and evaluated it on hundreds of code revisions and dozens of configurations of five large projects (HCommon, HDFS, HBase, Alluxio, and ZooKeeper). Some of our experiments are among the *largest* RTS experiments performed on open-source projects: e.g., running all regression tests in HDFS for just one project revision takes over six hours on a powerful server. uRTS reduces the end-to-end testing time, on average, by 3.64× compared to executing all tests and by 1.87× compared to a competitive reference solution that directly extends RTS for URT.

This chapter makes the following contributions:

- **Concept:** We introduce Unified Regression Testing (URT) and motivate the need for RTS in URT.
- **Algorithm:** We develop uRTS, the first RTS for URT. uRTS works with project

changes on code, configurations, or both, and provides the same safety guarantee as existing RTS while being more effective and more broadly applicable.

- **Implementation:** We implement uRTS on the state-of-the-art RTS tools for regular tests and Ctests.
- **Evaluation:** We show the effectiveness of uRTS in reducing the cost of URT with large-scale experiments. Our artifacts are available at `https://github.com/xlab-uiuc/uRTS-ae`.

3.2 Background

3.2.1 Configuration Testing

Configuration testing is introduced in §2.2; we recall here only what uRTS builds on. A Ctest $\hat{t}(\hat{P})$ is parameterized by a set of configuration parameters $\hat{P}$, and running it instantiates each parameter in $\hat{P}$ with a concrete value from a production configuration [41]. Two properties matter for uRTS. First, when a Ctest is instantiated by the default configuration, it is equivalent to a regular test; this equivalence is the foundation for a *unified* solution that treats both Ctests and regular tests the same way. Second, Ctests are generated by transforming regular tests [41], so $\hat{P}$ is a subset of the parameters read by the original test t, which is what lets uRTS select Ctests by tracking the parameters each test reads. Figure 4 illustrates a Ctest (annotated by `@Ctest`) from prior work [41]. Unlike Sun et al. [41], who assumed that the code does *not* change, this chapter supports the general case where the code, as well as the configurations, can change. Configuration testing also differs from approaches that explore *multiple* configurations (e.g., configuration-aware testing, combinatorial testing, or misconfiguration-injection testing [8], [22], [92], [93], [94], [95], [96], [97]) in that it focuses only on the specific production configurations to be deployed.



Figure 4: A Ctest instantiated by a changed production configuration. The test fails as the code under the new production configuration throws an exception.

3.2.2 Regression Test Selection (RTS) and Ekstazi

Regression testing is widely used as projects evolve to test whether recent changes break existing functionality; it is important but also costly, as many tests are run for many changes. Regression test selection (RTS) [29], [30], [62], [63], [64], [88], [89] reduces this cost by selecting to run only a subset of tests, based on the most recent *code* changes. Traditional regression testing does *not* consider configuration changes. A typical RTS technique finds dependencies of each test on code parts and selects to run only the tests whose execution can reach the changed parts.

Ekstazi. Ekstazi [32] is an open-source RTS tool [59] for Java programs. Ekstazi determines test dependencies dynamically, at the level of files, including code `.class` files and *optionally* other files. When a test runs, Ekstazi monitors the execution to determine what files the test depends on. The tool default, which we call *Ekstazi*$^-$, tracks only `.class` files, although that option is *unsafe* [32]. We also evaluate an alternative, which we call *Ekstazi*$^+$, that tracks `.class` files and configuration files.

For the test in Figure 4, Ekstazi$^-$ finds that `testGetMasterInfoPort` depends on the test class `HttpServerTest`, the `HttpServer` class directly under test, and all other project and library classes that the execution reaches. Ekstazi$^+$ additionally finds that

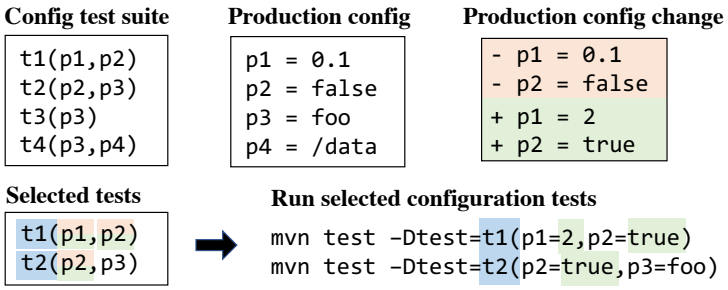


Figure 5: RTS of Ctests for a configuration change.

`testGetMasterInfoPort` depends on `hbase-default.xml` when run under the default configuration. When the project changes, Ekstazi selects to run the entire class `HttpServerTest` if *any* of the dependent files changes in *any* way. If all the classes (and other tracked files) remain the same, Ekstazi does not select a test because its behavior will be the same as before the changes.

Ekstazi operates at the level of classes, not methods: 1) it is safer for object-oriented code [32], and 2) it was shown to work faster *end-to-end* [32], [61]. A key aspect of RTS is to consider testing time *end-to-end*, from the moment developers initiate testing (e.g., via `mvn test`) until they get the result. This time, called $AE(C)$ [32], includes the *analysis* phase (A) that determines what tests to run, the *execution* phase (E) that executes the tests, and the *collection* phase (C) that collects the dependencies for the next revision. Note that in the very first run of a test, Ekstazi has no dependency information, so it always selects to run the test. An important point is that Ekstazi is *not* configuration aware: Ekstazi^{-} ignores configuration files, and Ekstazi^{+} tracks entire configuration files.

3.2.3 RTS for Configuration Tests

Prior work [41] developed an RTS algorithm for Ctests but under a restrictive assumption that the *code never changes*. As typical configuration changes only update a small number of configuration parameters [5], not all available Ctests need to run. A Ctest $\hat{t}(\hat{P})$ is selected to run for a given configuration change if at least one parameter in $\hat{P}$ is changed.

A configuration change passes if all *selected* Ctests pass, and it fails if any selected Ctest fails (Figure 5). Note that this prior RTS does *not* consider code changes.

3.3 Unified Regression Testing

The goal of *unified regression testing* (URT) is to test code changes and configuration changes *collectively* for production reliability. We define a project change as a "diff" D that updates the system code S, the default configuration C_{def}, production configurations $\mathbb{C}_{prod}$, the test suite T (for regular tests), or the test suite $\hat{T}$ (for Ctests):

$$D : (S', C'_{def}, \mathbb{C}'_{prod}, T', \hat{T}') \rightarrow (S, C_{def}, \mathbb{C}_{prod}, T, \hat{T})$$

D can be one commit or a bundle of several commits, depending on how the code/configuration repositories are maintained and how the project changes are deployed [5], [18], [64], [86], [98]. Table 1 lists the notation we use.

URT runs regression testing for the following combinations:

- Run regular tests T on the (changed) code S under the (changed) default configuration C_{def}; and
- Run Ctests $\hat{T}$ on the (changed) code S under every (changed) production configuration $C \in \mathbb{C}_{prod}$.

The former tests code under the default configuration; the latter tests code under production configurations. URT thus generalizes both traditional code-oriented regression testing [29] (§3.3.1) and configuration testing [41] (§3.3.2), and additionally handles diffs that co-change both code and configurations (§3.3.3). For now, we assume no RTS: we are unaware of any prior RTS tailored for URT.

3.3.1 Testing Code Changes

For a diff that only changes code or the default configuration, URT tests the changed code S under 1) the default configuration C_{def} and 2) every production configuration $C \in \mathbb{C}_{prod}$.

Table 1: Notation used in this chapter and their descriptions.

Notation	Description
S	The code (including test code) of the target system
C_{def}	The default configuration
$\mathbb{C}_{prod}$	The set of production configurations
$\hat{t}(\hat{P})$	A Ctest, where $\hat{P}$ is its input parameter set
$\hat{T}$	The Ctest suite of all Ctests $\hat{T} = \{\hat{t}(\hat{P})\}$
T	The regular test suite $T = \{t\}$, where each t only runs with C_{def}
D	A diff from $(S', C'_{def}, \mathbb{C}'_{prod}, T', \hat{T}')$ to new $(S, C_{def}, \mathbb{C}_{prod}, T, \hat{T})$

The former is equivalent to traditional regression testing: URT runs all the tests in the test suite T, and passes iff all the tests pass. However, traditional regression testing does not test code changes under production configurations. URT runs Ctests in $\hat{T}$ to check whether the code can be deployed under different production configurations. This is viable because each Ctest essentially tests the code under a specific configuration: a Ctest $\hat{t}(\hat{P})$ should pass for *any* correct values of parameters in $\hat{P}$. Without RTS, for a diff that only changes code or the default configuration, URT runs $|T| + |\hat{T}| \times |\mathbb{C}_{prod}|$ tests.

3.3.2 Testing Production Configuration Changes

For a diff that only changes production configurations, URT tests the code under the changed configurations $\mathbb{C}_{prod}$ by running Ctests $\hat{T}$ against the code $S = S'$ under each $C \in \mathbb{C}_{prod}$. URT runs every Ctest $\hat{t}(\hat{P})$ in $\hat{T}$ by instantiating $\hat{P}$ with values in each production configuration C, and passes iff every $\hat{t}(\hat{P})$ under every C passes. This is equivalent to the original configuration testing described in §3.2.1, except that all prior work [41], [42], [57] assumes there is only one distinct production configuration, while we allow multiple. Multiple production configurations are the norm in real-world deployments, e.g., production systems are constantly under staged deployments with multiple production configurations in place [75], [80]. Without RTS, for a diff that only changes production configurations, URT needs to run $|\hat{T}| \times |\mathbb{C}_{prod}|$ tests.

Table 2: Type of tests run for various types of changes.

Change Type	**Run T**	**Run $\hat{T}$**
$S' \neq S \vee C'_{def} \neq C_{def}$, $\mathbb{C}'_{prod} = \mathbb{C}_{prod}$	Prior work	**Our work**
$S' = S$, $C'_{def} = C_{def}$, $\mathbb{C}'_{prod} \neq \mathbb{C}_{prod}$	Need not run	Recent work [41] for $\lvert\mathbb{C}_{prod}\rvert = 1$ **Our extension** for $\lvert\mathbb{C}_{prod}\rvert > 1$
$S' \neq S \vee C'_{def} \neq C_{def}$, $\mathbb{C}'_{prod} \neq \mathbb{C}_{prod}$	**Our work**	**Our work**

3.3.3 Testing Code-Configuration Co-Changes

For a diff that co-changes both code and production configurations, URT runs all the tests in T against S under C_{def}, and all the Ctests in $\hat{T}$ against S under every $C \in \mathbb{C}_{prod}$. Without RTS, this runs $|T| + |\hat{T}| \times |\mathbb{C}_{prod}|$ tests. Table 2 summarizes the types of tests run for various types of changes.

3.4 The uRTS Algorithm

Unified regression testing (URT) is very expensive without effective RTS, and no existing RTS technique is directly tailored for URT due to not being able to reason about code and configuration co-changes collectively *and precisely*. We develop *unified regression test selection* (uRTS) to fill this gap. The goal of uRTS is to reduce the testing time by minimizing the number of tests (both regular tests from T and Ctests from $\hat{T}$) to run for a given diff that could change code, configurations, or both. We base uRTS on the following key observations:

- A diff typically changes a small piece of code or a small number of configuration values [5]. Thus, only a subset of T or $\hat{T}$ needs to be run for any diff.

- A production configuration typically changes only a small number of configuration values from the default values [7]. Assuming $\hat{T}$ has been run against C_{def}, a Ctest $\hat{t}(\hat{P}) \in \hat{T}$ need not be run for any production configuration that changes none of $p \in \hat{P}$ from the default.

- The n production configurations typically only differ in a small number of configuration values [7]. A Ctest $\hat{t}(\hat{P})$ need not be repeatedly run for production configurations that share the same values of $\hat{P}$.

For a diff $D : (S', C'_{def}, \mathbb{C}'_{prod}, T', \hat{T}') \rightarrow (S, C_{def}, \mathbb{C}_{prod}, T, \hat{T})$, uRTS employs a two-step test selection algorithm:

1. Select a subset of regular tests from T to check the new code S for the new default configuration C_{def}.

2. Select a subset of Ctests from $\hat{T}$ to check the new code S for each production configuration in $\mathbb{C}_{prod}$.

3.4.1 Step 1: Selecting Regular Tests

If the target diff D changes the code (i.e., $S' \neq S$), uRTS first selects regular tests from T to test the code changes by applying traditional RTS, i.e., a test t is selected as long as D changes some code exercised by t. Our implementation uses Ekstazi, which checks for each $t \in T$ whether D changes any code file on which t depends. If the target diff D changes the default configuration (i.e., $C'_{def} \neq C_{def}$), uRTS further selects from the remaining $t \in T$ every test that exercises at least one configuration parameter whose values differ in C'_{def} and C_{def}. Finally, uRTS runs every selected t against S under C_{def}. Recall that running a Ctest $\hat{t}(\hat{P})$ against S under C_{def} is equivalent to running a regular test t against S.

3.4.2 Step 2: Selecting Configuration Tests

In this step, uRTS checks the new code S under every new production configuration $C \in \mathbb{C}_{prod}$. uRTS only selects Ctests from $\hat{T}$, because not all tests in T can test production configurations that differ from the default. One could use file-level RTS, like Ekstazi^{+}, to track the configuration at the level of the *entire file(s)*. However, this is inefficient: a configuration file could include hundreds of parameters and be several KBs or even MBs [5],

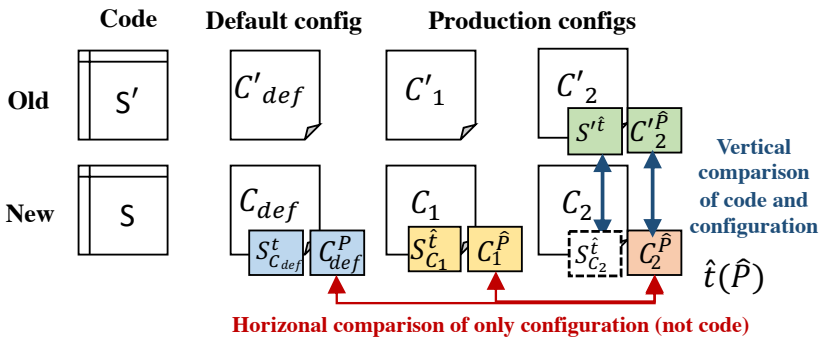


Figure 6: Illustration of Step 2: $\hat{t}(\hat{P})$ is *not* selected for S and C_2 if either the vertical or horizontal comparison matches. As an optimization, the horizontal comparison does *not* compare code dependencies.

so even if a diff only changes one configuration value, all the Ctests that read the file need to be run. uRTS therefore works at the granularity of *configuration parameters* instead of entire configuration files.

For each $C \in \mathbb{C}_{prod}$, uRTS selects a $\hat{t}(\hat{P}) \in \hat{T}$ iff no prior test run of $\hat{t}(\hat{P})$ exercises the same code under the same values of all configuration parameters in $\hat{P}$. In other words, uRTS unselects a Ctest $\hat{t}(\hat{P})$ as long as it has been run against the same code dependencies and the same values of all parameters from $\hat{P}$ in either 1) Step 1, where a regular test t is equivalent to a Ctest $\hat{t}$ instantiated with C_{def}; or 2) the last test round, on $(S', C'_{def}, \mathbb{C}'_{prod})$. Let $\mathbb{C}^{<C}_{prod}$ be the set of production configurations that run before C. The analysis uses a two-dimensional comparison, illustrated in Figure 6:

- **Horizontal comparison:** if the value of every parameter $p \in \hat{P}$ is the same in C as in another $C^* \in \{C_{def}\} \cup \mathbb{C}^{<C}_{prod}$, then $\hat{t}(\hat{P})$ is *not* selected to run for C, because running $\hat{t}$ against S under C is equivalent to a prior test run. This case does *not* compare code dependencies; it is an intentional optimization, as the code is the same.

- **Vertical comparison:** if the value of every parameter $p \in \hat{P}$ is the same in C as in C' *and* every code dependency of $\hat{t}$ under C' has the same content in S' as in S, then $\hat{t}(\hat{P})$ is *not* selected to run for C, because running $\hat{t}$ against S under C is equivalent to running $\hat{t}$ against S' under C', which was done in the last test round.

Correctness. A critical property of RTS is *safety* [29], [30]: selecting all tests whose results could change due to the project changes. For diffs that only change code or the default configuration, uRTS provides the same safety as traditional file-based RTS such as Ekstazi [32] in the Ekstazi$^+$ mode, which implicitly considers the default configuration. uRTS tracks configuration dependencies at the granularity of configuration *parameters* instead of configuration *files*, thus providing more efficient end-to-end testing. uRTS collects configuration dependencies by making an over-approximate estimate, which assumes that each configuration parameter read by a test influences the execution of the test. This over-approximation simplifies the implementation and ensures that the analysis is comprehensive. For diffs that only change production configurations, uRTS provides the same level of safety as the RTS for configuration testing [41], and is more efficient when multiple production configurations exist. Moreover, uRTS supports the most general case where both code and configurations change. For precision, uRTS selects a regular test t iff the test result could change due to the changed code or C_{def}, and selects a Ctest $\hat{t}(\hat{P})$ iff the code and the configuration parameters exercised by $\hat{t}$ were not already run in the current test round (horizontal comparison) or the last test round (vertical comparison). In summary, $\hat{t}(\hat{P})$ runs only *once* for the same code and the same set of configuration values of $\hat{P}$.

3.5 Implementation

We implemented uRTS for Java and JUnit on top of Ekstazi [32] and Ctest [41]. We use Ekstazi to track the code executed by each test, be it a regular or Ctest, and integrate Ctest into Ekstazi to support running Ctests. We extend Ekstazi's dependency collection and analysis to track configuration parameters and their values during each test run, and encode them in the *dependency file* format that Ekstazi stores for each test [59]. Figure 7 shows the workflow. uRTS first selects tests (§3.4) during the analysis phase (§3.5.2). To execute regular tests, uRTS uses the existing Ekstazi integration with JUnit and Maven

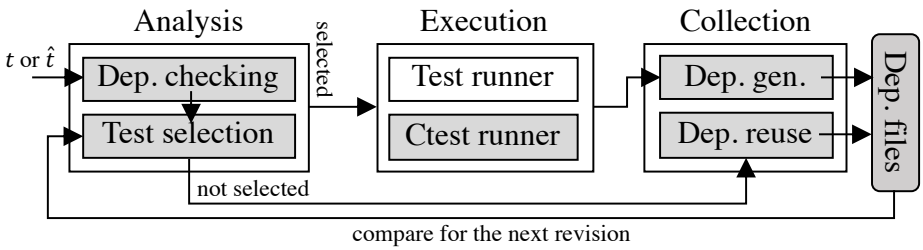


Figure 7: The uRTS implementation uses Ekstazi's integration with JUnit; we added or enhanced the grayed components.

Surefire. For each test that is not selected, uRTS reuses the test's dependency file from an earlier equivalent run; for each test that is selected, uRTS runs the test and generates the new dependency file (§3.5.3).

3.5.1 Dependencies

uRTS maintains two types of dependencies for each test under each configuration $C \in \{C_{def}\} \cup \mathbb{C}_{prod}$:

- **Code dependency:** the code files that the test depends on, in the Ekstazi form of ⟨`URI`, `checksum`⟩ pairs. The same test could have different code dependencies when run under different configurations.
- **Configuration dependency:** the names and values of configuration parameters read by the test, in the form of ⟨`parameter`, `value`⟩ pairs. For each Ctest $\hat{t}(\hat{P})$, its configuration dependencies only include the parameters in $\hat{P}$.

uRTS maintains a dependency file for each pair of a test and a configuration. Each dependency file is 1) (re)generated during the test run if the test is selected, or 2) reused from an equivalent run if the test is not selected.

3.5.2 Analysis Phase

The analysis phase analyzes the latest code and configuration dependencies for each test and decides whether to select the test to execute.

Table 3: Two examples of sophisticated configuration in the Hadoop project, which make it difficult to analyze configuration changes by simply parsing a configuration file.

Type	Parameter	Value
Value dep.	**hadoop.tmp.dir** io.seqfile.local.dir	/tmp/hadoop-${**user.name**} ${**hadoop.tmp.dir**}/io/local
Complex dep.	dfs.ha.namenodes.**CID** dfs.namenode.rpc-address.**CID**.**NN1** dfs.namenode.rpc-address.**CID**.**NN2**	**NN1**,**NN2** machine1.example.com:8020 machine2.example.com:8020

Analyzing dependency changes. Like Ekstazi, uRTS checks whether a file dependency changes between two code versions by comparing the checksums of the file content. To analyze configuration dependencies, uRTS needs to obtain the value of each configuration parameter in the corresponding configuration $C \in \{C_{def}\} \cup \mathbb{C}_{prod}$ and compare it with the value recorded in a dependency file. A seemingly easy solution is to parse the configuration files based on their format (e.g., INI or XML). However, our experience implementing uRTS for real-world projects shows that configurations could have complex representations, making it difficult to understand how they are interpreted by the project. Table 3 presents examples from Hadoop involving configuration variables and complex dependencies based on configuration values; it is hard to duplicate such logic for each project. Rather than reverse-engineering sophisticated configuration logic, our insight is to obtain configuration parameter values by reusing the project's own configuration APIs for reading parameter values.[1] uRTS instruments the same configuration APIs for the collection phase (§3.5.3), which ensures the consistency of the analysis.

Test selection. uRTS implements the test selection algorithm (§3.4) based on dependency analysis. If there is no dependency file recorded, the test is selected to run. In Step 1, for each test t against S under C_{def}, uRTS checks whether the configuration dependen-

[1]Modern software projects use uniform APIs for reading configurations, which is the basis for recent configuration analysis techniques [22], [40], [41], [57], [78], [99], [100], [101], [102], [103], [104]. For example, in many Java projects, the `Get` APIs can be abstracted as a method `String get(String parameter)`, often declared in wrapper classes on top of `java.util.Properties`. Ctest [41] instruments these `Get` APIs to generate Ctests from regular tests.

cies of t change; if so, t is selected. Otherwise, uRTS checks whether any code dependency changes, selecting t if so. In Step 2, for each Ctest $\hat{t}(\hat{P})$ under a production configuration $C \in \mathbb{C}_{prod}$, uRTS first "horizontally" checks whether any value of $\hat{t}$'s input parameters $\hat{P}$ in C differs from the corresponding value in the configuration dependency of $\hat{t}$ previously executed under C_{def} and other production configurations. (Note that $\hat{t}$ could read more parameters than those in $\hat{P}$, but the values of the parameters not in $\hat{P}$ are the same as in the default configuration.) If there is no such difference, $\hat{t}$ is not selected. Otherwise, uRTS further "vertically" compares the current values of $\hat{P}$ in C against the prior configuration dependency of C', and the current code dependency against the prior code dependency of $\hat{t}$, selecting $\hat{t}$ iff either changes.

3.5.3 Collection Phase

If a test under $C \in \{C_{def}\} \cup \mathbb{C}_{prod}$ is not selected, uRTS has found an equivalent test run with the same code and configuration dependencies during horizontal/vertical comparison, and copies the corresponding dependency files to update current dependencies. For tests that are selected, uRTS executes the test and (re)generates the code and configuration dependencies. uRTS uses Ekstazi to track code dependencies as the `.class` files on which the test execution depends, and collects configuration dependencies by applying the techniques of Ctest [41] to instrument the configuration APIs and monitor the configuration parameters read by each test during execution. Figure 8 shows the instrumentation for the `Get` API of the Hadoop project, and Figure 9 shows the method we added to read all the configuration parameter values of a given file.

3.5.4 Optimizations

We apply two optimizations to reduce the analysis time. First, the horizontal comparison only analyzes configuration dependencies, not code dependencies, because the code revision does not change when testing C_{def} and any $C \in \mathbb{C}_{prod}$ (see Figure 6); if the configuration

```
  public String get(String name) {
+     String urtParam = name;
      String[] names = handleDeprecation(deprecationContext.get(), name);
      String result = null;
      for(String n : names) {
+         urtParam = n;
          result = substituteVars(getProperty(n));
      }
+     ConfigListener.record(urtParam, result);
      return result;
  } /* .../hadoop/conf/Configuration.java */
```

Figure 8: Instrumentation for the `Get` API in Hadoop. The `get` method is the lowest-level API used by high-level APIs (e.g., `getInt` and `getBool`); `handleDeprecation` replaces deprecated names.

```
public static void recordConfig(String confFile) {
    Configuration conf = new Configuration(confFile);
    for(String parameter : conf.getAllKeys()) {
        String value = conf.get(parameter);
        ConfigListener.record(parameter, value);
    }
}
```

Figure 9: The method we added for the Hadoop project to read all the configuration parameter values of a given file. Hadoop code uses the `Get` API to read configuration values.

dependencies are the same, the test executes the same, and if they differ, the test is selected to rerun anyway. This avoids the expensive recomputation of checksums of all dependent files. Second, because configuration dependency comparison is computationally much cheaper than code dependency comparison (string comparison without checksum calculation), our implementation always analyzes configuration dependencies before code dependencies, saving the overhead of comparing code dependencies when configuration dependencies already differ.

3.6 Evaluation

Our evaluation aims to answer the following research questions:

- **RQ1:** How effective is uRTS at reducing the cost of URT?

- **RQ2:** What is the overhead of uRTS to support URT?
- **RQ3:** How much does uRTS save for configuration changes?
- **RQ4:** Are both dimensions of the two-dimensional comparison needed?

3.6.1 Experimental Setup

Evaluating uRTS on open-source projects is challenging, because most projects do not offer a DevOps-based environment with both code and production configuration changes. RTS research often uses commits from the revision history of open-source projects for code (and default configuration) changes [32], [33], [105], but it is non-trivial to collect real-world production configurations and their revision histories, which are typically proprietary. We create an evaluation set on top of the Ctest dataset [76], which contains 100 deployed configurations collected from public Docker images on DockerHub [106] for five large, widely used open-source projects. We treat those deployed configurations as production configurations and use multiple deployed configurations as different, evolving revisions of one production configuration. We use the most recent compilable commits as our evaluation project changes. Table 4 shows the evaluated projects and the statistics of code commits, configurations, and test suites.

Evaluated projects and commits. We use the same set of open-source projects as recent work on configuration testing [41], [57]: HCommon, HDFS, HBase, Alluxio, and ZooKeeper. As all these projects use JUnit as their testing framework,[2] we integrated Ekstazi and uRTS into their build and test systems. All projects use Apache Maven for building and the Maven Surefire plugin for running tests. For each project, our evaluation uses 50 recent commits: the newest released version as of 2022 as the last commit and 49 prior compilable commits that each modify relevant code or configuration. Among these

[2]The Ekstazi tool that we obtained [59] supported only JUnit 4 (and 3); we extended it to support JUnit 5 to evaluate ZooKeeper.

Table 4: Projects, commits, configuration files, and tests (regular and configuration) used in the evaluation.

Project	Module	LOC	# Rev.	# Conf. files	# Rev. on $\mathbb{C}_{prod}$	# Test Classes [avg] Regular	Config
HCommon	hadoop-common	256K	50	20	18	510	259
HDFS	hadoop-hdfs	371K	50	20	18	751	751
HBase	hbase-server	427K	50	20	17	295	158
Alluxio	core	154K	50	20	17	247	118
ZooKeeper	zookeeper-server	101K	50	20	19	299	187

commits, some in HCommon, HDFS, and Alluxio change the default configuration included with the codebase; no commit in HBase or ZooKeeper changes the default configuration.

Production configurations. We use real-world configurations of each project from public Docker images, collected in the Ctest dataset [76], as the production configurations. We treat each project as having two production configurations in the evolution, to represent the simplest case of multiple configurations; in general, the more production configurations a system deploys, the *more* benefits uRTS brings. As Docker images do not provide a revision history of the deployed configurations, we divide all the configurations of each project into two groups to represent two production configurations and treat each group of n configurations as n revisions of one configuration. We associate each configuration change with randomly chosen project commits; the number of commits with at least one production configuration change is 18 for HCommon, 18 for HDFS, 17 for HBase, 17 for Alluxio, and 19 for ZooKeeper.

Configuration tests. The evaluated projects do not come with explicit Ctest suites. We use Ctest [76] to transform existing tests into Ctests by parameterizing them with configuration parameters (§3.2.1). Because the Ctests provided by the Ctest dataset were generated for only one (older) version of each project, we regenerate all the Ctests for each commit in our evaluation following Sun et al. [41].

Hardware and metrics. All experiments run on Azure VMs with dual-core CPUs and 14 GB of RAM [107], Ubuntu 20.04.2, and Java 64-bit 1.8.0, spending 2,000+ hours of machine time in total. We compare uRTS with the ReTestAll baseline that conducts URT without RTS, a common baseline in RTS research [30]. With two production configurations, ReTestAll runs $|T| + 2 \times |\hat{T}|$ tests for every commit. To limit machine time, we do not measure ReTestAll time for all 50 commits but run only the first and last commit in the range and use their average for all commits; the difference between the first- and last-commit time is, on average, just 1.0%, and for the number of test classes (which we extract for every commit) just 0.6% between the minimum and maximum. We also compare with two reference solutions, Ekstazi$^+$ and Ekstazi$^-$ (§3.2.2): Ekstazi$^+$ integrates Ctest with Ekstazi and tracks each Ctest's file dependencies (code and configuration files), while Ekstazi$^-$ is the Ekstazi default that ignores configuration files and only concerns code changes. We measure two metrics: 1) the end-to-end testing time (the time to execute the build command developers use, `mvn test`), and 2) the number of selected test classes. We select tests at the granularity of test *classes* (not test methods), because selecting classes is safer [32] and was shown to outperform selecting methods [32], [60], [61].

3.6.2 RQ1: Effectiveness

End-to-end testing time is the key metric for the effectiveness of RTS in reducing testing costs; as an additional metric, we use the number of selected test classes. Table 5 shows the RTS results for uRTS compared with ReTestAll and Ekstazi$^+$ (lower numbers are better). We compute the ratio for each commit, average the ratios via geometric mean over all commits, and then average across all projects, obtaining an *unweighted* average that equally treats all projects.

On average, uRTS takes 53.35% of the testing time of Ekstazi$^+$ and 27.51% of the testing time of ReTestAll. Stated differently, *uRTS reduces the end-to-end testing time by 1.87× compared with Ekstazi$^+$ and 3.64× compared with ReTestAll.* In terms of the number of test

Table 5: Test run results of ReTestAll, Ekstazi$^+$, uRTS, and Ekstazi$^-$, averaged over all commits. T_{UR}% and T_{U+}% denote the testing-time reduction of uRTS over ReTestAll and Ekstazi$^+$, respectively.

Project	**ReTestAll**		**Ekstazi$^+$**		**uRTS**		**Ekstazi$^-$**		T_{UR}%	T_{U+}%
	time [s]	#cls	time [s]	#cls	time [s]	#cls	time [s]	#cls		
HCommon	4467.06	1030.50	1303.06	133.40	542.01	51.12	220.67	14.82	12.13%	41.60%
HDFS	65283.01	2049.00	45346.87	759.27	30614.97	478.44	13818.32	199.58	46.90%	67.51%
HBase	2844.60	611.00	1303.90	105.76	652.03	45.27	381.32	31.38	22.92%	50.01%
Alluxio	1831.13	485.50	1401.36	212.22	998.18	97.53	529.78	73.65	54.51%	71.23%
ZooKeeper	3168.50	677.00	1624.30	208.54	701.71	69.21	458.92	46.59	22.15%	43.20%
Σ/avg	5451.09	842.33	2810.57	216.35	1499.39	94.34	776.72	50.19	**27.51%**	**53.35%**

classes, uRTS only selects 43.61% and 11.20% of the test classes compared with Ekstazi$^+$ and ReTestAll, respectively. It is not uncommon for RTS to select a smaller percentage of tests than the percentage of overall time saved [32], [33], for two reasons: 1) the overall time includes the building overhead that is the same with and without RTS; and 2) the test execution time is not proportional to the number of selected tests, because the selected tests are typically longer-running and larger, with more dependencies. For example, for Alluxio, the test class `FileSystemFactoryTest` runs significantly longer than others and is selected often (104 times by uRTS and 135 times by Ekstazi$^+$ out of 150 cases), which explains the gap between 71.23% and 45.96%. For HDFS, both ratios are higher due to a configuration parameter, `dfs.namenode.datanode.registration.ip-hostname-check`, on which 584 test classes depend, so all 584 are selected when it changes.

The Ekstazi$^+$ results show that even a coarse-grained RTS technique can effectively reduce the testing time to 51.56%, on average, and the number of test classes to 25.69%, compared with ReTestAll. The uRTS results show the further effectiveness of configuration-aware RTS for URT: tracking configuration dependencies at the parameter granularity can further reduce the testing time to 53.35% and the number of test classes to 43.61%, compared with the file granularity (Ekstazi$^+$). In brief, *Ekstazi$^+$ almost halves the time of ReTestAll, and uRTS further almost halves the time of Ekstazi$^+$*; with uRTS, URT is substantially cheaper.

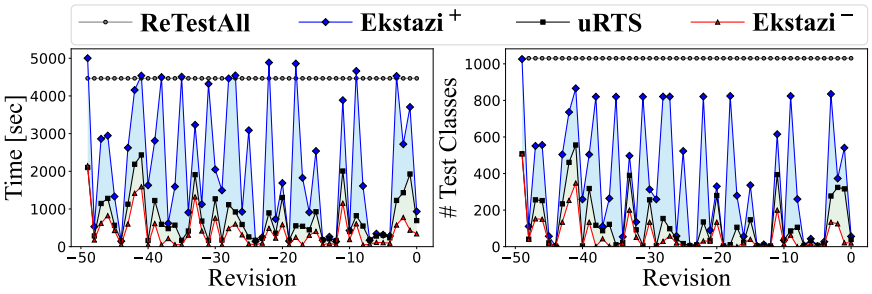
Figure 10 visualizes the RTS results per commit across the evaluated projects. Ekstazi$^+$ has an even higher testing cost than ReTestAll in the initial commits, because Ekstazi$^+$ needs to run the entire test suites (both T and $\hat{T}$) and collect test dependencies; collecting dependencies is also why the testing time of Ekstazi$^+$ is sometimes higher than that of ReTestAll for later commits. In contrast, uRTS reduces the testing time effectively compared with Ekstazi$^+$ and ReTestAll *from the very first commit*, by avoiding redundant test runs via the "horizontal" selection across the configurations.

3.6.3 RQ2: Overhead

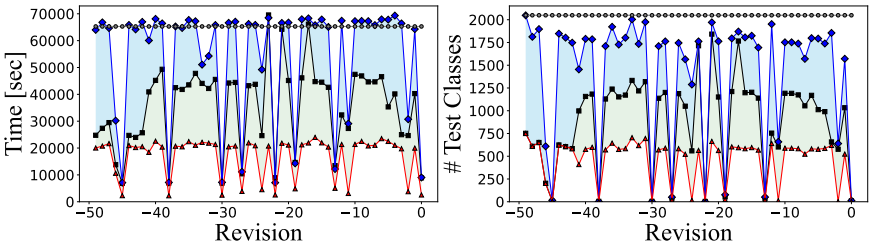
To understand the overhead of URT over *unsafe* regression testing that ignores configuration changes, we compare uRTS against Ekstazi$^-$. Note that Ekstazi$^-$ only tests code changes under the default configuration, while uRTS tests three different configurations (one default and two production). On average, uRTS for three configurations takes 1.93× of the testing time and 1.88× of the test classes over Ekstazi$^-$ (per-project Ekstazi$^-$ times and class counts are in the last two Ekstazi$^-$ columns of Table 5). Testing three configurations does not even double the testing costs: with uRTS, the cost of URT is close to traditional regression testing.

3.6.4 RQ3: Focus on Configuration Changes

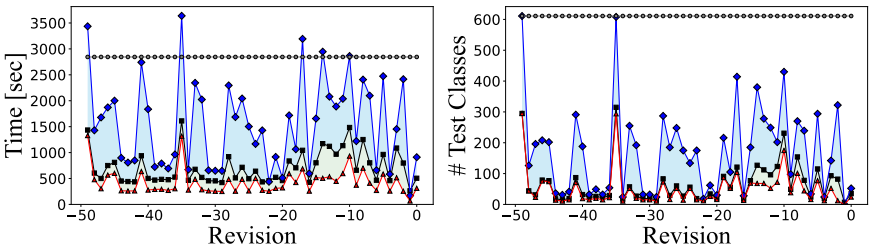
uRTS is expected to reduce more testing cost over Ekstazi$^+$ when configuration changes are more frequent; Meta, for example, reported that configuration changes are even more frequent than source-code changes [5]. We focus on the subset of commits with at least one production configuration change and break the testing time down for each of the two production configurations separately. uRTS takes 21.85% and 14.13% of the time of Ekstazi$^+$ for production configurations C_1 and C_2, respectively, roughly 1/4 for C_1 and 1/7 for C_2. Considering both, uRTS takes only 18.50% of the time of Ekstazi$^+$, substantially lower than the 53.35% overall average, as expected. Table 6 shows the per-



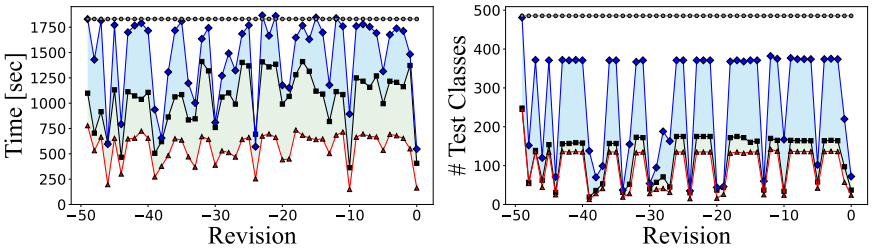
(a) HCommon (time, left; # tests, right)



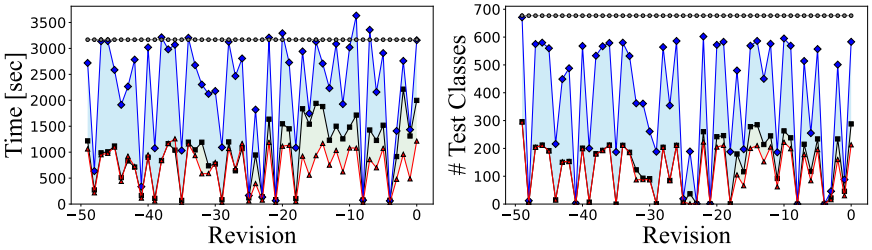
(b) HDFS (time, left; # tests, right)



(c) HBase (time, left; # tests, right)



(d) Alluxio (time, left; # tests, right)



(e) ZooKeeper (time, left; # tests, right)

Figure 10: End-to-end testing time in seconds (left of each pair) and number of selected test classes (right of each pair) per commit for all five projects.

Table 6: Testing time for Ekstazi$^+$ and uRTS on commits with a production configuration change, broken down by the two production configurations (C_1, C_2). T_{U+}% is the testing time of uRTS as a fraction of Ekstazi$^+$.

Project	**Ekstazi$^+$** [s]		**uRTS** [s]		T_{U+}%		
	C_1	C_2	C_1	C_2	C_1	C_2	C_1+C_2
HCommon	1555.20	1471.42	260.01	255.73	16.72%	17.38%	17.04%
HDFS	22215.74	19965.39	13748.08	3978.08	61.88%	19.92%	42.02%
HBase	1154.25	1127.30	183.58	88.16	15.90%	7.82%	11.91%
Alluxio	555.61	535.49	276.40	206.85	49.75%	38.63%	44.29%
ZooKeeper	1011.87	985.09	61.51	52.98	6.08%	5.38%	5.73%
Σ/avg	1862.64	1771.97	406.92	250.32	21.85%	14.13%	**18.50%**

project breakdown.

3.6.5 RQ4: Effectiveness of the Two-Dimensional Comparison

For each Ctest, uRTS makes the selection decision by two-dimensional comparison (§3.4.2): it "horizontally" compares the dependencies with an earlier test run of the same code version but a different configuration, while "vertically" comparing with the previous code version but (potentially) the same configuration. We analyzed uRTS with only horizontal and only vertical comparison; the results show that both dimensions are necessary: neither subsumes the other. On average, uRTS selects 330 test classes by vertical comparison and 164 by horizontal comparison, but *only* 108 by the two-dimensional comparison.

3.6.6 Threats to Validity

The threats to external validity mainly lie in the evaluated projects and datasets. To reduce them, we use recent releases of real-world projects and deployed configuration files from the Ctest dataset [41]; however, we synthesized configuration changes using different configuration files from DockerHub as different versions, and future work should consider more diverse datasets. The threats to internal validity mainly lie in potential bugs in our implementations and experimental scripts; we extensively reviewed the code and checked the results. The threats to construct validity mainly lie in the metrics; we consider not

only the number of selected tests but chiefly the end-to-end testing time, which is the most relevant metric for RTS.

3.7 Discussion

Generality. Our uRTS implementation builds on Ekstazi with dynamic RTS, chosen because it is open source [59], robust, and used in several studies [32], [61], [108], [109], [110]. However, the key principles of uRTS, namely (1) tracking configuration dependencies at the level of configuration parameters rather than files, (2) performing both horizontal and vertical comparisons, and (3) comparing parameter values across multiple production configurations rather than just against the default, can be applied to other dynamic RTS, whether at a finer granularity of code dependencies (e.g., method) or coarser (e.g., modules). We view tracking configuration parameters rather than files as an application of the general idea of tracking dependencies at a finer rather than coarser granularity whenever the somewhat higher cost of collection and analysis provides an even higher saving by unselecting tests. Nanda et al. [111] discuss tracking configuration files, like Ekstazi$^+$, but not the granularity of tracking configuration parameters.

Selection granularity. Our evaluation uses the test class as the selection granularity. uRTS supports other granularities such as the test *method*: both Ekstazi and Ctest can select tests at the method granularity. Although prior studies [32], [60], [61] found that RTS based on class dependencies was more effective, recent work [61] shows that RTS can benefit from a hybrid approach using different granularities, which we leave as future work.

Nondeterministic tests. One concern, especially for large codebases, is that tests may be nondeterministic and have a different outcome even for the same code and configuration dependencies. An RTS technique is still safe if it unselects a test when its executions *observed* in prior runs do not change, even if the test may have other executions that could change. If developers want to check more executions of a test, they need to run the test

multiple times.

3.8 Summary

We presented uRTS, a test selection technique for unified regression testing (URT) of both code and configuration changes. uRTS selects a subset of tests to run and provides the same safety guarantees as traditional RTS. It reduces the end-to-end testing time by 1.87×, on average, compared to Ekstazi^{+}, and by 3.64× compared to executing all tests. uRTS is a step toward making URT practical and widely adopted. Our artifacts are available at `https://github.com/xlab-uiuc/uRTS-ae`.

Chapter 4 ConfFuzz: Parameter-Aware Greybox Configuration Fuzzing for Cloud Systems

4.1 Overview

Modern cloud systems are highly configurable, with hundreds to thousands of configuration parameters spanning diverse types, semantics, and constraints [5], [6], [7], [8]. The enormous configuration space makes it fundamentally difficult to test and verify software correctness [41], [112], and configurations have been a major source of cloud system failures. Faulty configurations were reported as the second-largest cause of service disruptions in a major Google production service [25], and at Meta, 16% of service-level incidents (including significant outages) were introduced by configuration changes [5], [16]. Mature systems often have abundant developer-written unit tests that achieve high code coverage [41], but those tests usually exercise only a small set of handpicked configurations, most often the default configuration, so systems remain vulnerable to failures triggered by untested configuration changes.

Recent research on automated configuration testing follows two directions. Configuration testing [41], [42] identifies the parameters exercised by each unit test and generates a limited set of configurations for testing, but does not use execution feedback to guide subsequent generation (Chapter 2). Configuration fuzzing [48], [113] instead mutates configuration values automatically. ECFuzz [48] targets large cloud systems but remains blackbox: it does not use execution feedback and relies on a small number of manually written system tests (2.6 per project on average), which limits both test execution speed and the breadth of code that testing can reach [23], [40]. Greybox fuzzing has recently been applied to configuration testing of command-line applications [47], showing that coverage can guide configuration fuzzing toward unexplored behavior. However, the configuration spaces of such command-line applications are orders of magnitude smaller than those of typical cloud systems, with only 1 to 14 options, 92.6% of them boolean [47]. Reducing the

search space and guiding fuzzing to efficiently reveal unexplored system behavior therefore remains a challenge for greybox configuration fuzzing of cloud systems.

This chapter applies coverage-based greybox fuzzing to configuration testing of cloud systems, addressing this challenge by leveraging the configuration parameters *exercised at runtime* to guide mutation and fuzzing-energy allocation. We make three design decisions:

- To provide the high test execution speed and code coverage that greybox fuzzing demands, we use existing developer-written unit tests as fuzz drivers. Such tests are abundant in mature projects and execute far faster than the system tests used by prior cloud-system configuration fuzzers, which spend seconds to minutes initializing clusters, starting services, and distributing workloads. We transform unit tests into fuzz drivers by instrumenting the unified configuration APIs (§4.2.3) that mature cloud systems employ, building on ideas from Ctest [41] (Chapter 2) and UTopia [65].

- Based on the observation that a unit test typically uses only a small fraction of all parameters (15.14% on average) and that mutating one parameter can steer execution to a path that reads additional parameters, we design *parameter-aware mutation*, which dynamically tracks the parameters exercised during each fuzzing round and restricts mutation to only those parameters.

- Because different unit tests can exercise dramatically different numbers of parameters, ConfFuzz introduces *parameter-aware fuzzing-energy allocation*, which allocates energy based on both code coverage and configuration-parameter coverage, giving more effort to seeds that explore more code and more of the configuration space.

We also provide debugging support by applying delta debugging [114] to locate the root-cause parameters of test failures.

We implement these ideas in ConfFuzz and evaluate it on nine large cloud systems with hundreds to thousands of configuration parameters. ConfFuzz advances configuration fuzzing to cloud-system scale by integrating these three decisions: unit tests as fuzz drivers,

reduction of the configuration space to the parameters each seed dynamically exercises, and exercised parameters as feedback for seed selection and energy allocation. In contrast, ECFuzz [48], the prior cloud-system configuration fuzzer, is blackbox with no feedback, and ConfigFuzz [47], the prior greybox configuration fuzzer for command-line options, does not scale to the large configuration space of cloud systems. We compare ConfFuzz against a greybox baseline that reimplements the core strategy of ConfigFuzz for Java-based cloud systems. Across all campaigns, ConfFuzz and its variants expose 125 configuration-related bugs, including 14 fixed bugs and 12 additional confirmed bugs, and ConfFuzz covers 13.3%–14.2% more branches than its parameter-aware ablations while raising the median number of exercised parameters per execution from 22 to 35. This chapter makes the following contributions:

- A greybox configuration fuzzing design for cloud systems with unified configuration APIs, instantiated with existing unit tests as fuzz drivers;
- Dynamic parameter-aware fuzzing with per-execution tracking, mutation over exercised parameters, and fuzzing-energy allocation guided by code coverage and parameter usage;
- An evaluation of ConfFuzz on nine widely used cloud systems, including ablation studies and 125 detected bugs.

4.2 Background and Motivation

ConfFuzz reuses *configuration tests* (Ctests, Chapter 2) as the starting point for fuzzing: a unit test that exercises configuration parameters is a Ctest, and ConfFuzz fuzzes the configuration it reads. We refer the reader to §2.2 for the Ctest primitive; this section motivates *fuzzing* Ctests and the guidance problem it raises.

4.2.1 Blackbox and Greybox Configuration Fuzzing

Prior work has explored fault injection and fuzzing for configurable systems through both configuration-file and command-line interfaces [22], [47], [48], [75], [112], [113], [115]. Blackbox fuzzing mutates configurations and detects bugs through coarse outcomes such as crashes and failures; greybox fuzzing additionally uses execution feedback, such as code coverage, to guide mutations. ECFuzz [48] is representative of blackbox configuration fuzzing for large cloud systems: it targets systems with large configuration spaces but does not use execution feedback, so its search remains unguided by program behavior. When a system exposes hundreds to thousands of parameters but any single execution exercises only a small fraction of them, unguided search is expensive: the fuzzer may spend many executions mutating parameters that the current test never reads, making little progress toward new paths. The problem is more severe in complex cloud systems, where end-to-end testing often relies on a small number of system tests or validators and may not reach important modules such as recovery and fault-tolerance mechanisms; prior work has shown that these modules are difficult to reach [23], [40], [53], [116] and prone to configuration-related defects [40]. ConfigFuzz [47] is a greybox configuration fuzzer that uses code coverage to guide fuzzing of command-line options, but in its evaluation the studied programs expose only 1 to 14 options, 25 of 27 of them boolean. In such a small option space, the lack of parameter-level guidance is less restrictive. Cloud systems present a different challenge: code coverage alone does not reveal which configuration parameters were exercised in the current execution, and therefore does not indicate which part of the configuration space to mutate next.

4.2.2 Unit Tests as Fuzz Drivers

Greybox fuzzing needs fuzz drivers that can be executed repeatedly and at low cost, and existing developer-written unit tests satisfy this requirement well. Ctest [41] showed that many unit tests exercise configuration parameters and therefore have strong potential for

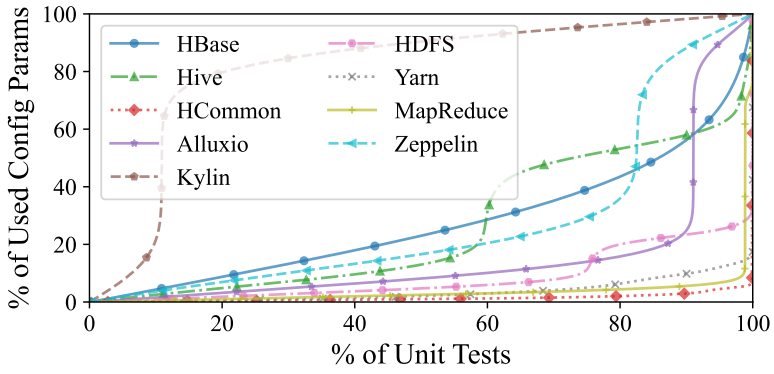


Figure 11: The cumulative distribution function (CDF) of the percentage of configuration parameters used by unit tests across the nine evaluated projects.

configuration testing, and UTopia [65] showed that existing unit tests can be transformed into fuzz drivers by turning them into parameterized unit tests [117]. However, unit tests also sharpen the guidance problem because a unit test typically exercises only a small slice of the full configuration space.

Figure 11 shows the percentage of configuration parameters exercised by unit tests across the nine evaluated cloud systems. On average, a unit test exercises only 15.14% of the total parameters; in HCommon, MapReduce, and Yarn, 90% of unit tests exercise fewer than 10% of the parameters. Directly mutating parameters from the full configuration space is therefore inefficient: when a fuzzer mutates parameters that the current test does not read, the resulting configuration is likely to produce the same behavior as the parent seed. The exercised-parameter set is also not fixed: mutating one parameter can steer execution to a new path that exercises additional parameters. A one-time parameter set collected before fuzzing (e.g., the set observed under the default configuration) is therefore insufficient; the fuzzer must track parameter usage in each round and use it, together with code coverage, to guide later mutation.

4.2.3 Configuration APIs

ConfFuzz instruments configuration APIs for dynamic tracking of configuration parameters and instruments configuration constructors for in-situ mutation of configuration values

during fuzzing (§4.3.2). Mature software projects, including the nine in our evaluation, typically expose unified configuration APIs to manage hundreds to thousands of parameters systematically, a design that prior work on configuration testing and analysis has repeatedly relied on [22], [40], [41], [57], [74], [78], [100], [102], [103]. Configurations are commonly represented as dedicated classes that expose a pair of operations: a GET API of the form "`<T> get(String parameter)`" that returns a parameter's value, and a SET API of the form "`void set(String parameter, <T> value)`" that updates it. In practice these raw APIs are often wrapped by typed APIs such as `getInt` and `getBoolean`. These simple, unified APIs provide natural points for instrumentation to track runtime parameter usage and to mutate the in-memory configuration object, without reasoning about complex parsing logic or diverse file formats (XML, INI, JSON, YAML).

4.3 Design

ConfFuzz is a greybox configuration fuzzer for highly configurable cloud systems that uses developer-written unit tests as fuzz drivers. Unlike prior configuration fuzzers [47], [48], [113], ConfFuzz explicitly captures the interaction between configuration parameters and system code, leading to two key decisions: ConfFuzz mutates only the configuration parameters exercised by the selected fuzz driver (rather than a random subset of the full parameter space), and it uses both code coverage and exercised-parameter information to decide which executions to save as seeds and how much fuzzing energy to allocate to each seed.

Algorithm 1 shows conventional code-coverage-guided greybox fuzzing. The fuzzer maintains a seed set, allocates a mutation budget with `numCandidates(s, S)`, which scores the selected seed s against the rest of the queue S using code coverage alone, mutates saved inputs, and saves a candidate to the seed pool only when executing it expands total code coverage. ConfFuzz, shown in Algorithm 2, keeps this greybox structure but specializes it to configuration fuzzing with unit tests (the teal statements mark the ConfFuzz-specific

Algorithm 1: Coverage-based greybox fuzzing.

Input: program p, set of initial inputs I
Output: a set of test inputs and failing inputs

$S \leftarrow I$;
$F \leftarrow \emptyset$;
$totalCoverage \leftarrow \emptyset$;
repeat
  foreach *input in* S **do**
    for $1 \leq i \leq numCandidates(input, S)$ **do**
      $candidate \leftarrow mutate(input, S)$;
      $coverage, result \leftarrow run(p, candidate)$;
      if $result = Failure$ **then**
        $F \leftarrow F \cup \{candidate\}$;
      else if $coverage \not\subseteq totalCoverage$ **then**
        $S \leftarrow S \cup \{candidate, coverage\}$;
        $totalCoverage \leftarrow totalCoverage \cup coverage$;
until *given time budget expires*;
return S, F

extensions). ConfFuzz extends traditional greybox fuzzing in three ways. First, it runs the fuzz driver once under the default configuration D and uses that execution to create the initial seed, because the default configuration is the natural valid starting point for a unit test. Second, each saved seed stores not only a configuration and its code coverage but also the set of parameters exercised in that execution, which lets ConfFuzz mutate only the parameters that affected the parent execution. Third, ConfFuzz extends the energy-allocation and seed-saving logic: `numCandidates(s, S)` uses both the code coverage of the selected seed s and its exercised parameters, relative to the current seed pool S, to decide how many candidates to generate, and ConfFuzz saves a seed when it either covers new code or exercises a previously unseen parameter.

4.3.1 Transforming Unit Tests into Fuzz Drivers

ConfFuzz follows the same high-level idea as UTopia [65]: it reuses existing unit tests as fuzz drivers. The transformation differs because the fuzzed input is not a conventional API argument but a configuration object. A unit test typically creates a configuration object and later exercises only a small number of parameters through configuration APIs,

Algorithm 2: ConfFuzz: parameter-aware greybox configuration fuzzing.

Input: test t, default configuration D
Output: seeds S (each with configuration, code coverage, and exercised parameters) and failing configurations F

$coverage, params, _ \leftarrow run(t, D)$;
$S \leftarrow \{(D, coverage, params)\}$;
$F \leftarrow \emptyset$;
$totalCoverage \leftarrow coverage$;
$totalParams \leftarrow params$;
repeat
  foreach $(config, coverage, params) \in S$ **do**
    /* param-aware energy allocation */
    for $1 \leq i \leq numCandidates(coverage, params, S)$ **do**
      /* param-aware configuration mutation */
      $candidate \leftarrow mutateConfig(config, params)$;
      $coverage', params', result \leftarrow run(t, candidate)$;
      if $result = Failure$ **then**
        $F \leftarrow F \cup \{candidate\}$;
      else if $coverage' \not\subseteq totalCoverage \vee params' \not\subseteq totalParams$ **then**
        $S \leftarrow S \cup \{(candidate, coverage', params')\}$;
        $totalCoverage \leftarrow totalCoverage \cup coverage'$;
        $totalParams \leftarrow totalParams \cup params'$;
until *given time budget expires*;
return S, F

so ConfFuzz must connect a mutated configuration object to the test under fuzz. It does so by instrumenting the constructor of the configuration class to load mutated parameter values during fuzzing. Figure 12a shows an instrumentation example in Hadoop: whenever a test creates a configuration object, the instrumented constructor invokes the ConfFuzz configuration generator, obtains a mutated object, and passes it to the test. If the class does not expose such a constructor, ConfFuzz instruments the GET APIs and returns the generated value when the test invokes the GET API (Figure 12b). By connecting the unit test with the fuzzed configuration object, ConfFuzz transforms existing unit tests into fuzz drivers without modifying the tests. Not all unit tests are suitable, because some exercise no configuration parameter; ConfFuzz automatically identifies suitable tests by running the full unit-test suite once under the default configuration and selecting only the tests that exercise at least one configuration parameter.

```
public class Configuration {
  public Configuration() {
-   this(loadDefaultConfig);
+   this(ConfFuzzGenerator.genConfig());
  }
} /* .../hadoop/conf/Configuration.java */
```

(a) Configuration constructor.

```
public String get(String name) {
+   String confFuzzParam = name;
    String[] names = handleDeprecation(deprecationContext.get(),
        name);
    String result = null;
    for(String n : names) {
+       confFuzzParam = n;
        result = substituteVars(getProperty(n));
    }
+   ConfigTracker.recordGet(confFuzzParam, result);
    return result;
} /* .../hadoop/conf/Configuration.java */
```

(b) Configuration GET API.

Figure 12: Instrumentation for the Hadoop configuration constructor and GET API. The constructor loads a generated configuration; the GET API records each exercised parameter in the ConfFuzz tracker.

4.3.2 Parameter-Aware Configuration Mutation

Parameter-aware configuration mutation lets ConfFuzz use unit tests effectively as fuzz drivers. Traditional greybox fuzzing (Algorithm 1) mutates a saved input without knowing which parts of it affected the current execution. As §4.2.2 showed, a unit test typically exercises only a small subset of parameters, so mutating across the full parameter space wastes effort. ConfFuzz instead tracks the parameters exercised by each execution and passes that information to `mutateConfig(config, params)` in Algorithm 2, so each round mutates only the parameters exercised by the parent execution.

Dynamic parameter tracking. ConfFuzz instruments the configuration GET API(s) to dynamically track the parameters that a fuzz driver actually exercises in each fuzzing execution (§4.2.3). Figure 12b shows the instrumentation for Hadoop; only three new lines

are required, because typed wrappers such as `getInt` all call the `get` method. When a parameter is accessed during fuzzing, ConfFuzz records it in the parameter tracker (`ConfigTracker`). In the GET API of Figure 12b, `handleDeprecation` resolves a requested name to a set that may include aliases and deprecated names, so ConfFuzz updates the recorded name inside the same name-resolution loop that produces the returned value; `ConfigTracker` therefore records the resolved name whose value the GET API actually returns, rather than an unrelated alias. The instrumentation strategy resembles that of Ctest [41] (Chapter 2); the key difference is that Ctest logs the parameters used under the default configuration as a one-time analysis step, whereas ConfFuzz tracks exercised parameters in *every* fuzzing execution and uses the tracked set to guide later mutation.

Mutation workflow. The dynamically tracked parameter set determines which parameters ConfFuzz may mutate in each round. Each saved seed stores the configuration used in that execution, its code coverage, and the set of parameters exercised under that configuration; the saved configuration becomes the base for later mutation, and the stored parameter set determines which parameters may be mutated. ConfFuzz starts fuzzing from the default configuration because it is valid and well tested; if a fuzz driver fails even under the default configuration, that failure already indicates a bug and the driver is unsuitable. The blackbox fuzzer ECFuzz [48] also starts from the default configuration, but it mutates from the default in every round, whereas ConfFuzz uses the default only to create the initial seed. When ConfFuzz selects a seed as parent, it starts from that seed's configuration and mutates only the parameters in that seed's parameter set; after executing the mutated configuration, it records the parameters exercised in the new execution. The exercised-parameter set can therefore change dynamically during fuzzing. Figure 13 shows an example in which a MapReduce test exercises a new parameter after a boolean parameter changes from `false` to `true`, enabling a new control flow.

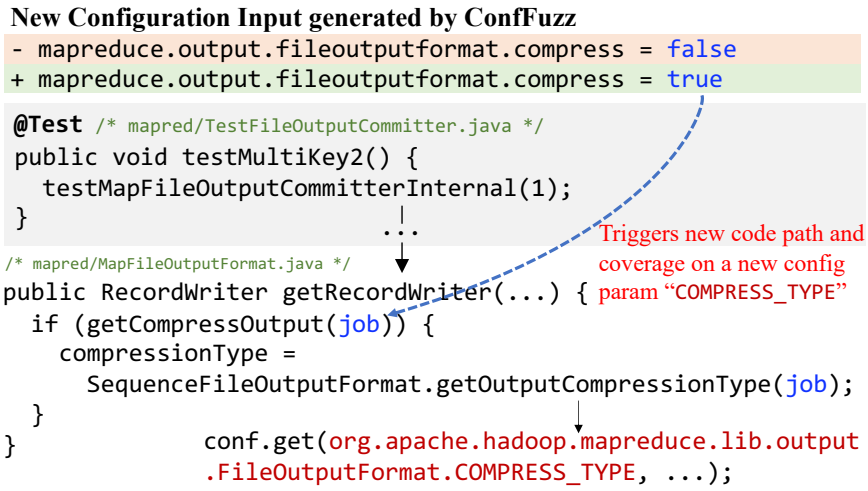


Figure 13: A new configuration parameter `COMPRESS_TYPE` is used by the unit test `testMultiKey2`, via a new control flow, when a boolean parameter changes.

Generating values for selected parameters. After selecting the parameters to mutate, ConfFuzz generates values for them while all other parameters retain their values from the parent seed. ConfFuzz follows prior configuration fuzzers [47], [48] in performing type inference and semantic inference for each parameter: it infers the type of a parameter from the type of its default value, and for `String`-typed parameters uses predefined regular-expression patterns for semantic inference. Despite its simplicity, this approach is effective in practice.

4.3.3 Parameter-Aware Energy Allocation

We now describe how ConfFuzz extends seed admission and mutation budgeting. Traditional greybox fuzzing saves a candidate as a seed when it expands total code coverage; ConfFuzz keeps that rule and adds a second condition, saving a candidate when its exercised-parameter set contains a parameter outside `totalParams`. New exercised parameters often accompany new code coverage, but the converse does not always hold, so this second condition captures marginal novelty in the configuration space directly. Traditional greybox fuzzing also uses code coverage to allocate more energy to seeds with broader coverage [44]; ConfFuzz extends this with parameter information. For a selected seed s, let C_s

be its covered branches and P_s its exercised parameters, and let S be the current seed pool. ConfFuzz defines $w_c = |C_s| / \max_{i \in S} |C_i|$ and $w_p = |P_s| / \max_{i \in S} |P_i|$, and `numCandidates` returns $\max(1, \lceil b \times w_c \times w_p \rceil)$, where b is the base mutation budget. The coverage term preserves the standard greybox intuition that broader-coverage seeds deserve more follow-up mutations; the parameter term adds an analogous preference for seeds whose executions span more of the configuration space. The saving rule rewards marginal discoveries in the current round, while the budgeting rule prioritizes saved seeds in later rounds according to the breadth of behavior they already exercise. Each iteration generates candidates only within the time and per-seed mutation budget, so although the seed pool S grows as new seeds are saved, the algorithm does not overrun its time budget. Algorithm 2 runs on a single unit-test fuzz driver, and ConfFuzz applies it independently to each driver.

4.3.4 Debugging Support

When ConfFuzz detects test failures, users still need to localize the underlying bugs. An immediate challenge is identifying the failure-inducing configuration parameters among a large set of mutated parameters. For a given failure, ConfFuzz uses a delta-debugging-inspired algorithm [114] (Algorithm 3) that applies randomized binary search to find a minimal set of parameters that reproduces the failure. It repeatedly splits the set of mutated parameters (relative to the default configuration) into two subsets and tries to reproduce the failure with each (lines 6–9); if a subset reproduces it, the search continues with that subset, and when the minimal triggering set spans both halves (so neither subset reproduces the failure) ConfFuzz retries with a new random partition until it reaches the attempt limit (lines 3–5). Finally it falls back to one-by-one removal (line 10): it removes one parameter, reruns the test, and adds the parameter back if the failure no longer reproduces.

ConfFuzz then filters out *benign* failures whose exception type and message indicate that the system rejected the configuration as intended. Specifically, a failure is

Algorithm 3: ConfFuzz fault-localization algorithm.

Input: test t, failing configuration C_{fail}, max attempt limit $MAX_{attempts}$, default configuration $C_{default}$
Output: minimal configuration that triggers the failure C_{min}
/* C_{fail} stores only the parameters that differ from $C_{default}$ */

```
1  failure, exception ← RUN(t, C_default ⊎ C_fail);
2  attempts ← 0;
3  while attempts < MAX_attempts ∧ SIZE(C_fail) > 1 do
4      attempts ← attempts + 1;
5      C_left, C_right ← RANDOMSPLITINTOHALVES(C_fail);
6      if RUN(t, C_default ⊎ C_left) = (failure, exception) then
7          C_fail ← C_left;
8      else if RUN(t, C_default ⊎ C_right) = (failure, exception) then
9          C_fail ← C_right;
10 C_min ← REMOVEONEBYONE(t, C_fail);
11 return C_min
```

benign if the exception is either an `IllegalArgumentException` or an `IOException` with configuration- or parameter-related keywords in the message, or if it is a `ComparisonFailure` or `AssertionError` thrown from the system code and the message includes the name or value of at least one failure-inducing parameter. These rules are neither sound nor complete, but we find them empirically effective; such benign failures are better understood as misconfigurations [41], which are not the focus of ConfFuzz. Finally, ConfFuzz groups failures that share the same exception type, the same failure-inducing parameters, and the same stack trace (excluding external library frames), so that users inspect one representative failure per group. In our evaluation, this reduced hundreds of thousands of failures to a few hundred groups.

4.4 Implementation

We implement ConfFuzz in 9,800+ lines of Java code. ConfFuzz uses JQF [118] as the greybox coverage-guided fuzzing backend, sets the base mutation budget b to 50 by default [118], and uses RgxGen [119] to generate randomized parameter values that match a given regular expression (deterministically per seed, to support debugging). ConfFuzz uses

two maps for dynamic parameter tracking: a local map for the parameter set exercised in the current round and a global map for all parameters exercised across rounds. When a candidate execution is saved as a seed, ConfFuzz stores the local map with that seed and retrieves it when that seed is selected again as a parent. We measure the overhead of dynamic parameter tracking by comparing test execution time with and without tracking; the average overhead is only 3%.

4.5 Evaluation

Our evaluation answers the following research questions:

- **RQ1:** Does ConfFuzz find more bugs than its variants?
- **RQ2:** Does ConfFuzz explore more of the configuration parameter space?
- **RQ3:** Does ConfFuzz achieve higher code coverage?
- **RQ4:** What bugs do ConfFuzz and its variants find?

4.5.1 Experimental Setup

Evaluated projects. We evaluate ConfFuzz on nine widely used open-source cloud systems: HBase, HCommon, HDFS, Alluxio, Hive, Kylin, MapReduce, Yarn, and Zeppelin. Table 7 summarizes the selected module of each project, its number of configuration parameters and fuzz drivers, its size, and the evaluated commit. All selected projects are highly configurable (HDFS exposes 2771 parameters in the selected module, HBase 1348, and HCommon 892), and even the smaller systems still expose many (e.g., 150 in Kylin and 146 in Zeppelin). Following prior testing work [41], [57], we evaluate one core module per project to keep cost manageable while still covering the main configuration-API surface.

Evaluated techniques. We evaluate four greybox techniques: ConfFuzz, ConfFuzz$_{\text{Base}}$, ConfFuzz$_{\text{Track+Def}}$, and ConfFuzz$_{\text{Track}}$ (Table 8). ConfFuzz$_{\text{Base}}$ serves as the prior-work base-

Table 7: Cloud system projects used in our evaluation.

Project/Module	**# Config Params**	**# Fuzz Drivers**	**LOC**	**SHA**
HCommon/hadoop-common	892	998	253K	`d37586`
HDFS/hadoop-hdfs	2771	1890	362K	`d37586`
HBase/hbase-server	1348	980	431K	`d38552`
Alluxio/core	847	158	63K	`bc8e1d`
Hive/ql	669	1944	560K	`6b05d6`
MapReduce/client-core	270	232	63K	`d37586`
Yarn/hadoop-yarn-common	242	341	119K	`d37586`
Kylin/core-cube	150	105	21K	`322ab6`
Zeppelin/zeppelin-zengine	146	182	23K	`f2b16f`

line: it reimplements the basic greybox strategy of command-line option fuzzers such as ConfigFuzz [47] for Java-based cloud systems (ConfigFuzz itself targets C/C++ programs and command-line options), starting from the default configuration, mutating from the full configuration space, and relying only on code-coverage guidance. ConfFuzz$_{\text{Track+Def}}$ adds parameter-aware mutation (§4.3.2); ConfFuzz$_{\text{Track}}$ keeps that mutation but does not start from the default configuration, isolating the effect of default initialization; and ConfFuzz further adds parameter-aware energy allocation (§4.3.3). All four share the same API instrumentation, unit-test fuzz drivers, value generator, and failure processing. We do not include ECFuzz [48] as a blackbox baseline because its design targets a different population of bugs. ECFuzz drives system tests and uses unit tests as a *filter*: a generated configuration that fails a unit test is discarded before system testing. ConfFuzz takes the opposite view, using unit tests as fuzz drivers and treating configuration-induced unit-test failures as the bugs to expose. The failures ConfFuzz targets are therefore, by construction, the ones ECFuzz filters out, so a head-to-head bug count would reflect the two designs' different targets rather than their relative effectiveness. The two also fuzz at different granularities: ECFuzz fuzzes a whole system, whereas ConfFuzz fuzzes each unit-test driver independently, so ConfFuzz runs many more executions within the same time budget. Prior work has also shown greybox fuzzing to be more effective than blackbox fuzzing [43], [44], [47], [120]. We also do not compare against configuration-testing techniques [41], [42], [70]: they

Table 8: Evaluated techniques and the components enabled.

Technique	Code Coverage Guidance	Start From Default Config	Param-Aware Config Mutation	Param-Aware Energy Allocation
$\text{ConfFuzz}_{\text{Base}}$	✓	✓	✗	✗
$\text{ConfFuzz}_{\text{Track}}$	✓	✗	✓	✗
$\text{ConfFuzz}_{\text{Track+Def}}$	✓	✓	✓	✗
ConfFuzz	✓	✓	✓	✓

test a specific configuration or code under a specific configuration rather than search the configuration space, so they are not fuzzers and thus not directly comparable baselines.

Fuzzing setup. For each project we keep only the unit tests that exercise at least one configuration parameter as fuzz drivers, fuzzing at most 1,000 tests per project (all eligible tests when a project has at most 1,000, otherwise a random sample of 1,000). After discarding drivers that fail under the default configuration, this process yields 4,838 fuzz drivers across the nine projects. We fuzz each driver for 30 minutes using the same unit-test-driven setup as UTopia [65] and repeat each experiment five times to reduce the effect of fuzzing randomness. Using unit tests as fuzz drivers gives high test execution speed: across all campaigns, ConfFuzz averages 20.57 executions per second (48.6 ms per execution). For comparison, ECFuzz [48] reports that system testing on the same class of cloud systems executes 0.0371 tests per second on average, about 27 seconds per execution, so ConfFuzz's unit-test drivers are roughly 550 times faster. One run of one variant therefore requires 2,419 VM-hours, and the 20 main campaigns in this study (four variants and five repetitions per variant) require 48,380 VM-hours (over 2,000 VM-days) before reruns caused by VM preemption and subsequent bug triage, well beyond the usual 24-hour by 10-trial fuzzer evaluations. The experiments run on Azure VMs [107] with dual-core CPUs, 8 GB RAM, Ubuntu 20.04.2, and Java 11.0.19. Our conclusions hold within this fixed time budget with unit-test drivers; we do not claim that they extrapolate to arbitrarily long campaigns. To transform unit tests into fuzz drivers, we instrument the

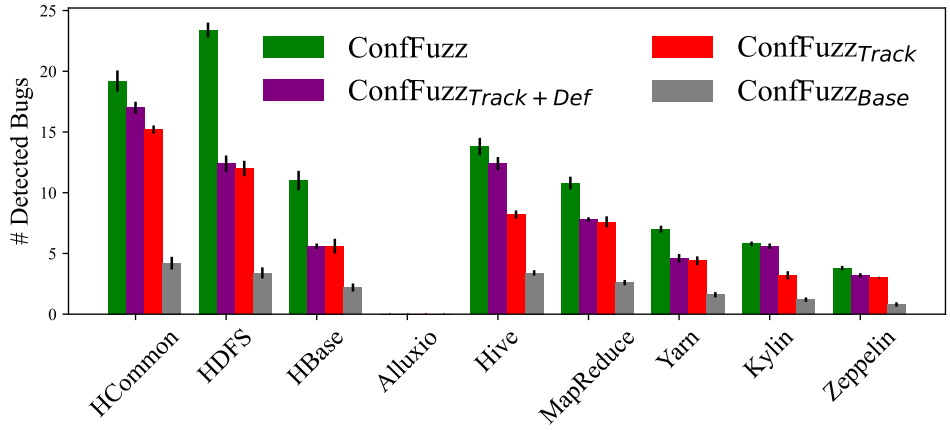
configuration APIs of each project (§4.3.1); instrumenting the GET APIs and constructors took about half an hour per project, adding 20 lines of code on average (minimum 11 for Kylin, maximum 37 for Hive).

4.5.2 RQ1: Does ConfFuzz Find More Bugs?

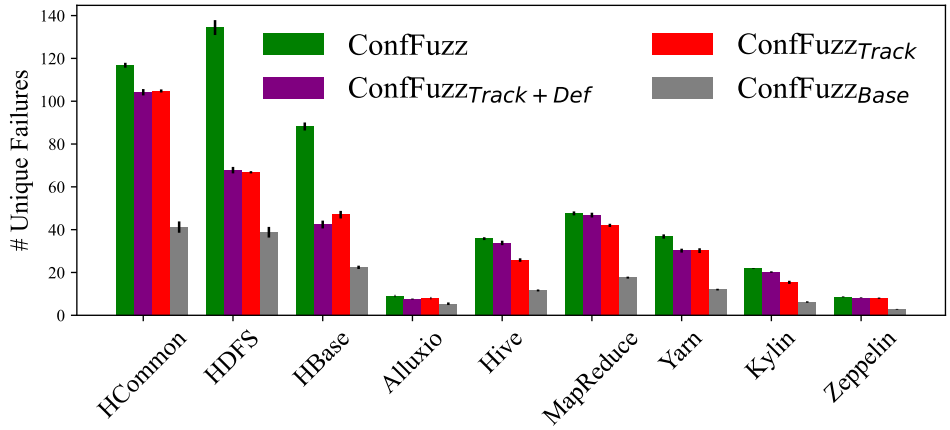
We compare the bugs found by ConfFuzz with those found by its three variants across all nine projects, running each technique five times and reporting (1) the average number of unique failures and bugs per project and (2) the union of unique bugs across the five runs. Figure 14 reports the per-project averages.

Across the nine projects, ConfFuzz finds the most bugs in eight and ties only on Alluxio, where no evaluated technique finds a bug. ConfFuzz finds 94.8 bugs on average, compared with 68.6 for $\text{ConfFuzz}_{\text{Track+Def}}$, 59.2 for $\text{ConfFuzz}_{\text{Track}}$, and 19.4 for $\text{ConfFuzz}_{\text{Base}}$. The largest gains appear on HDFS (23.4 vs. 12.4, 12.0, and 3.4 bugs) and HBase (11.0 vs. 5.6, 5.6, and 2.2 bugs).

Comparing $\text{ConfFuzz}_{\text{Base}}$ with $\text{ConfFuzz}_{\text{Track+Def}}$ isolates the effect of parameter-aware mutation (§4.3.2), which raises the average bug count from 19.4 to 68.6. $\text{ConfFuzz}_{\text{Base}}$ mutates from the full, very large configuration space, whereas $\text{ConfFuzz}_{\text{Track+Def}}$ mutates only the much smaller exercised-parameter subset, which consists of parameters more likely to affect execution; the improvement is largest in systems with large configuration spaces such as HDFS and HBase. Comparing $\text{ConfFuzz}_{\text{Track+Def}}$ with $\text{ConfFuzz}_{\text{Track}}$ isolates the effect of starting from the default configuration, and the effect is limited (68.6 vs. 59.2 on average): starting from the default configuration, although common in recent work [47], [48], is not a critical design choice as long as parameter-aware mutation steers fuzzing toward exercised parameters. Comparing ConfFuzz with $\text{ConfFuzz}_{\text{Track+Def}}$ isolates the effect of parameter-aware energy allocation (§4.3.3), which raises the average bug count from 68.6 to 94.8 by steering fuzzing toward seeds that exercise more parameters. Unique test failures show the same ranking: ConfFuzz triggers 499.0 unique failures on average,



(a) # bugs



(b) # unique test failures

Figure 14: Number of bugs and unique test failures found on all evaluated projects, averaged over five runs.

compared with 361.2, 348.0, and 158.0 for the three variants. HDFS and HBase again show the largest margins: ConfFuzz reaches 134.4 and 88.2 unique failures, while the three variants reach 67.8/66.8/38.8 and 42.4/47.0/22.4, respectively.

Figure 15 reports the union of unique bugs across five runs, where a bug counts for a technique if that technique exposes it in at least one run. We omit ConfFuzz$_{\text{Base}}$ from the diagram because every bug it finds is also found by another technique, so it adds nothing to the union. Together, ConfFuzz and its variants expose 125 unique bugs. ConfFuzz finds 118 of them, whereas ConfFuzz$_{\text{Track+Def}}$ and ConfFuzz$_{\text{Track}}$ find 76 and 83, respectively.

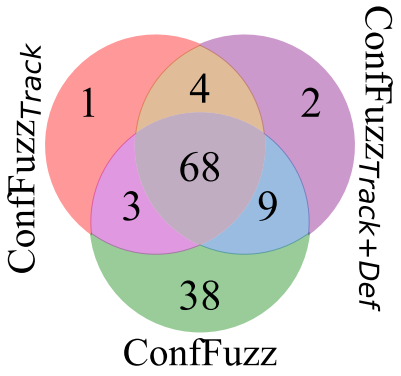


Figure 15: Union of unique bugs across five runs for ConfFuzz and its two parameter-aware ablations (a bug counts for a technique if it is exposed in at least one run).

ConfFuzz alone contributes 38 bugs that neither ablation finds, whereas the two ablations contribute only 2 and 1 exclusive bugs. Seven bugs are found by an ablation but not by ConfFuzz (most likely because ConfFuzz prioritizes seeds that exercise more parameters, which can cause it to miss some seeds that exercise fewer parameters but still trigger unique bugs), but the tradeoff is worthwhile, yielding a much larger exclusive bug count (38 vs. 2 and 1) and a much higher mean bug count (94.8 vs. 68.6 and 59.2). Formal statistics support the same conclusion (Table 9). ConfFuzz reaches a mean total bug count of 94.8 (95% CI $[90.6, 99.0]$) and a mean total failure count of 499.0 (95% CI $[486.2, 511.8]$). Relative to ConfFuzz$_{\text{Track+Def}}$, ConfFuzz gains 26.2 bugs and 137.8 failures on average, and the gains over ConfFuzz$_{\text{Track}}$ and ConfFuzz$_{\text{Base}}$ are larger. Every paired comparison between ConfFuzz and a baseline is significant under a paired t-test ($p < 0.001$), every 95% confidence interval for the difference excludes zero, and every effect size is large (Cohen's d_z from 5.6 to 38.6).

4.5.3 RQ2: Does ConfFuzz Explore More of the Parameter Space?

We next evaluate whether ConfFuzz explores more of the configuration parameter space than its variants, which can explain why it finds more bugs. We compare ConfFuzz with ConfFuzz$_{\text{Track+Def}}$ and ConfFuzz$_{\text{Track}}$, since ConfFuzz$_{\text{Base}}$ does not track exercised parameters. Figure 16 shows, per project, the ratio between the number of exercised parameters

Table 9: Formal statistical summary for RQ1 across five runs. Δ denotes the difference of the mean between ConfFuzz and the compared technique.

Metric	Technique	Mean	95% CI	Δ	p-value	d_z
# Bugs	ConfFuzz	94.8	$[90.6, 99.0]$	–	–	–
	$\text{ConfFuzz}_{\text{Track+Def}}$	68.6	$[66.7, 70.5]$	+26.2	4.7e-05	8.41
	$\text{ConfFuzz}_{\text{Track}}$	59.2	$[54.1, 64.4]$	+35.6	2.3e-04	5.64
	$\text{ConfFuzz}_{\text{Base}}$	19.4	$[17.0, 21.8]$	+75.4	5.7e-07	25.42
# Fail.	ConfFuzz	499.0	$[486.2, 511.8]$	–	–	–
	$\text{ConfFuzz}_{\text{Track+Def}}$	361.2	$[349.3, 373.1]$	+137.8	6.3e-05	7.81
	$\text{ConfFuzz}_{\text{Track}}$	348.0	$[340.8, 355.2]$	+151.0	4.9e-07	26.49
	$\text{ConfFuzz}_{\text{Base}}$	158.0	$[150.8, 165.2]$	+341.0	1.1e-07	38.61

during fuzzing and the number exercised by the initial seed; a ratio of 1.0 means fuzzing never moves beyond the initial parameter set, while a ratio of 2.0 means it exercises twice as many.

ConfFuzz finishes with the highest ratio in every project, showing that parameter-aware mutation and parameter-aware energy allocation together help it explore more parameters than the two ablations. The largest gaps appear where ConfFuzz also gains the most bugs: on HDFS, ConfFuzz grows the exercised-parameter set to 39.5× the initial size, versus 25.6× and 25.0× for the ablations; on HBase, the final ratios are 6.0×, 4.7×, and 4.1×; and on HCommon, 2.18×, 1.37×, and 1.41×. Kylin, Zeppelin, and Alluxio stay close to 1.0 for all techniques, indicating that their fuzz drivers reveal few additional parameters beyond the initial seed, but even there ConfFuzz reaches the largest explored space. The two ablations show similar growth, which suggests that starting from the default configuration has no consistent effect on the explored parameter space; only ConfFuzz, with parameter-aware energy allocation (§4.3.3), consistently explores more parameters, because once a mutation exposes a new parameter, ConfFuzz records it, saves the seed, and allocates more effort to expanding that region.

Beyond ratios, the absolute counts confirm the same trend. Across all ConfFuzz driver executions, the median execution grows from 22 exercised parameters in the initial seed to 35 after fuzzing, and the mean grows from 118.8 to 194.2 parameters, an increase of

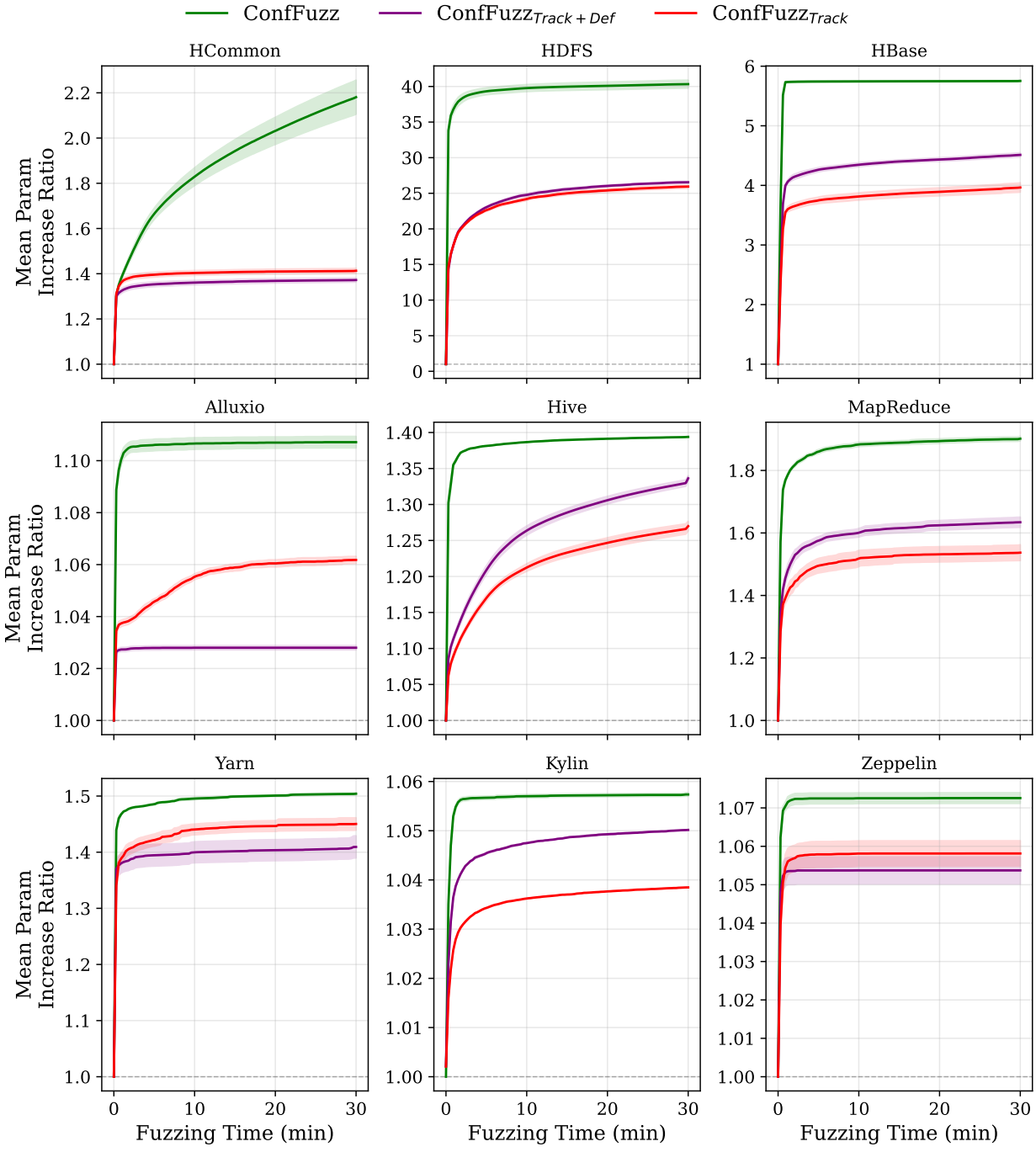


Figure 16: Mean growth ratio of the exercised configuration-parameter count per technique and project, averaged across five runs. Shaded regions show variation across the five runs.

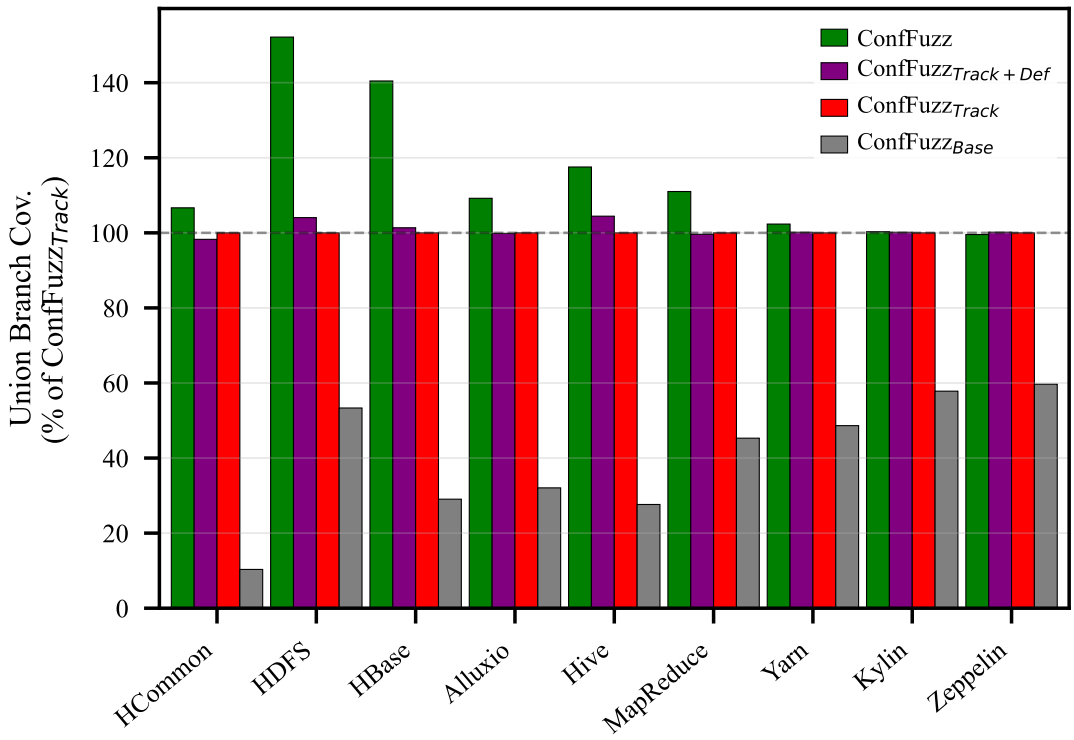


Figure 17: Union branch coverage per project, averaged over five runs and normalized to ConfFuzz$_{\text{Track}}$.

75.4 exercised parameters on average. In total, 60.1% of drivers exercise at least one new parameter during fuzzing.

4.5.4 RQ3: Does ConfFuzz Achieve Higher Code Coverage?

Having shown that ConfFuzz explores more of the configuration parameter space, we next examine whether this translates into higher code coverage. We measure the branch coverage achieved by ConfFuzz and compare it with ConfFuzz$_{\text{Track+Def}}$, ConfFuzz$_{\text{Track}}$, and ConfFuzz$_{\text{Base}}$. Following [65], we report *union* branch coverage: for each project we union the branches covered across all of the project's fuzz drivers, averaged over five runs. Figure 17 normalizes each project to ConfFuzz$_{\text{Track}}$ = 100% so that projects ranging from about 700 to 27,000 branches are comparable.

ConfFuzz attains the highest union branch coverage on eight of nine projects against

ConfFuzz$_{\text{Track}}$ and ConfFuzz$_{\text{Track+Def}}$, and on all nine projects against ConfFuzz$_{\text{Base}}$. Taking the geometric mean of the per-project coverage ratios, ConfFuzz covers 14.2% more branches than ConfFuzz$_{\text{Track}}$, 13.3% more than ConfFuzz$_{\text{Track+Def}}$, and roughly 3.2 times the coverage of ConfFuzz$_{\text{Base}}$. The largest gains concentrate on the high-parameter projects (HDFS +52.2%, HBase +40.5%, Hive +17.5% over ConfFuzz$_{\text{Track}}$), while low-parameter projects show small differences (Kylin +0.3%, Yarn +2.3%). The single loss is Zeppelin, where ConfFuzz is 0.4% below ConfFuzz$_{\text{Track}}$ and 0.6% below ConfFuzz$_{\text{Track+Def}}$, a five-branch tie on the smallest-parameter project.

The coverage trend mirrors the parameter-exploration trend of Figure 16: the projects where ConfFuzz drives the largest parameter exploration are exactly the ones with the largest coverage gains, since exercising more configuration parameters generally exercises more code. On projects with few configuration parameters, the three ConfFuzz variants perform almost identically because there is little parameter space to exploit. ConfFuzz$_{\text{Base}}$ trails all three on every project, since it mutates parameters chosen uniformly at random from the whole parameter set rather than from the exercised set, which shows the benefit of restricting mutation to exercised parameters.

4.5.5 RQ4: Summary of Bugs Found

We define a bug as a defect with an observable failure symptom, and we report only bugs that reproduce on the test with the offending configuration. We reported 77 of the 125 bugs to developers; so far, 26 have received responses, of which 14 have been fixed and 12 explicitly confirmed, and none has been rejected. The 125 bugs span several categories, whose frequencies are shown in Table 10. The most common are crashes with no or cryptic error messages, which leave users without actionable diagnosis; `NullPointerException`s, where a mutated configuration enables a feature without initializing all required state; *semantic bugs* that are deeply buried in the code and tightly coupled with specific features (manifesting, e.g., as inconsistent behavior, infinite loops, or resource leaks); *arithmetic*

Table 10: Bug types found by ConfFuzz and its variants, and their frequency.

ID	Bug Type	Count	%
B1	Crashing with no message	41	32.8%
B2	NullPointerException	25	20.0%
B3	Semantic bugs	23	18.4%
B4	Arithmetic bugs	19	15.2%
B5	Bugs in test code	15	12.0%
B6	Test polluter	2	1.6%
Σ		125	100.0%

bugs from calculations that do not account for unexpected parameter values; *bugs in test code*; and *test polluters* that cause configuration-related flaky tests. We highlight one case study per category.

Table 11 presents one representative bug per category, with the fuzzed configuration input that triggers it and the fuzz driver (unit test) that exposes it.

For a *crash with no message* (HD-A), setting `fs.permissions.umask-mode` to 612 makes HDFS fail to perform I/O operations without any error message, making diagnosis difficult. For a *NullPointerException* (HB-B), setting HBase's `codec` parameter to `SecureWALCellCodec` causes a `NullPointerException`: the codec class lacks an expected constructor, so the RPC client fails to initialize and a `finally` block then calls `shutdown()` on an uninitialized executor; the fix adds a null check. For a *semantic bug* (HD-C), setting `dfs.namenode.max.extra.edits.segments.retained` to a negative value crashes the HDFS NameNode's storage-retention manager with an `IndexOutOfBoundsException` during edit-log purging, because the purge loop drains the list past empty; the fix adds bounds handling. In another semantic bug (MR-D), enabling a MapReduce option that skips cleanup of the `_temporary` folder also prevents the `_SUCCESS` marker from being written, so later commit attempts duplicate task output and corrupt data. For an *arithmetic bug* (HB-E), setting `hbase.regionserver.logroll.multiplier` to 0.0 triggers a division-by-zero exception that crashes the HBase RegionServer at startup. For a *bug in test code* (HC-F), the HCommon test casts every caught exception to `RemoteException`;

Table 11: Sample bugs found by ConfFuzz, one per bug type (the **Type** column refers to the bug-type IDs in Table 10).

Bug	Exception	Fuzzed configuration input (*parameter*=*value*)	Fuzz driver (test)	Type
HD-A	`AssertionError`	`fs.permissions.umask-mode=612`	`TestDecommissioningStatusWithBackoffMonitor.testDecommissionDeadDN`	B1
HB-B	`NullPointerException`	`hbase.client.default.rpc.codec=SecureWALCellCodec`	`AbstractTestIPC.testLauncher`	B2
HD-C	`IndexOutOfBoundsException`	`dfs.namenode.max.extra.edits.segments.retained=-1974676133`	`TestNNStorageRetentionManager.testNoLogs`	B3
MR-D	`AssertionError`	`mapreduce.fileoutputcommitter.cleanup.skipped=true`	`TestFileOutputCommitter.testCommitterWithDuplicatedCommitV1`	B3
HB-E	`ArithmeticException`	`hbase.regionserver.logroll.multiplier=0.0`	`TestWALMethods.testGetSplitEditFilesSorted`	B4
HC-F	`ClassCastException`	`hadoop.security.authentication=kerberos`	`TestRPC.testReaderExceptions`	B5
MR-G	`IOException`	`fs.permissions.umask-mode=243`	`TestFileOutputCommitter.testRecoveryUpgradeV1V2`	B6

when `hadoop.security.authentication` is `kerberos`, an `IOException` is thrown and the test crashes with a `ClassCastException`. Finally, for a *test polluter* (MR-G), mutating `fs.permissions.umask-mode` to revoke write permission leaves behind a directory the test never deletes, so all subsequent runs fail with permission denials; we fix the test by deleting the directory after every run.

4.6 Discussion

ConfFuzz assumes that configuration parameters are accessed through unified GET and SET APIs and that values can be represented in a key-value format; it can be difficult to ap-

ply to projects whose configuration API does not meet these assumptions. This GET-API abstraction originates in `java.util.Properties` and is pervasive in mature Java systems and libraries (for example, Apache Commons Configuration), which is the same uniformity that underlies a broad class of configuration analysis and testing tools [6], [7], [41], [70]. For a unit test to serve as a fuzz driver, it must (1) read at least one parameter through an instrumentable GET API, (2) exhibit configuration-dependent behavior, (3) pass under the default configuration, and (4) be reproducible enough to support confirmation; ConfFuzz's effectiveness relies on the abundance of such tests in mature projects [41], [65]. The value generator uses simple type and semantic inference, which is effective in practice but could be strengthened with program-analysis-based type inference [22], [78]. More broadly, ConfFuzz's central lesson (that code coverage alone is insufficient for configuration fuzzing in highly configurable cloud systems, and that the configuration parameters exercised at runtime are a second, complementary source of guidance) applies to any greybox fuzzer whose interface exposes structure that plain coverage does not capture.

4.7 Summary

We presented ConfFuzz, a parameter-aware greybox configuration fuzzer for highly configurable cloud systems. ConfFuzz uses existing unit tests (Ctests, Chapter 2) as fuzz drivers, dynamically tracks the configuration parameters exercised in each execution, restricts later mutation to those parameters, and allocates fuzzing energy using both code coverage and parameter coverage. We evaluated ConfFuzz and three of its variants on nine widely used cloud systems with 4,838 fuzz drivers. The results show that code coverage alone is insufficient for configuration fuzzing in highly configurable cloud systems: effective search also requires parameter-aware mutation and parameter-aware energy allocation. Across all campaigns, ConfFuzz and its variants expose 125 configuration-related bugs, including 14 fixed and 12 additional confirmed bugs. Where Ctest4J (Chapter 2) runs Ctests and uRTS (Chapter 3) selects them across changes, ConfFuzz transforms them into fuzz drivers to

actively search the configuration space for bugs.

Chapter 5 Palingenesis: Testing Graceful Restart in Distributed Systems

5.1 Overview

Graceful node restarts are essential routines in production distributed systems [1], [2], [11]. A graceful restart is a coordinated shutdown/rejoin protocol: operators restart nodes for rolling upgrades, configuration changes, maintenance, and performance tuning [11], [12], [51], [52]. These restarts are *planned*: before exiting, a node must actively execute shutdown logic (reject new requests, flush in-memory state, step down from leadership, drain resources, hand off responsibilities), and after restarting it must rejoin the cluster with correct role-specific semantics [3], [4], [51]. Bugs in this restart protocol cause long-tail correctness violations, rare races, or restart hangs that are hard to reproduce and often invisible under steady-state testing.

Automated robustness testing for distributed systems has long focused on crash failures, injecting faults and observing whether the system can recover [14], [20], [27], [54], [55], [121], with recent work broadening to crash/recovery, partial-failure, and greybox fault fuzzing [28], [122], [123], [124], [125]. However, planned graceful restarts remain under-tested. Unlike crash-centric recovery, a planned graceful restart must validate the shutdown/rejoin protocol itself, so restart-related bugs that arise only during shutdown or rejoin can remain undetected even after extensive crash testing. Moreover, detecting such bugs depends strongly on the workload: many surface only after the cluster has been driven into a restart-sensitive state, whereas restarting a freshly started or quiescent cluster exercises only shallow restart paths.

Many distributed systems already maintain extensive suites of *cluster tests* (also called MiniCluster-style tests) that start a small in-process cluster, drive realistic workloads, and assert externally visible outcomes [41], [71], [126], [127]. Crucially, MiniCluster frameworks typically expose programmatic node-control operations (stop, start, restart a node, and wait for it to rejoin), making a graceful restart an action the test can trigger directly [126],

[127]. Cluster tests are a natural fit for graceful restart validation because they execute real system components and protocols and already include correctness oracles, so restart-induced violations surface as test failures. However, leveraging cluster tests for graceful restart testing is challenging for two reasons. First, injecting a restart between every step for every combination of nodes creates a huge test space, and many such tests are ineffective: restart bugs surface only when the workload has driven the system into a particular state and the test continues after the restart to observe post-rejoin behavior. Selecting restart points therefore requires semantic understanding of the test intent, the cluster lifecycle phases, and project-specific helper abstractions that hide when key state transitions occur. Second, most cluster tests implicitly assume steady-state execution (nodes remain alive and references remain valid until teardown), so restarting at arbitrary points often produces *invalid* executions that fail on stale cluster-derived references unrelated to any restart bug.

Motivated by these challenges, we propose a test-centric perspective: *graceful restart testing as automated test transformation*. We present Palingenesis, an end-to-end system that transforms existing cluster tests into restart tests by injecting graceful node restarts while preserving the original workloads and test oracles. Palingenesis uses large language models (LLMs) [66], [67], [128] to infer test intent and lifecycle phases from the test code and project context, then injects graceful restarts at restart points aligned with the test intent and critical state transitions. To keep transformed tests executable, Palingenesis provides a practical stale-reference refresher that tracks cluster-derived references during execution and refreshes them after restart when possible, reporting explicit errors otherwise.

We evaluate Palingenesis on seven widely used Java-based distributed systems with MiniCluster-style tests, transforming 4,459 cluster test methods and executing 18,316 restart runs. The transformed tests expose 17 restart-related bugs: 6 fixed, 5 more confirmed, and the remainder pending with patches submitted. The bugs span shutdown,

reboot/initialization, and post-restart recovery, and manifest as node crashes, hangs, or safety-invariant violations. This chapter makes the following contributions:

- We automate graceful restart testing by reusing developer-written cluster tests and transforming them into restart tests, injecting graceful node restarts while preserving the original workloads and correctness oracles.
- We show how to use LLMs with project context to infer test intent and identify meaningful points for restart injection. A restart point is *meaningful* if the preceding workload has driven the cluster into a restart-sensitive state and the original oracle remains applicable afterward, so the test does not fail when no bug is present.
- We identify stale cluster-derived references as a key challenge in running transformed restart tests and provide a practical stale-reference refresher.
- We build Palingenesis, an end-to-end system, and evaluate it on seven systems with 4,459 transformed tests and 18,316 restart runs, exposing 17 restart-related bugs.

5.2 Background and Motivation

5.2.1 Graceful Restarts Are Common but Under-Tested

Graceful node restarts are essential routines in production distributed systems [1], [2], [11]. Operators restart nodes for planned maintenance, configuration changes, and upgrades, typically via rolling restart procedures intended to preserve availability; ZooKeeper, for instance, documents rolling restart sequences for configuration changes and upgrades [52], and Kubernetes relies on graceful pod termination to let workloads shut down cleanly [11], [51]. Automated robustness testing has long focused on crash failures and recovery [27], [28], [54], [55], [121]. These approaches effectively test crash recovery but model failure as an immediate stop and do not exercise the system-specific shutdown and rejoin procedures used in planned restarts. Crash injection can therefore miss restart-related bugs that

```
public void shutdown() {
  running = false;
  ...
  if (leader != null) {leader.shutdown("quorum Peer shutdown");}
  if (follower != null) {follower.shutdown();}
  ...
  try {
    zkDb.close(); // Missing null checking during shutdown: ZOOKEEPER-5012
  } catch (...) {...}
}
```

Figure 18: ZooKeeper `QuorumPeer` graceful shutdown.

arise when shutdown hooks run, responsibilities are handed off, or a rebooted node must rejoin and re-establish role-specific state. Unlike a crash, a graceful restart requires a node to execute a system-specific shutdown protocol (suspending background work, persisting metadata, draining requests, and transferring responsibilities) before rejoining with correct membership and reconstructed state. The same functional workload can expose different bugs depending on when and how the restart occurs, even if the system passes steady-state tests. Figure 18 shows a simplified ZooKeeper graceful shutdown that tears down internal services and closes persistent state before the process exits; this path can fail, e.g., ZOOKEEPER-5012 shows that `QuorumPeer.shutdown()` may call `zkDb.close()` without a null check.

5.2.2 Cluster Tests: Realistic Workloads with Built-In Oracles

Many distributed systems maintain extensive suites of *cluster tests*: developer-written tests that start a small local cluster (often in-process), execute realistic workloads, and verify correctness via developer-defined oracles. Hadoop's `MiniDFSCluster` and HBase's `HBaseTestingUtility` are representative [126], [127]. Cluster tests are attractive for robustness validation because they approximate production semantics (real components, real protocols), run quickly, and already encode correctness oracles, and prior work reuses such tests as workloads to validate different behaviors at scale [41], [57], [70], [71]. Some cluster tests already validate restart behavior, but only in narrow forms. The HDFS test in

```java
@Test
public void testFinalizedReplicas() throws Exception {
  Configuration conf = new HdfsConfiguration();
  MiniDFSCluster cluster = new MiniDFSCluster
      .Builder(conf).numDataNodes(3).build();
  ...
  try {
    createFiles("/test", (short) 3);
    ...
    util.waitReplication(fs, TopDir, (short) 3);
    cluster.restartDataNodes(); // restart all DataNodes
    ...
    checkFiles("/test");
  } finally { cluster.shutdown(); }
}
```

Figure 19: A simplified HDFS cluster test that tests restart.

Figure 19, for example, restarts all DataNodes and checks that finalized replicas remain readable, but covers only one restart granularity and one workload pattern, and required substantial manual effort to write. Many other restart-sensitive workloads exist in the cluster test suite but are never exercised under restarts.

5.2.3 Challenges of Restart Testing in Cluster Tests

To test graceful restart behavior, we transform existing cluster tests into restart tests by injecting graceful node restarts while preserving the original workload and oracles. The challenge is not injecting a restart anywhere, but performing restarts consistent with the test intent and cluster lifecycle.

Restart point matters. Meaningful restart points align with the test intent and with the state-changing or stabilization steps the test performs. Figure 20 shows a real HBase cluster test: it starts a `MiniHBaseCluster` with BucketCache persistence enabled, creates a table, writes and flushes data, and continues the workload while checking expected outcomes. Restarting the `RegionServer` immediately after the flush, before later operations warm `BucketCache`, can trigger a known restart bug (HBASE-28839): with BucketCache persistence enabled, shutdown persists cache metadata and the next startup restores it,

```java
@Test
public void testPrefetchPersistence() throws Exception {
  ...
  cluster = TEST_UTIL.startMiniHBaseCluster(option);
  Table table = TEST_UTIL.createTable(td, null);
  try {
    table.put(...);
    ...
    TEST_UTIL.flush(tableName);
    // Restart the HBase RegionServer
    // right after flush: HBASE-28839
  }
  ...
}
```

Figure 20: An existing HBase cluster test that can find a (previously known) restart bug (HBASE-28839) once a restart is injected.

and if the cache is empty at that point the persisted state can be inconsistent and the restarted `RegionServer` can crash while rebuilding cache state. Restarting later misses the bug, which shows that restart placement must expose restart-specific behavior the original oracle can observe. Existing techniques facing similar challenges often rely on program analysis with system-specific domain knowledge [27], [54]; we instead leverage the semantic reasoning of LLMs to infer program intent from implementation and identify restart-sensitive execution steps [66], [67].

Executability across restarts. Beyond restart-point selection, cluster test-based restart testing faces a second, often overlooked challenge: *reference staleness*. MiniCluster-style tests frequently cache cluster-derived objects (client handles, service stubs, node-local components used for inspection and assertions). During a graceful restart, shutdown tears down in-memory objects and the restarted node rejoins with fresh instances, so previously cached references can become invalid (e.g., pointing to closed connections) and cause transformed tests to fail before exercising meaningful post-restart behavior.

5.3 Design

This section describes Palingenesis's end-to-end workflow: transforming developer-written cluster tests into restart tests (§5.3.1), executing them with runtime support (§5.3.2), and inspecting restart failures (§5.3.3). Figure 21 summarizes the workflow. Starting from an existing cluster test, Palingenesis (1) assembles a prompt containing the test, relevant project context, and the unified restart API; (2) uses LLMs to inject multiple restart points and generate a restart test that follows the Palingenesis template; (3) compiles the generated restart test and iteratively repairs it by feeding compiler diagnostics back to the LLM; (4) executes the compiled restart test, activating exactly one restart point per run and dispatching the restart to a project restart adapter; (5) uses a stale-reference refresher to keep cluster-derived references valid after restart; and (6) reports failing executions to the failure inspector, which filters and groups failures and uses human-guided LLM debugging to decide whether a failure is a real restart bug. Because a graceful restart models a planned reboot, our default assumption is *invariant preservation*: once the restarted node rejoins and recovery stabilizes, the system should satisfy the same safety invariants checked by the original cluster test. We therefore do not apply heuristic oracle relaxation, which could mask restart bugs.

We make this assumption precise as a *restartability contract* that defines when a restart-injection point is semantically valid. A point is valid if, after a graceful shutdown and rejoin and once the cluster stabilizes, the test's original oracle still observes the durable, documented behavior the system is expected to preserve across a restart; equivalently, the transformed restart test passes when no restart bug is present. A failure at a valid point is a real restart bug. A failure at an invalid point is a *false alarm*: it reflects an assumption the original test makes that a restart legitimately breaks, falling into one of three classes. *Uninterrupted execution* assumes that a handle obtained before the restart (a session, stream, transaction, or client connection) remains usable afterward, which a graceful shutdown invalidates. *Transient in-memory state* assumes non-durable state (a

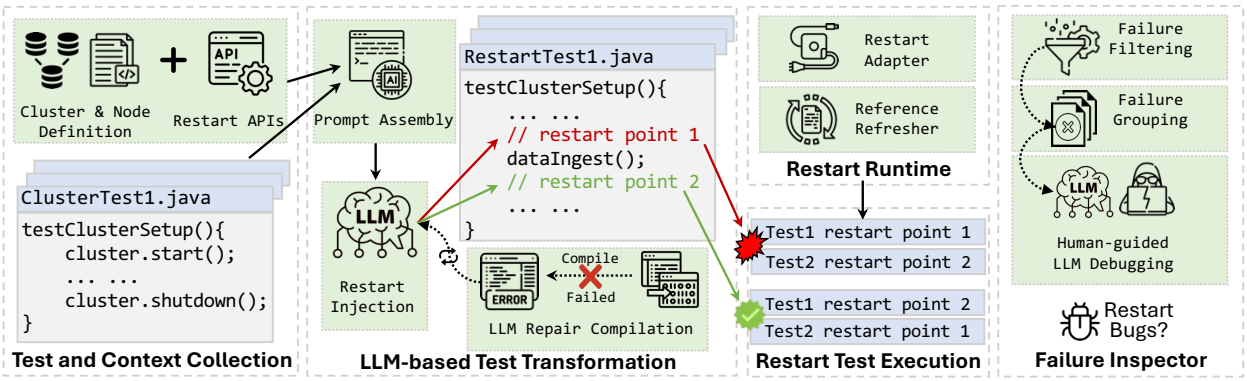


Figure 21: End-to-end workflow of Palingenesis. Green blocks highlight the key components.

cache, scheduler state, or in-memory leader state) that a restart legitimately resets. *Strict immediate availability* assumes the cluster is serving the instant the restart call returns, before recovery, leader election, or reconnection completes. The contract gives a decision procedure for excluding a point: exclude it if the test logic after the restart uses a stale unrefreshed handle, asserts before a recovery barrier, or observes only non-durable state. We apply this procedure manually during inspection (§5.3.3); automating it is future work.

5.3.1 Component 1: LLM-Based Cluster Test Transformer

Palingenesis's transformer takes a cluster test and produces a restart test by injecting multiple restart points into the test body, resolving three semantic constraints: test intent, cluster/node lifecycle and roles, and project-specific abstractions. Relying on LLMs alone to synthesize precise graceful restart procedures is brittle (different systems expose different cluster APIs, and small syntactic differences can correspond to large semantic differences), so Palingenesis constrains generated code to a unified restart API and uses a compilation-repair loop [128], [129].

Unified restart template API. Palingenesis exposes a unified API (Figure 22) for restart injection, designed to be shared across distributed systems with MiniCluster-style test frameworks. The API lets a restart test request a graceful restart of a specific node

```
Palingenesis.at("<label>") // restart point label
  .on(clusterVar)          // cluster instance
  .restart("<role>")       // node role
  .withIndex(<idx>)        // node index (if applicable)
  .execute();              // restart and wait for stabilization
```

Figure 22: Unified restart API used by transformed restart tests.

role, and Palingenesis translates the request into the project-specific shutdown and rejoin procedure. This API is the bridge between transformation and runtime: it standardizes how restart logic is expressed, so the transformer can reliably generate a fixed call pattern instead of synthesizing project-specific code, which substantially improves compilation success (§5.5.3). When the transformer injects multiple restart points into one restart test, Palingenesis executes *one* restart point per run, activating a single point (identified by its label and target) and treating the others as no-ops, which isolates the effect of each restart injection.

Prompt construction and restart injection. For each target cluster test, Palingenesis constructs an LLM prompt (Figure 23) that includes (1) the cluster test source, (2) project source needed to understand the cluster instance, node roles, and test helpers, and (3) the unified restart API the transformed test must use. Providing project context is critical because restart placement is position-sensitive: a useful restart point is a program point at a critical state-changing or stabilization step in the behavior under test, while still leaving the original test checks to execute after restart: typically at lifecycle boundaries where externally visible or durable state has advanced and the test has reached a stable checkpoint (e.g., after a write/flush/commit or after membership or leader-election stabilization). Based on these inputs, the LLM decides where to place restart points and which nodes to restart, then injects a labeled restart call at each selected point. Each restart point is identified by a string label and parameterized by a target node role and (when applicable) a node index; the generated code is *restart-aware but non-invasive*, so when no restart point is activated, all injected calls behave as no-ops and the restart test executes

```
You are a senior software engineer who will
transform mini cluster tests.
Goal: Given a cluster test, create a restart-aware
test that preserves the original workload and
oracles, while injecting multiple graceful restart
points using the unified restart API.

Inputs:
(A)  Original cluster test code:${CLUSTER_TEST_CODE}
(B)  Relevant project context (snippets to
     understand helpers and lifecycle):
     - Mini Cluster source code:${MINI_CLUSTER_CODE}
     - Helper utilities used by the test:${UTIL_CODE}
     - Nodes source code and definitions:${NODE_CODE}
(C)  Unified restart API (must be used verbatim):
     Palingenesis.at("<label>").on(<clusterVar>).res
     tart("<role>").withIndex(<idx>).execute();

Task:
1)   Identify the test phases (setup, workload,
     synchronization barriers, oracle checks,
     teardown) and cluster state-changing statements
2)   Choose K restart positions that are
     semantically meaningful and stable (e.g.,
     before or after cluster operations), and avoid
     invalid placements (e.g., before node/cluster
     is ready or after node/cluster is shutdown)
3)   Inject a restart at each selected position with:
     - a stable, descriptive label (e.g.,
     "after_flush", "before_write")
     - an explicit restart target (role + index)
4)   Ensure the new test keeps the same workloads,
     with only restarts injected

Output requirements:
- Output ONLY the compilable test code diff
- Do NOT modify the original test workloads; keep
edits minimal
- Do NOT invent new APIs; only use the provided
context and the restart API
```

Figure 23: Example prompt for cluster test transformation. Purple boxes are placeholders for the code snippets.

equivalently to the original test.

Compilation-repair loop. After transformation, Palingenesis compiles the generated restart test. If compilation fails, it feeds the compiler diagnostics, together with the current transformed code, back to the LLM and requests a repaired version. This loop continues until the restart test compiles or Palingenesis reaches the repair-round budget, after which the cluster test is reported as not transformable.

5.3.2 Component 2: Runtime Support for Restarts

Palingenesis provides runtime support for restart tests to perform restarts and keep the test logic executable after restart. This is necessary because executing a restart inside a test requires a multi-step, system-specific shutdown and rejoin, waiting for the cluster to recover, and maintaining the validity of cluster-derived references the remaining test logic depends on. Palingenesis provides (1) the unified restart template API used by the transformer, (2) a project restart adapter that maps unified API calls to each project's role-specific restart procedure, and (3) a stale-reference refresher.

Restart adapter. The restart adapter translates unified API calls into the system's shutdown and restart procedures. When deploying Palingenesis on a new project, the user supplies this adapter as a thin integration layer (typically a few hundred lines), specifying how to gracefully shut down, restart, and rejoin nodes of each role and how to wait for the cluster to return to a healthy state. In the MiniCluster setting, a graceful restart is an in-process shutdown/restart that executes the system's intended shutdown and rejoin logic: when the project provides MiniCluster restart APIs the adapter calls them, otherwise it directly invokes the role's shutdown and startup methods and runs health checks until the cluster stabilizes. This targets application-level restart semantics used in planned maintenance and rolling upgrades, rather than OS-level process signals or crash injection. Leader behavior after restart follows the project's own MiniCluster restart semantics: in most systems a graceful restart restores the same node to its prior role, and Palingenesis preserves this project-specific behavior rather than imposing its own. A node that legitimately rejoins as a follower, losing leadership per the system's own semantics, is therefore not flagged as a restart bug.

Stale-reference refresher. A key challenge in restart testing via cluster tests is reference staleness: cluster-derived objects obtained before restart can become invalid after

graceful shutdown cleans up resources. Palingenesis provides a stale-reference refresher that tracks such references and refreshes them after restart by re-invoking the same getter methods. The refresher is a pragmatic mechanism for *executability*: it keeps the remaining test logic running after restart so recovery code paths and original oracles can still be exercised, but does not claim semantics-preserving reconstruction for all objects. It is best-effort and fail-fast: it either refreshes a tracked reference by replaying a recorded derivation chain or throws an explicit refresh exception rather than continuing with a stale reference. Crucially, the refresher does not mask restart semantics. It reconstructs only getter-like, side-effect-free, cluster-derived references, that is, object identity and handles, and never system state or oracle outcomes, so the original workload and assertions are unchanged and a genuine restart fault still surfaces through the test's own oracle. When a reference cannot be reconstructed under this getter-like assumption, the refresher fails explicitly rather than substituting a guessed value. At a high level it has three phases: (i) detect the cluster root object, (ii) track cluster-derived references via *expression chains*, and (iii) replay these chains after restart to obtain fresh references. During execution, when instrumented code invokes a method on a tracked receiver, the refresher records the receiver, method, and arguments, and associates the return value with an expression chain that extends the receiver's chain; immutable arguments are snapshotted at capture time, while cluster-derived object arguments are represented by their own chains and refreshed first (yielding a dependency graph over chains). Figure 24 shows an example: the refresher detects `cluster` as a root and records chains for derived references, then on restart replays the chain for the `Table` reference against the new cluster root to obtain a fresh object. The refresher performs *lazy* refresh (replaying a chain only when the reference is accessed after restart) and uses a manually maintained blacklist of non-reconstructible types (e.g., I/O streams) that it never refreshes, failing explicitly instead of returning a guessed replacement.

```
Client c = cluster.getClient();// Get client from root cluster
Table t = c.getTable("table1");// Get table from client
```

Figure 24: Example of expression-chain capture for refreshing.

5.3.3 Component 3: Failure Inspector

The failure inspector decides whether failures from many restart test executions at different restart points correspond to real restart bugs. It reduces inspection cost by filtering noisy failures, grouping the rest, and debugging one representative execution per group. It conservatively filters three kinds of failures: Palingenesis framework exceptions (e.g., a stale-reference refresh exception, which indicates a framework issue rather than a system bug); pure timeouts whose logs lack fatal signals, which in this setting usually reflect convergence latency after restart rather than a bug; and resource-exhaustion failures due to testing-environment limits. It then groups the remaining failures by a *generalized stack trace* (the deepest exception and its first frame, with line numbers and run-specific tokens removed) so that failures with the same root cause collapse into one group. For each group, the inspector selects one representative execution and an LLM agent performs guided debugging to propose a root-cause hypothesis, which a human then verifies to decide whether the group reflects a real bug, judging it against the restartability contract (§5.3). The human inspector makes the bug-versus-false-alarm decision; the LLM only proposes root-cause hypotheses and never makes the final call.

5.4 Implementation

Palingenesis is implemented in Java with three components: a transformation driver that generates restart tests using LLMs, a runtime library that executes injected restarts and provides post-restart support, and an inspector that summarizes and groups failing executions. To integrate with existing build pipelines, we implement a Maven plugin that extends Maven Surefire [130]; for each transformed restart test it runs the test once for

every injected restart point and target node, enabling only the selected restart call. We provide per-project restart adapters that implement role-specific graceful shutdown, restart, and rejoin logic, adding on average 250 lines of code per project; adapters are discovered at runtime via Java's Service Provider Interface [131]. This line count is only a partial proxy for integration effort, which also requires understanding the project's restart APIs, node roles, health checks, and stabilization conditions. In practice most of this logic comes directly from the project's existing MiniCluster and node-lifecycle APIs, which already encapsulate role-specific shutdown, restart, rejoin, and recovery checks; the adapter mainly maps the unified API onto these entry points. With code-agent assistance, a developer can integrate Palingenesis with a new system in about one hour. Maintenance cost is also low: these APIs are stable, and commit history shows they change on the order of once every 2.5 years, so an adapter rarely needs updating as the system evolves. We implement the stale-reference refresher as a Java agent using bytecode instrumentation (ASM) [132].

5.5 Evaluation

Our evaluation answers the following research questions:

- **RQ1 (Bug finding):** How good is Palingenesis at finding restart bugs?
- **RQ2 (Transformation):** How effective and reproducible is the restart test transformation, and how sensitive is it to the choice of LLM?
- **RQ3 (Restart-point selection):** How does LLM-based restart injection compare to baselines?
- **RQ4 (Ablations):** How do the unified restart API and the reference refresher contribute?
- **RQ5 (Bug study):** What are the root causes of the discovered restart bugs?

Table 12: Summary of the evaluated projects.

Project	Version (commit)	# Cluster test methods
Accumulo	2.1.4 (e557fd9)	438
Cassandra	5.0.6 (4cba279)	518
Flink	2.2.0 (5a33689)	115
HBase	2.6.3 (01ac2f5)	903
HDFS	3.3.5 (706d882)	1,759
Yarn	3.3.5 (706d882)	160
ZooKeeper	3.9.4 (7246445)	566

5.5.1 Experimental Setup

We evaluate Palingenesis on seven open-source distributed systems: Accumulo, Cassandra, Flink, HBase, HDFS, Yarn, and ZooKeeper. For each system, we identify MiniCluster-style test methods and retain only methods that pass in the original test *without* injected restarts. Table 12 lists the evaluated version and commit of each project and the number of cluster test methods in the corpus.

The transformer runs the LLM with temperature 0.2 and up to three compilation-repair rounds per attempt, similar to prior LLM-based test transformation work [129], [133]. We evaluate three LLMs (Claude Opus 4.5, GPT-5.2, and Gemini 3 Pro), repeating the transformation three times per model to account for nondeterminism, and use Claude Opus 4.5 as the default for all RQs except RQ2. We compare Palingenesis against two baselines that use the same restart runtime: a budget-matched *Random* baseline that samples restart points uniformly from syntactically valid statement boundaries (using the same number of points per test as the LLM transformer, over 3 seeds), and a *Heuristic-based restart (HBR)* baseline that injects after common wait/synchronization helpers and at cluster lifecycle boundaries. We further use CrashFuzz [28], a state-of-the-art crash/reboot testing technique, as an external reference on its three original subjects (HBase, HDFS, and ZooKeeper), running 48 hours per project. We run all experiments on CloudLab [134] d430 machines (2×8-core CPU, 64 GB RAM), Ubuntu 22.

Table 13: Palingenesis main restart-testing results, per project. Columns report restart test methods, restart executions, failing executions, failures after filtering, failure reports, confirmed bugs, and false alarms.

Project	Restart tests	Restart execs	Failing execs	Fail. after filtering	Failure reports	Bugs	False alarms
Accumulo	438	1,580	71	28	4	3	1
Cassandra	518	2,162	91	36	4	3	1
Flink	115	244	22	8	3	1	2
HBase	903	7,318	182	73	4	3	1
HDFS	1,759	5,420	211	84	5	4	1
Yarn	160	248	18	8	2	0	2
ZooKeeper	566	1,344	57	22	4	3	1
Total	**4,459**	**18,316**	**652**	**259**	**26**	**17**	**9**

5.5.2 RQ1: How Many Bugs Does Palingenesis Find?

Across the seven projects, Palingenesis transforms 4,459 cluster test methods into compilable restart tests and executes 18,316 restart runs. Of these, 652 executions fail (3.6%); the failure inspector filters and groups them into 26 failure reports for inspection. In total, Palingenesis finds 17 distinct restart bugs across the projects, appearing in every project except Yarn. Table 13 reports the per-project breakdown; HDFS contributes the most bugs (4) and HBase and HDFS contribute the most restart executions (7,318 and 5,420), reflecting both more cluster test methods and more injected restart points per method (Yarn's 1.55 to HBase's 8.10 on average).

Grouping keeps the inspection burden practical: reducing 259 filtered failing executions to 26 reports, inspected at roughly half an hour per report, or 1.7 bugs per hour (17 bugs over 10 hours), with LLM assistance. To validate these labels, an LLM-based failure analyzer independently labeled each of the 26 failure reports as a real restart bug or a false alarm, using a fixed prompt and a fresh context per report. The analyzer received the restartability contract (§5.3) and the same materials we used, with our labels and the issue identifiers removed. It disagreed with us on 6 of the 26 reports, in every case labeling as a false alarm a report we labeled a real restart bug; the resulting Cohen's kappa is 0.56. Developers have since confirmed four of those six reports as real bugs

(one already fixed), which supports our labels and suggests that current LLMs remain conservative in failure inspection. A second inspector independently reviewed and agreed with all 26 labels. Compared against CrashFuzz on the three shared projects, CrashFuzz finds 2 overlapping bugs but misses 8 bugs that Palingenesis finds. Among these missed bugs, 3 (HBASE-29804, HDFS-17876, and ZOOKEEPER-5012) lie outside CrashFuzz's crash/reboot scope because they occur only on graceful shutdown paths, 2 (HDFS-17870 and ZOOKEEPER-5013) are exposed only in the projects' existing testing cluster infrastructure, and the rest were conceptually reachable but not explored in CrashFuzz's 48 hours-per-project campaign. This highlights the complementary value of Palingenesis's developer-written workloads and oracles, consistent with CrashFuzz's own note that "it is challenging to automatically construct good workloads for cloud systems" [28]; CrashFuzz also finds 3 additional crash-only bugs outside this chapter's graceful-restart target, which we do not count. Beyond the 17 bugs, Palingenesis also identifies 9 failure groups caused not by restart bugs but by original test assumptions that do not survive restart. Each violates one of the three unsupported-assumption classes of the restartability contract (§5.3): 4 groups are from *uninterrupted execution*, 2 from *transient in-memory state*, and 3 from *strict immediate availability*. Uninterrupted-execution failures reuse a pre-restart handle that the restart invalidates, such as a Cassandra client session that throws `NoHostAvailableException`, a ZooKeeper session that throws `ConnectionLoss`, or a stale Yarn `AMRMToken` after a `ResourceManager` restart. Transient-in-memory-state failures check non-durable state, such as an HBase test that expects a region-split state the restart does not preserve (and HBase correctly refuses to split). Strict-immediate-availability failures assert before recovery completes, such as Accumulo and HDFS tests that require servers registered the moment the restart returns. Unlike the pure timeouts the inspector filters, these failures carry a fatal signal and reach inspection, where they are judged false alarms against the contract. Applying Palingenesis to a new system can therefore include a one-time calibration step, applying the contract's decision procedure once per system to

Table 14: LLM comparison for restart test transformation (single-iteration, without the compilation-repair loop). Bold marks the best per column.

Model	Avg time per test (s)	Avg cost per test (USD)	Success rate	Total restart positions
Claude Opus 4.5	**20.5**	0.21	**96.5%**	**18,316**
GPT-5.2	41.2	0.19	95.1%	17,521
Gemini 3 Pro	86.5	**0.12**	94.2%	16,892

identify these oracle patterns and add them to a per-system filter list.

5.5.3 RQ2: Transformation Effectiveness and Reproducibility

For each LLM we run transformation three times and report success rate, latency, cost, and the number of restart points selected, using single-iteration results (without the compilation-repair loop) to isolate code-generation capability. All three models achieve high first-attempt compilation success (94.2%–96.5%), showing that the task is robust under Palingenesis's prompt structure, unified restart API, and template constraints. Claude Opus 4.5 has the best overall profile, with the highest success rate (96.5%) and lowest average latency (20.5 seconds), so we use it as the default. Gemini 3 Pro has the lowest API cost (0.12 USD per attempt) but the highest latency (86.5 seconds) and the lowest success rate (94.2%), and GPT-5.2 sits between the two (Table 14). The differences among strong models are small because the unified restart API narrows the valid output space.

LLM-guided restart-point selection is also highly reproducible. For Claude Opus 4.5, 91.6% of union restart points recur in all three runs and 98.3% in at least two; the other two models show similarly high recurrence. Failure reports and bugs are likewise stable: 20 of 26 failure reports recur in all three runs (the rest in two), and 14 of 17 bugs recur in all three runs. Table 15 reports the full recurrence breakdown.

Table 15: Recurrence across the three repeated runs in RQ2.

Model	All 3	Exactly 2	Exactly 1	Union	All 3 (%)	≥2
Claude Opus 4.5	16,780	1,218	318	18,316	91.6%	98.3%
GPT-5.2	15,690	1,470	361	17,521	89.6%	97.9%
Gemini 3 Pro	14,910	1,554	428	16,892	88.3%	97.5%

(a) Restart-point recurrence.

Artifact	All 3	Exactly 2	Exactly 1	Union	All 3 (%)	≥2
Failure reports	20	6	0	26	76.9%	100%
Bug groups	14	3	0	17	82.4%	100%

(b) Failure and bug finding recurrence.

5.5.4 RQ3: Restart-Point Selection Validation

RQ3 isolates the contribution of restart-point selection by comparing Palingenesis against the HBR and Random baselines. Across all projects, HBR finds 4 bugs (from 15 failure reports) and Random finds 4 (union), compared with 17 bugs for LLM-guided selection (Table 16, left and middle). Both baselines report fewer failure reports than LLM-guided selection (15 and 17.67 vs. 26), which indicates that triggering restart bugs requires injecting restarts at narrow points that demand semantic reasoning about test intent and implicit phase boundaries. For example, neither Random nor HBR finds HBASE-29805, which requires a node restart between enabling an HBase region split and submitting the split.

Estimating, for each of the 17 bugs, the size of its triggering window relative to all candidate statement boundaries, the average probability that a uniformly random restart-point choice discovers a given bug is only 7.7%, which explains why budget-matched random sampling often misses bugs that LLM-guided selection targets.

5.5.5 RQ4: Ablation Studies

No unified restart API. Removing the unified restart-API specification and template constraints, so the LLM must synthesize project-specific restart logic from context, sharply

Table 16: Restart test results for Heuristic-based restart (RQ3), the Random baseline (RQ3), and the ablation without the stale-reference refresher (RQ4b). Random reports the mean over 3 seeds and the sum of bugs found across all seeds (union).

	Heuristic-based restart			**Random (avg over 3 seeds)**		**No refresher**	
Project	Restart execs	Failure reports	Bugs	Failure reports	Bugs	Failure reports	Bugs
Accumulo	1,228	3	1	5.33	0.67	39	2
Cassandra	1,312	1	0	2.33	0.00	23	0
Flink	391	2	0	2.33	0.67	16	0
HBase	2,881	2	1	1.33	0.0	229	2
HDFS	4,772	3	1	2.67	0.67	178	2
Yarn	495	1	0	1.33	0.0	14	0
ZooKeeper	1,629	3	1	2.33	0.67	51	2
SUM	**12,708**	**15**	**4**	**17.67**	**4 (union)**	**550**	**8**

Table 17: RQ4a ablation without the unified restart APIs: average latency, cost, and transformation success rate per model.

Model	Avg time per test (s)	Avg cost per test (USD)	Success rate
Claude Opus 4.5	32.5	0.28	39.6%
GPT-5.2	53.2	0.24	42.1%
Gemini 3 Pro	106.4	0.19	38.4%

reduces transformer effectiveness: first-attempt compilation success drops from the 94–96% range to roughly 38.4%–42.1% (lowest for Gemini 3 Pro; Table 17), confirming that the unified API is a key enabler of reliable transformation.

No stale-reference refresher. Disabling the refresher substantially degrades restart-testing quality: 73.1% (13,392 of 18,316) of restart executions then fail (versus 3.6% with the refresher), and most become false positives caused by stale in-memory references rather than true restart bugs, often surfacing as `NullPointerException` or `IllegalStateException`. The number of failure reports rises sharply from 26 to 550. More importantly, 9 of the 17 bugs become masked without the refresher, and only 8 bugs, those that surface immediately during the restart process (graceful shutdown violations and reboot failures), remain detectable. Without the refresher, Palingenesis cannot reliably validate post-restart recovery correctness.

5.5.6 RQ5: Bug Study

We analyze the 17 bugs along two dimensions: when the failure occurs and where the buggy code resides. Table 18 lists the full inventory by issue, phase, symptom, and buggy location, together with which technique finds each bug. 3 bugs occur during shutdown, 5 during reboot/re-initialization, and 9 during post-restart recovery; 10 are in system code, 5 in cluster infrastructure, and 2 in test code. The coverage matrix shows that Palingenesis finds all 17 bugs, while the HBR and Random baselines each find only a few and CrashFuzz (run only on its three originally supported projects) finds fewer still, underscoring that triggering restart bugs requires injecting restarts at narrow, intent-aligned points.

We highlight one representative bug per category. For a *shutdown protocol violation* (ZOOKEEPER-5012), `QuorumPeer.shutdown()` unconditionally calls `zkDb.close()` when `zkDb` is uninitialized, causing a `NullPointerException`. For a *reboot failure* (HDFS-17870), restarting a `DataNode` via `MiniDFSCluster.restartDataNode()` triggers an out-of-bounds exception in `setDataNodeStorageCapacities`, which assumes the `DataNode` index still matches the `storageCapacities` array; the restart invalidates that correspondence, so the index can exceed the array length and the node fails to rejoin. For a *recovery-window race* (HBASE-29805), restarting a `RegionServer` between enabling a region split and submitting the split leaves the region `OPEN` with a temporarily `null` server location, leading to a `NullPointerException` in the split path.

5.5.7 Performance Overhead

The transformed restart tests are slower than the original cluster tests because each execution includes an explicit shutdown, restart, and post-restart stabilization phase. Figure 25 summarizes the per-project comparison; overall, the transformed restart tests incur an average execution overhead of 2.27× compared to the original cluster tests, raising the average per-test time from 18.28 to 41.47 seconds. The per-project slowdown ranges from 1.16× for Flink to 4.68× for ZooKeeper (from 2.91 to 13.64 seconds), which is highest

Table 18: Restart bug inventory by phase, symptom, and buggy location, with the technique that finds each bug (✓ found, ✗ not found, - not run). CrashFuzz is evaluated only on its three originally supported projects (HBase, HDFS, and ZooKeeper).

Issue	**Phase**	**Symptom**	Buggy loc.	Crash-Fuzz	Ran-dom	HBR	Palin-genesis
HBASE-29804	Shutdown	Resource not unregistered during shutdown	System code	✗	✗	✓	✓
HDFS-17876	Shutdown	Node shutdown missing resource checker	System code	✗	✗	✓	✓
ZOOKEEPER-5012	Shutdown	Node shutdown missing resource checker	System code	✗	✗	✓	✓
ACCUMULO-6071	Reboot	Missing node role during reboot	Cluster infra	-	✗	✗	✓
ACCUMULO-6070	Reboot	No checker for resource access during reboot	Cluster infra	-	✓	✗	✓
HBASE-29857	Reboot	Improper checker for resource access during reboot	System code	✗	✗	✗	✓
HDFS-17870	Reboot	Missing field checking during reboot initialization	Cluster infra	✗	✗	✗	✓
ZOOKEEPER-5013	Reboot	Inconsistent node constructor missing field initialization	Cluster infra	✗	✗	✗	✓
ACCUMULO-6050	Recovery	Incorrect waiting logic for recovery	Test code	-	✗	✓	✓
CASSANDRA-21128	Recovery	No checker for resource access after restart	System code	-	✗	✗	✓
CASSANDRA-21097	Recovery	Inconsistent assumption on data format after node restart	Cluster infra	-	✗	✗	✓
CASSANDRA-21096	Recovery	No checker for resource access after restart	Test code	-	✗	✗	✓
FLINK-38870	Recovery	Illegal application status after restart	System code	-	✓	✗	✓
HBASE-29805	Recovery	Accessing resources during recovery causes a crash	System code	✓	✗	✗	✓
HDFS-17860	Recovery	No checker for closed resource access	System code	✗	✗	✗	✓
HDFS-17863	Recovery	Data not visible after restart	System code	✓	✓	✗	✓
ZOOKEEPER-5011	Recovery	No checker for closed resource access	System code	✗	✓	✗	✓

because its original tests are relatively lightweight. Despite the overhead, the transformed tests remain practical to execute, with per-bug execution times ranging from 10.45 to 77.55 seconds.

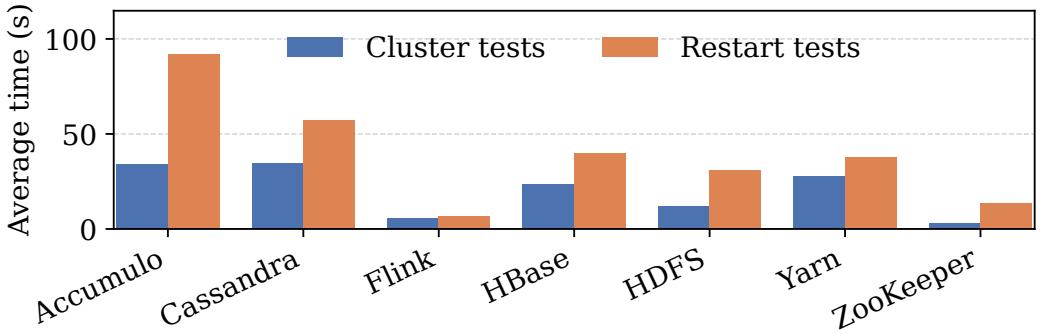


Figure 25: Average execution time of original cluster tests and transformed restart tests across the seven projects.

5.6 Discussion

Refresher limitations. The stale-reference refresher makes in-process restart testing practical (RQ4 shows 9 of 17 bugs become undetectable without it), but it cannot eliminate all restart-induced noise. It only refreshes references that flow through instrumented restart test bytecode, excludes non-reconstructible resources such as I/O streams, and replays recorded derivation chains, so nondeterministic or side-effecting chains may fail or be unsafe to refresh, in which case the refresher raises an explicit exception.

Oracle brittleness under restart. Oracle preservation is a principled design choice: a graceful restart models a planned shutdown and reboot, and after recovery stabilizes the system is expected to satisfy the same safety invariants as in steady-state execution (17,664 of 18,316 restart runs pass under the original tests). Unchanged oracles have two limits, however: they can miss *silent failures* that violate intended invariants without triggering existing assertions, and they can flag false alarms when tests assume uninterrupted execution, immediate availability, or specific timing. The second limit is exactly a violation of the restartability contract (§5.3): a false alarm occurs when a restart is injected at a point the original test never intended to support across a restart. Rather than weakening oracles to hide such failures, we keep them visible and rely on the contract to distinguish them from real restart bugs during inspection.

Detection scope and false negatives. Palingenesis is bounded by the coverage of the cluster tests it transforms: a restart bug on a code path no cluster test exercises is a false negative. It inherits the detection power of the original tests, augmented with crash and hang detection, and reports a restart bug only when a restart trips an existing oracle, a crash, or a hang. It can therefore miss silent failures that leave no such signal.

When to run restart testing. Because transformed tests are slower than the originals (§5.5.7), Palingenesis fits scheduled validation, such as nightly or weekly runs, and targeted pre-merge checks on restart-sensitive components, rather than a per-commit drop-in. When the testing budget is limited, a developer can sample restart scenarios, prioritizing changed code paths and restart-bug-prone windows.

5.7 Summary

Graceful restarts are routine in production distributed systems yet remain under-tested. This chapter frames graceful restart testing as an automated transformation task and presents Palingenesis, which uses LLMs to transform MiniCluster-style cluster tests into restart tests by injecting graceful restarts while preserving the original workloads and oracles, and provides the runtime support needed to execute these tests reliably under restarts. Across seven evaluated systems, Palingenesis transforms 4,459 cluster tests and runs 18,316 restart executions, uncovering 17 restart bugs: 6 fixed, 5 more confirmed, and the remainder pending. Where Chapter 2 through Chapter 4 reuse tests against configuration changes, Palingenesis reuses them against a lifecycle operation, the planned graceful restart, using runtime signals to keep the transformed tests executable.

Chapter 6 Related Work

6.1 Regression Test Selection

Regression test selection (RTS) has been studied for over 25 years since seminal work in the late 1990s [30], [135], with several surveys providing a broad overview [29], [31], [90], [91]. Early research focused on selecting as few tests as possible from the regression test suite, but later work focused on reducing the end-to-end regression testing time. While early techniques tracked test dependencies at fine granularity (e.g., basic blocks [30], [136]), over time the dependencies got coarser (e.g., methods [137], classes [32], [60], [61], [110], [138], and modules [33], [139]). RTS is widely used in practice [62], [63], [64], [88], [89], [140], [141]. uRTS (Chapter 3) differs from this body of work in that no existing RTS technique reasons about code and *configuration* changes collectively: existing techniques are designed either for code changes only [29] or for configuration changes only [41]. Test prioritization for configuration testing [57], [58] is complementary and discussed under configuration testing below.

6.2 Configuration Testing and Fuzzing

Configuration testing. Configuration testing validates software behavior under non-default configurations by running tests in the presence of configuration values and changes [41], [42], [57], [71], [142]. Ctest [41] introduced the configuration-test primitive (Chapter 2) by linking configuration parameters with developer-written unit tests; subsequent work improved its practicality through test prioritization [57] and regression test selection for configuration changes (uRTS, Chapter 3), and Ctest4J (Chapter 2) made writing, running, and selecting configuration tests practical for real Java projects. A separate line of work analyzes configuration parameters and configuration bugs through option extraction, parameter diagnosis, dependency analysis, and misconfiguration detection [6], [22], [40], [78], [112].

Configuration-aware regression testing. RTS for configuration-aware regression testing [94], [143] focuses on the *generation* of configurations and the *prioritization* of configurations during regression testing, while uRTS (Chapter 3) focuses on *production* configurations and the *selection of tests*. The prior work implicitly assumes that all regular tests can be used as configuration tests (i.e., $\hat{T} = T$) and aims to generate (or "select") configurations from the large configuration space. However, bugs that manifest for generated configurations may not manifest for real production configurations, and vice versa.

Configuration fuzzing. Configuration fuzzing automatically mutates configuration values to expose crashes and semantic failures, originally targeting configuration files and command-line interfaces [113], [115]. Recent techniques include ConfigFuzz [47] and POWER [49] for program options and ECFuzz [48] for large-scale cloud systems. Subsequent option-fuzzing work applies whitebox analysis [144], [145], LLM-guided option-combination prediction [50], [146], interleaved option-file fuzzing [147], coverage-guided dynamic configuration scheduling [148], differential performance fuzzing of options [149], variability-aware fuzzing [150], and fault injection through configuration dependencies in distributed systems [151]. ConfFuzz (Chapter 4) is a greybox configuration fuzzer that targets API-level configuration objects in highly configurable cloud systems, uses unit tests as fuzz drivers, and incorporates runtime parameter-usage information as a second source of guidance beyond code coverage.

Greybox fuzzing with richer guidance. Greybox fuzzing began with code-coverage guidance [43], [44], [45] and later incorporated additional signals when coverage alone did not capture the structure of the search space. Generator-based greybox fuzzers preserve input structure and validity through custom generators [118], [120], [152], [153]. Others use target distance [154], rare-path information [155], static or dynamic data-flow information [156], [157], and domain-specific runtime signals such as memory usage or

typestate [158], [159], and some learn mutation or seed-selection strategies from prior executions [160], [161], [162], [163]. ConfFuzz follows the same principle: when the interface exposes information that plain coverage does not capture (here, the set of configuration parameters exercised by each execution), the fuzzer should exploit it as guidance.

6.3 Lifecycle and Reliability Testing

Fault injection for distributed systems. Prior work has extensively explored fault injection for distributed systems. CrashFuzz [28] performs coverage-guided crash/reboot injection; DeMiner [122] predicts crash points from common-data-related operations; Legolas [123] exposes partial-failure bugs with inferred abstract states and fault hooks; DepState [125] targets phase-sensitive synchronization failures; and Mallory [124] applies adaptive greybox fuzzing over node faults and network partitions. Like broader crash-centric fault-injection work [14], [20], [27], [54], [55], these systems target abrupt crashes, partial failures, or external-harness fault injection rather than planned *graceful shutdown* completion and rejoin semantics, and they do not transform existing MiniCluster tests while preserving the original workloads and oracles. The distinction is fundamental: a graceful restart exercises the *planned-termination* path, draining in-flight requests, running cleanup, persisting metadata, transferring leadership and resources, passing health checks, and performing role-specific rejoin, whereas crash and reboot injection skips this path and tests unplanned-restart recovery. The two are complementary, and crash injection can miss bugs that occur only on the graceful shutdown and rejoin path. Palingenesis (Chapter 5) instead targets planned graceful restarts by reusing existing cluster tests as realistic workloads with built-in oracles.

Test transformation and LLM-based testing. This dissertation reuses developer-written tests to explore under-tested behaviors throughout: configuration changes (Chapter 2, Chapter 3, Chapter 4) and the graceful-restart lifecycle (Chapter 5). Other work

transforms developer tests to reach under-tested behaviors [48], [75], and recent LLM-driven work describes this style as *test amplification* [164], [165]. Unlike automated test-generation tools such as Randoop [166] and EvoSuite [167], which synthesize tests from scratch, Palingenesis reuses the domain knowledge in existing cluster tests to inject graceful restarts while preserving the original workloads and oracles. Pretrained code models support code understanding and transformation [66], [168], [169], [170], [171], and agentic systems have begun to tackle real-world software-engineering tasks [67], [68], [69]. Palingenesis applies these capabilities to a constrained transformation problem, restart-point selection, using a unified template API plus compilation-guided repair to reduce brittleness, aligning with findings that constrained workflows improve LLM reliability [128]. Recent LLM-based testing systems likewise show the viability of improving developer tests at industrial scale [133], using generation–validation–repair loops for executable tests [129], and guiding test creation with specific fault classes such as mutants [172].

Chapter 7 Conclusions and Future Work

7.1 Conclusions

Distributed systems are highly configurable and run through routine lifecycle operations, yet testing under-covers both: regression testing exercises code under a single default configuration and a steady-state lifecycle. This dissertation improves the reliability of configurable and distributed systems by reusing the tests developers already write and maintain to test code, configuration, and lifecycle changes, using runtime signals to make that reuse precise and efficient. It develops this idea through four lines of work: the first three target configuration changes, and the fourth targets a lifecycle operation, graceful restart.

We first present Ctest4J, a practical configuration testing framework for Java. A configuration test (Ctest) couples a configuration with a test that checks the code under it, catching both misconfigurations and configuration-exposed code bugs. Ctest4J lets developers write, generate, and run Ctests like ordinary unit tests, integrating with standard Java test frameworks. Across 12 open-source Java projects, Ctest4J runs Ctests 3.4× faster on average than prior Ctest tooling.

We next present uRTS, the first unified regression test selection technique that tests code changes under both default and production configurations and tests configuration changes against the latest code. uRTS drives selection from the runtime dependencies among tests, code, and configuration parameters, and avoids rerunning a test redundantly across many production configurations. On five large systems, uRTS reduces end-to-end testing time by 3.64× over running all tests.

We then present ConfFuzz, which reuses existing unit tests as fuzz drivers to find configuration-related bugs. ConfFuzz is parameter-aware: it tracks at runtime which configuration parameters a test exercises and concentrates mutation and fuzzing effort on those parameters. Using 4,838 unit tests as drivers across nine cloud systems, ConfFuzz

and its variants found 125 unique configuration-related bugs, of which 14 have been fixed and 12 further confirmed.

We finally present Palingenesis, which tests the restart lifecycle operation by reusing existing cluster tests. Palingenesis transforms a cluster test into a restart test, injecting a graceful restart at a meaningful point while preserving the test's workload and oracle, using a large language model to place the restart and runtime signals to refresh references the restart invalidates. Across seven distributed systems, Palingenesis transformed 4,459 cluster tests and found 17 restart bugs, of which 6 have been fixed and 5 further confirmed.

Together, these lines of work show that the tests developers already have, reused with runtime signals, can systematically expose reliability problems that arise from configuration and lifecycle changes.

7.2 Future Work

Many open challenges remain in testing configurable and distributed systems against code, configuration, and lifecycle changes. We discuss future directions that could continue along the lines of this dissertation.

7.2.1 Testing the Rolling-Upgrade Lifecycle Operation

Restart is one lifecycle operation (Chapter 5); rolling upgrade is another, and arguably harder. During a rolling upgrade, operators replace nodes one at a time, so the cluster runs a mix of old and new versions [11], [12], and failures arise from cross-version interaction rather than a single version's logic [26]. Reusing existing tests for rolling upgrade raises a new question that restart does not: a correct execution must hold across the mixed-version window, not just before and after a single restart. A promising direction is a differential oracle that contrasts an old-to-new rolling-upgrade run against old-to-old and new-to-new baselines, driven by trace-guided injection of the upgrade. This extends the lifecycle-testing direction this dissertation begins with restart.

7.2.2 Unifying Testing Across Code, Configuration, and Lifecycle Changes

This dissertation reuses existing tests against three kinds of change, but treats them with separate tools. In practice a single commit can change code, configuration, and the lifecycle behavior of a system at once, and the runtime signals each tool collects overlap: Ctest4J, uRTS, and ConfFuzz all track which configuration parameters a test exercises, and uRTS and Palingenesis both track runtime dependencies. A direction is to unify these signals into one analysis that, for a given change, selects the tests to rerun, the configurations to explore, and the lifecycle operations to inject. Such unification could avoid redundant instrumentation and surface bugs that only the interaction of change kinds exposes.

7.2.3 Reusing Tests for More Systems and More Lifecycle Operations

The evaluations in this dissertation target Java systems and the restart operation. The underlying idea, reusing existing tests guided by runtime signals, does not depend on either. Extending it to systems written in other programming languages requires instrumenting their configuration and lifecycle interfaces to collect the same signals, and extending it to other lifecycle operations, such as scaling, failover, and data migration, requires identifying the points at which each operation can be injected into an existing test without breaking its oracle. Large language models, which Palingenesis uses to place restarts, may help generalize this injection across operations and systems where the meaningful points are not obvious from static structure alone.

7.3 Closing Remarks

Reliability testing has long focused on code changes under a fixed configuration and a steady-state lifecycle. The configuration and lifecycle changes that systems undergo in production are tested far less, even though they are leading causes of failure. This dissertation shows that the tests developers already have are a resource for closing that gap: with the right runtime signals, the same tests can be reused to check code, configuration,

and lifecycle changes. We hope this perspective encourages testing that follows software not only as its code changes, but as it is configured and operated.

References

[1] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-Scale Cluster Management at Google with Borg," in *EuroSys*, 2015.

[2] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.

[3] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," in *USENIX ATC*, 2014.

[4] F. P. Junqueira, B. C. Reed, and M. Serafini, "Zab: High-performance Broadcast for Primary-backup Systems," in *DSN*, 2011.

[5] C. Tang, T. Kooburat, P. Venkatachalam, A. Chander, Z. Wen, A. Narayanan, P. Dowell, and R. Karl, "Holistic Configuration Management at Facebook," in *SOSP*, 2015.

[6] Q. Chen, T. Wang, O. Legunsen, S. Li, and T. Xu, "Understanding and Discovering Software Configuration Dependencies in Cloud and Datacenter Systems," in *ESEC/FSE*, 2020.

[7] T. Xu, L. Jin, X. Fan, Y. Zhou, S. Pasupathy, and R. Talwadker, "Hey, You Have Given Me Too Many Knobs! Understanding and Dealing with Over-Designed Configuration in System Software," in *ESEC/FSE*, 2015.

[8] M. Mukelabai, D. Nešić, S. Maro, T. Berger, and J.-P. Steghöfer, "Tackling Combinatorial Explosion: A Study of Industrial Needs and Practices for Analyzing Highly Configurable Systems," in *ASE*, 2018.

[9] Y. Zhang, H. He, O. Legunsen, S. Li, W. Dong, and T. Xu, "An Evolutionary Study of Configuration Design and Implementation in Cloud Systems," in *ICSE*, 2021.

[10] M. Sayagh, N. Kerzazi, B. Adams, and F. Petrillo, "Software Configuration Engineering in Practice: Interviews, Surveys, and Systematic Literature Review," *IEEE Transactions on Software Engineering*, vol. 46, no. 6, 2020.

[11] "Kubernetes Documentation: Rolling Update Deployment," Accessed: 2026-01-13. [Online]. Available: `https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#rolling-update-deployment`

[12] "HDFS Rolling Upgrade," Accessed: 2026-01-13. [Online]. Available: `https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsRollingUpgrade.html`

[13] H. S. Gunawi, M. Hao, R. O. Suminto, A. Laksono, A. D. Satria, J. Adityatama, and K. J. Eliazar, "Why Does the Cloud Stop Computing? Lessons from Hundreds of Service Outages," in *SoCC*, 2016.

[14] H. S. Gunawi, M. Hao, T. Leesatapornwongsa, T. Patana-anake, T. Do, J. Adityatama, K. J. Eliazar, A. Laksono, J. F. Lukman, V. Martin, and A. D. Satria, "What Bugs Live in the Cloud? A Study of 3000+ Issues in Cloud Systems," in *SoCC*, 2014.

[15] L. A. Barroso, J. Clidaras, and U. Hölzle, *The Datacenter as a Computer: An Introduction to the Design of Warehouse-scale Machines*, 2nd ed. Morgan & Claypool Publishers, 2013.

[16] B. Maurer, "Fail at Scale: Reliability in the Face of Rapid Change," *Communications of the ACM*, vol. 58, no. 11, 2015.

[17] D. Oppenheimer, A. Ganapathi, and D. A. Patterson, "Why Do Internet Services Fail, and What Can Be Done About It?" in *USITS*, 2003.

[18] S. Mehta, R. Bhagwan, R. Kumar, B. Ashok, C. Bansal, C. Maddila, C. Bird, S. Asthana, and A. Kumar, "Rex: Preventing Bugs and Misconfiguration in Large Services Using Correlated Change Analysis," in *NSDI*, 2020.

[19] T. Xu and Y. Zhou, "Systems Approaches to Tackling Configuration Errors: A Survey," *ACM Computing Surveys*, vol. 47, no. 4, 2015.

[20] D. Yuan, Y. Luo, X. Zhuang, G. R. Rodrigues, X. Zhao, Y. Zhang, P. U. Jain, and M. Stumm, "Simple Testing Can Prevent Most Critical Failures: An Analysis of Production Failures in Distributed Data-Intensive Systems," in *OSDI*, 2014.

[21] Z. Yin, X. Ma, J. Zheng, Y. Zhou, L. N. Bairavasundaram, and S. Pasupathy, "An Empirical Study on Configuration Errors in Commercial and Open Source Systems," in *SOSP*, 2011.

[22] T. Xu, J. Zhang, P. Huang, J. Zheng, T. Sheng, D. Yuan, Y. Zhou, and S. Pasupathy, "Do Not Blame Users for Misconfigurations," in *SOSP*, 2013.

[23] K. Nagaraja, F. Oliveira, R. Bianchini, R. P. Martin, and T. D. Nguyen, "Understanding and Dealing with Operator Mistakes in Internet Services," in *OSDI*, 2004.

[24] A. Rabkin and R. Katz, "How Hadoop Clusters Break," *IEEE Software*, vol. 30, no. 4, pp. 88–94, 2013.

[25] L. A. Barroso, U. Hölzle, and P. Ranganathan, *The Datacenter as a Computer: Designing Warehouse-Scale Machines*. Morgan & Claypool Publishers, 2018.

[26] Y. Zhang, J. Yang, Z. Jin, U. Sethi, K. Rodrigues, S. Lu, and D. Yuan, "Understanding and Detecting Software Upgrade Failures in Distributed Systems," in *SOSP*, 2021.

[27] J. Lu, C. Liu, L. Li, X. Feng, F. Tan, J. Yang, and L. You, "CrashTuner: Detecting Crash-Recovery Bugs in Cloud Systems via Meta-Info Analysis," in *SOSP*, 2019.

[28] Y. Gao, W. Dou, D. Wang, W. Feng, J. Wei, H. Zhong, and T. Huang, "Coverage Guided Fault Injection for Cloud Systems," in *ICSE*, 2023.

[29] S. Yoo and M. Harman, "Regression Testing Minimisation, Selection and Prioritization: A Survey," *Software Testing, Verification, and Reliability*, vol. 22, no. 2, 2012.

[30] G. Rothermel and M. J. Harrold, "A Safe, Efficient Regression Test Selection Technique," *ACM Transactions on Software Engineering and Methodology*, vol. 6, no. 2, 1997.

[31] E. Engström, P. Runeson, and M. Skoglund, "A Systematic Review on Regression Test Selection Techniques," *Information and Software Technology*, vol. 52, no. 1, 2010.

[32] M. Gligoric, L. Eloussi, and D. Marinov, "Practical Regression Test Selection with Dynamic File Dependencies," in *ISSTA*, 2015.

[33] A. Shi, P. Zhao, and D. Marinov, "Understanding and Improving Regression Test Selection in Continuous Integration," in *ISSRE*, 2019.

[34] P. Huang, W. J. Bolosky, A. Sigh, and Y. Zhou, "ConfValley: A Systematic Configuration Validation Framework for Cloud Services," in *EuroSys*, 2015.

[35] J. Zhang, R. Piskac, E. Zhai, and T. Xu, "Static Detection of Silent Misconfigurations with Deep Interaction Analysis," in *OOPSLA*, 2021.

[36] S. Nadi, T. Berger, C. Kästner, and K. Czarnecki, "Mining Configuration Constraints: Static Analyses and Empirical Results," in *ICSE*, 2014, pp. 140–151.

[37] A. Whitaker, R. S. Cox, and S. D. Gribble, "Configuration Debugging as Search: Finding the Needle in the Haystack," in *OSDI*, 2004, pp. 77–90.

[38] M. Attariyan and J. Flinn, "Automating Configuration Troubleshooting with Dynamic Information Flow Analysis," in *OSDI*, 2010.

[39] Y.-Y. Su, M. Attariyan, and J. Flinn, "AutoBash: Improving Configuration Management with Operating System Causality Analysis," in *SOSP*, 2007, pp. 237–250.

[40] T. Xu, X. Jin, P. Huang, Y. Zhou, S. Lu, L. Jin, and S. Pasupathy, "Early Detection of Configuration Errors to Reduce Failure Damage," in *OSDI*, 2016.

[41] X. Sun, R. Cheng, J. Chen, E. Ang, O. Legunsen, and T. Xu, "Testing Configuration Changes in Context to Prevent Production Failures," in *OSDI*, 2020.

[42] T. Xu and O. Legunsen, "Configuration Testing: Testing Configuration Values as Code and with Code," *arXiv:1905.12195*, 2019.

[43] M. Zalewski, "AFL," `http://lcamtuf.coredump.cx/afl`, 2014.

[44] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based Greybox Fuzzing as Markov Chain," in *CCS*, 2016.

[45] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "AFL++: Combining Incremental Steps of Fuzzing Research," in *WOOT*, 2020.

[46] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating Fuzz Testing," in *CCS*, 2018, pp. 2123–2138.

[47] Z. Zhang, G. Klees, E. Wang, M. Hicks, and S. Wei, "Fuzzing Configurations of Program Options," *ACM Transactions on Software Engineering and Methodology*, 2023.

[48] J. Li, S. Li, K. Li, F. Luo, H. Yu, S. Li, and X. Li, "ECFuzz: Effective Configuration Fuzzing for Large-Scale Systems," in *ICSE*, 2024.

[49] A. Lee, I. Ariq, Y. Kim, and M. Kim, "POWER: Program Option-Aware Fuzzer for High Bug Detection Ability," in *ICST*, 2022.

[50] D. Wang, G. Zhou, L. Chen, D. Li, and Y. Miao, "ProphetFuzz: Fully Automated Prediction and Fuzzing of High-Risk Option Combinations with Only Documentation via Large Language Model," in *CCS*, 2024.

[51] "Kubernetes Documentation: Pod Termination," Accessed: 2026-01-13. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#pod-termination

[52] "ZooKeeper Guide," Accessed: 2026-01-13. [Online]. Available: https://zookeeper.apache.org/doc/current/zookeeperAdmin.html

[53] H. S. Gunawi, T. Do, P. Joshi, P. Alvaro, J. M. Hellerstein, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau, K. Sen, and D. Borthakur, "Fate and Destini: A Framework for Cloud Recovery Testing," in *NSDI*, 2011.

[54] P. Alvaro, J. Rosen, and J. M. Hellerstein, "Lineage-driven Fault Injection," in *SIGMOD*, 2015.

[55] A. Basiri, N. Behnam, R. De Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, "Chaos Engineering," *IEEE Software*, 2016.

[56] T. Leesatapornwongsa, M. Hao, P. Joshi, J. F. Lukman, and H. S. Gunawi, "SAMC: Semantic-Aware Model Checking for Fast Discovery of Deep Bugs in Cloud Systems," in *OSDI*, 2014, pp. 399–414.

[57] R. Cheng, L. Zhang, D. Marinov, and T. Xu, "Test-Case Prioritization for Configuration Testing," in *ISSTA*, 2021.

[58] R. Cheng, "Regression Test Prioritization for Modern Software," Ph.D. dissertation, University of Illinois Urbana-Champaign, 2025.

[59] M. Gligoric, L. Eloussi, and D. Marinov, "Ekstazi: Lightweight Test Selection," in *ICSE (Demo)*, 2015.

[60] O. Legunsen, F. Hariri, A. Shi, Y. Lu, L. Zhang, and D. Marinov, "An Extensive Study of Static Regression Test Selection in Modern Software Evolution," in *FSE*, 2016.

[61] L. Zhang, "Hybrid Regression Test Selection," in *ICSE*, 2018.

[62] S. Elbaum, G. Rothermel, and J. Penix, "Techniques for Improving Regression Testing in Continuous Integration Development Environments," in *FSE*, 2014.

[63] A. Memon, Z. Gao, B. Nguyen, S. Dhanda, E. Nickell, R. Siemborski, and J. Micco, "Taming Google-Scale Continuous Testing," in *ICSE-SEIP*, 2017.

[64] M. Machalica, A. Samylkin, M. Porth, and S. Chandra, "Predictive Test Selection," in *ICSE-SEIP*, 2019.

[65] B. Jeong, J. Jang, H. Yi, J. Moon, J. Kim, I. Jeon, T. Kim, W. Shim, and Y. Hwang, "UTopia: Automatic Generation of Fuzz Driver Using Unit Tests," in *S&P*, 2023.

[66] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating Large Language Models Trained on Code," *arXiv:2107.03374*, 2021.

[67] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," in *NeurIPS*, 2024.

[68] Z. Nan, Z. Guo, K. Liu, and X. Xia, "Test Intention Guided LLM-Based Unit Test Generation," in *ICSE*, 2025.

[69] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" in *ICLR*, 2024.

[70] S. Wang, X. Lian, Q. Li, D. Marinov, and T. Xu, "Ctest4J: A Practical Configuration Testing Framework for Java," in *FSE Companion*, 2024.

[71] S. Wang, X. Lian, D. Marinov, and T. Xu, "Test Selection for Unified Regression Testing," in *ICSE*, 2023.

[72] S. Wang, H. Wang, D. Marinov, T. Xu, and Y. Zhang, "ConfFuzz: Parameter-Aware Greybox Fuzzing for Configurable Cloud Systems," in *ASE*, 2026.

[73] S. Wang, Y. Chen, D. Marinov, and Y. Zhang, "Graceful Restart Testing for Distributed Systems via LLM-Based Test Transformation," in *ISSRE*, 2026.

[74] J. Zhang, L. Renganarayana, X. Zhang, N. Ge, V. Bala, T. Xu, and Y. Zhou, "EnCore: Exploiting System Environment and Correlation Information for Misconfiguration Detection," in *ASPLOS*, 2014.

[75] S. Ma, F. Zhou, M. D. Bond, and Y. Wang, "Finding Heterogeneous-Unsafe Configuration Parameters in Cloud Systems," in *EuroSys*, 2021.

[76] "openctest," `https://github.com/xlab-uiuc/openctest`, 2022.

[77] M. Sayagh, Z. Dong, A. Andrzejak, and B. Adams, "Does the Choice of Configuration Framework Matter for Developers? Empirical Study on 11 Java Configuration Frameworks," in *SCAM*, 2017.

[78] A. Rabkin and R. Katz, "Static Extraction of Program Configuration Options," in *ICSE*, 2011.

[79] G. Kiczales, E. Hilsdale, J. Hugunin, M. Kersten, J. Palm, and W. G. Griswold, "An Overview of AspectJ," in *ECOOP*, 2001.

[80] T. Savor, M. Douglas, M. Gentili, L. Williams, K. Beck, and M. Stumm, "Continuous Deployment at Facebook and OANDA," in *ICSE*, 2016.

[81] C. Parnin, E. Helms, C. Atlee, H. Boughton, M. Ghattas, A. Glover, J. Holman, J. Micco, B. Murphy, T. Savor, M. Stumm, S. Whitaker, and L. Williams, "The Top 10 Adages in Continuous Deployment," *IEEE Software*, vol. 34, no. 3, 2017.

[82] C. Rossi, E. Shibley, S. Su, K. Beck, T. Savor, and M. Stumm, "Continuous Deployment of Mobile Software at Facebook," in *FSE*, 2016.

[83] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, "Usage, Costs, and Benefits of Continuous Integration in Open-Source Projects," in *ASE*, 2016.

[84] N. Tillmann and W. Schulte, "Parameterized Unit Tests," in *ESEC/FSE*, 2005.

[85] N. Tillmann and W. Schulte, "Unit Tests Reloaded: Parameterized Unit Testing with Symbolic Execution," *IEEE Software*, vol. 23, no. 4, 2006.

[86] R. Potvin and J. Levenberg, "Why Google Stores Billions of Lines of Code in a Single Repository," *Communications of the ACM*, vol. 59, no. 7, 2016.

[87] C. Jaspan, M. Jorde, A. Knight, C. Sadowski, E. K. Smith, C. Winter, and E. Murphy-Hill, "Advantages and Disadvantages of a Monolithic Repository: A Case Study at Google," in *ICSE*, 2018.

[88] Z. Mi, "Mobile Performance: Tooling Infrastructure at Facebook," `https://engineering.fb.com/2015/04/10/developer-tools/mobile-performance-tooling-infrastructure-at-facebook`, 2015.

[89] C. Leong, A. Singh, J. Micco, M. Papadakis, and Y. le Traon, "Assessing Transition-based Test Selection Algorithms at Google," in *ICSE-SEIP*, 2019.

[90] S. Biswas, R. Mall, M. Satpathy, and S. Sukumaran, "Regression Test Selection Techniques: A Survey," *Informatica*, vol. 35, 2011.

[91] R. Kazmi, D. N. A. Jawawi, R. Mohamad, and I. Ghani, "Effective Regression Test Case Selection: A Systematic Literature Review," *ACM Computing Surveys*, vol. 50, no. 2, 2017.

[92] H. Srikanth, M. B. Cohen, and X. Qu, "Reducing Field Failures in System Configurable Software: Cost-Based Prioritization," in *ISSRE*, 2009.

[93] C. H. P. Kim, D. Marinov, S. Khurshid, D. Batory, S. Souto, P. Barros, and M. D'Amorim, "SPLat: Lightweight Dynamic Analysis for Reducing Combinatorics in Testing Configurable Systems," in *ESEC/FSE*, 2013.

[94] X. Qu, M. B. Cohen, and G. Rothermel, "Configuration-Aware Regression Testing: An Empirical Study of Sampling and Prioritization," in *ISSTA*, 2008.

[95] C. Henard, M. Papadakis, G. Perrouin, J. Klein, P. Heymans, and Y. L. Traon, "Bypassing the Combinatorial Explosion: Using Similarity to Generate and Prioritize T-Wise Test Configurations for Software Product Lines," *IEEE Transactions on Software Engineering*, vol. 40, no. 7, 2014.

[96] F. Medeiros, C. Kästner, M. Ribeiro, R. Gheyi, and S. Apel, "A Comparison of 10 Sampling Algorithms for Configurable Systems," in *ICSE*, 2016.

[97] L. Keller, P. Upadhyaya, and G. Candea, "ConfErr: A Tool for Assessing Resilience to Human Configuration Errors," in *DSN*, 2008.

[98] C. Brindescu, M. Codoban, S. Shmarkatiuk, and D. Dig, "How Do Centralized and Distributed Version Control Systems Impact Software Changes?" in *ICSE*, 2014.

[99] M. Lillack, C. Kästner, and E. Bodden, "Tracking Load-time Configuration Options," *IEEE Transactions on Software Engineering*, vol. 44, no. 12, 2018.

[100] M. Lillack, C. Kästner, and E. Bodden, "Tracking Load-time Configuration Options," in *ASE*, 2014.

[101] A. Rabkin and R. Katz, "Precomputing Possible Configuration Error Diagnosis," in *ASE*, 2011.

[102] F. Behrang, M. B. Cohen, and A. Orso, "Users Beware: Preference Inconsistencies Ahead," in *ESEC/FSE*, 2015.

[103] S. Zhang and M. D. Ernst, "Automated Diagnosis of Software Configuration Errors," in *ICSE*, 2013.

[104] S. Zhang and M. D. Ernst, "Which Configuration Option Should I Change?" in *ICSE*, 2014.

[105] D. Elsner, F. Hauer, A. Pretschner, and S. Reimer, "Empirically Evaluating Readily Available Information for Regression Test Optimization in Continuous Integration," in *ISSTA*, 2021.

[106] T. Xu and D. Marinov, "Mining Container Image Repositories for Software Configuration and Beyond," in *ICSE-NIER*, 2018.

[107] "Azure Linux Virtual Machines Pricing," `https://azure.microsoft.com/en-us/pricing/details/virtual-machines/linux/#pricing`.

[108] A. Celik, M. Vasic, A. Milicevic, and M. Gligoric, "Regression Test Selection Across JVM Boundaries," in *ESEC/FSE*, 2017.

[109] C. Zhu, O. Legunsen, A. Shi, and M. Gligoric, "A Framework for Checking Regression Test Selection Tools," in *ICSE*, 2019.

[110] A. Shi, M. Hadzi-Tanovic, L. Zhang, D. Marinov, and O. Legunsen, "Reflection-Aware Static Regression Test Selection," in *OOPSLA*, 2019.

[111] A. Nanda, S. Mani, S. Sinha, M. J. Harrold, and A. Orso, "Regression Testing in the Presence of Non-Code Changes," in *ICST*, 2011.

[112] T. Wang, Z. Jia, S. Li, S. Zheng, Y. Yu, E. Xu, S. Peng, and X. Liao, "Understanding and Detecting On-The-Fly Configuration Bugs," in *ICSE*, 2023.

[113] H. Dai, C. Murphy, and G. Kaiser, "CONFU: Configuration Fuzzing Testing Framework for Software Vulnerability Detection," *International Journal of Secure Software Engineering*, 2010.

[114] A. Zeller, "Yesterday, My Program Worked. Today, It Does Not. Why?" in *ESEC/FSE*, 1999.

[115] A. Zeller, R. Gopinath, M. Böhme, G. Fraser, and C. Holler, "The Fuzzing Book: Testing Configurations," `https://www.fuzzingbook.org/html/ConfigurationFuzzer.html`, 2021.

[116] H. S. Gunawi, C. Rubio-González, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau, and B. Liblit, "EIO: Error Handling is Occasionally Correct," in *FAST*, 2008.

[117] S. Thummalapenta, M. R. Marri, T. Xie, N. Tillmann, and J. de Halleux, "Retrofitting Unit Tests for Parameterized Unit Testing," in *FASE*, 2011.

[118] R. Padhye, C. Lemieux, and K. Sen, "JQF: Coverage-Guided Property-Based Testing in Java," in *ISSTA*, 2019.

[119] V. Varslavans, "RgxGen: Generate Matching and Non-Matching Strings," `https://github.com/curious-odd-man/RgxGen/`, 2024.

[120] R. Padhye, C. Lemieux, K. Sen, M. Papadakis, and Y. Le Traon, "Semantic Fuzzing with Zest," in *ISSTA*, 2019.

[121] R. Chandra, R. Lefever, M. Cukier, and W. Sanders, "Loki: A State-Driven Fault Injector for Distributed Systems," in *DSN*, 2000.

[122] Y. Gao, D. Wang, Q. Dai, W. Dou, and J. Wei, "Common Data Guided Crash Injection for Cloud Systems," in *ICSE Companion*, 2022.

[123] H. Wu, J. Pan, and P. Huang, "Efficient Exposure of Partial Failure Bugs in Distributed Systems with Inferred Abstract States," in *NSDI*, 2024.

[124] R. Meng, G. Pîrlea, A. Roychoudhury, and I. Sergey, "Greybox Fuzzing of Distributed Systems," in *CCS*, 2023.

[125] C. Fang, J. Liang, Z. Wu, J. Fu, Z. Jia, C. Huang, Y. Jiang, and S. Li, "DepState: Detecting Synchronization Failure Bugs in Distributed Database Management Systems," in *ISSTA*, 2025.

[126] "HDFS MiniDFSCluster," Accessed: 2026-01-13. [Online]. Available: `https://github.com/apache/hadoop/blob/trunk/hadoop-hdfs-project/hadoop-hdfs/src/test/java/org/apache/hadoop/hdfs/MiniDFSCluster.java`

[127] "HBase HBaseTestingUtility," Accessed: 2026-01-13. [Online]. Available: `https://github.com/apache/hbase/blob/master/hbase-testing-util/src/main/java/org/apache/hadoop/hbase/HBaseTestingUtility.java`

[128] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying LLM-Based Software Engineering Agents," in *FSE*, 2025.

[129] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, "ChatUniTest: A Framework for LLM-based Test Generation," in *FSE Companion*, 2024.

[130] The Apache Software Foundation, "Maven Surefire Plugin," 2025, Accessed: 2026-01-25. [Online]. Available: `https://maven.apache.org/surefire/maven-surefire-plugin/index.html`

[131] Oracle, "Class ServiceLoader," Java Platform, Standard Edition Documentation, 2025, Accessed: 2026-01-25. [Online]. Available: `https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ServiceLoader.html`

[132] E. Bruneton, R. Lenglet, and T. Coupaye, "ASM: A Code Manipulation Tool to Implement Adaptable Systems," in *Adaptable and Extensible Component Systems*, 2002.

[133] N. Alshahwan, J. Chheda, A. Finogenova, B. Gokkaya, M. Harman, I. Harper, A. Marginean, S. Sengupta, and E. Wang, "Automated Unit Test Improvement Using Large Language Models at Meta," in *FSE Companion*, 2024.

[134] D. Duplyakin, R. Ricci, A. Maricq, G. Wong, J. Duerig, E. Eide, L. Stoller, M. Hibler, D. Johnson, K. Webb, A. Akella, K. Wang, G. Ricart, L. Landweber, C. Elliott, M. Zink, E. Cecchet, S. Kar, and P. Mishra, "The Design and Operation of CloudLab," in *USENIX ATC*, 2019.

[135] G. Rothermel and M. Harrold, "Analyzing Regression Test Selection Techniques," *IEEE Transactions on Software Engineering*, vol. 22, no. 8, 1996.

[136] M. J. Harrold, J. A. Jones, T. Li, D. Liang, A. Orso, M. Pennings, S. Sinha, S. A. Spoon, and A. Gujarathi, "Regression Test Selection for Java Software," in *OOPSLA*, 2001.

[137] L. Zhang, M. Kim, and S. Khurshid, "FaultTracer: A Change Impact and Regression Fault Analysis Tool for Evolving Java Programs," in *FSE*, 2012.

[138] A. Orso, N. Shi, and M. J. Harrold, "Scaling Regression Testing to Large Software Systems," in *FSE*, 2004.

[139] M. Vasic, Z. Parvez, A. Milicevic, and M. Gligoric, "File-level vs. Module-level Regression Test Selection for .NET," in *ESEC/FSE*, 2017.

[140] A. Shi, S. Thummalapenta, S. K. Lahiri, N. Bjorner, and J. Czerwonka, "Optimizing Test Placement for Module-level Regression Testing," in *ICSE*, 2017.

[141] D. Elsner, S. Kacianka, S. Lipp, A. Pretschner, A. Habermann, M. Graber, and S. Reimer, "BinaryRTS: Cross-language Regression Test Selection for C++ Binaries in CI," in *ICST*, 2023.

[142] C. Xiang, L. Zhong, E. Mugnier, N. Nguyen, Y. Zhou, and T. Xu, "Testing Access-Control Configuration Changes for Web Applications," *arXiv:2505.12770*, 2025.

[143] M. B. Cohen, J. Snyder, and G. Rothermel, "Testing across Configurations: Implications for Combinatorial Testing," in *A-MOST*, 2006.

[144] K. Wang, M. Chen, L. He, P. Su, Y. Cai, J. Chen, B. Zhang, C. Feng, and C. Tang, "OSmart: Whitebox Program Option Fuzzing," in *CCS*, 2024.

[145] K. Wang, M. Chen, L. He, Y. Wang, P. Su, J. Chen, Y. Cai, B. Zhang, C. Feng, C. Tang, and G. Peng, "OSmartPro: A Large Language Model-Assisted Option Fuzzing Approach," *Cybersecurity*, 2026.

[146] M. Shiraishi, Y. Cao, and T. Shinagawa, "PILOT: Command-Line Interface Fuzzing via Path-Guided, Iterative Large Language Model Prompting," in *S&P*, 2026.

[147] A. Lee, Y. Choi, S. Hong, Y. Kim, K. Cho, and M. Kim, "ZigZagFuzz: Interleaved Fuzzing of Program Options and Files," *ACM Transactions on Software Engineering and Methodology*, 2025.

[148] L. Chu, M. Huang, X. Li, and Q. Yan, "OptionFuzz: The Fuzzer with Coverage-Guided Dynamic Configuration Scheduling," in *ICCCS*, 2025.

[149] H. Baik, C. Yang, V. Vikram, P. Jamshidi, R. Padhye, and C. Kaestner, "Differential Performance Fuzzing of Configuration Options," in *SBFT*, 2025.

[150] M. T. Ahmed, A. Dev, and S. Wei, "Variability-Aware Fuzzing," in *ICSE*, 2026.

[151] Y. Chen, F. Ma, Y. Zhou, Z. Yan, and Y. Jiang, "CAFault: Enhance Fault Injection Technique in Practical Distributed Systems via Abundant Fault-Dependent Configurations," in *USENIX ATC*, 2025.

[152] C. Holler, K. Herzig, and A. Zeller, "Fuzzing with Code Fragments," in *USENIX Security*, 2012.

[153] M. Olsthoorn, A. van Deursen, and A. Panichella, "Generating Highly-structured Input Data by Combining Search-based Testing and Grammar-based Fuzzing," in *ASE*, 2020.

[154] H. Chen, Y. Xue, Y. Li, B. Chen, X. Xie, X. Wu, and Y. Liu, "Hawkeye: Towards a Desired Directed Grey-box Fuzzer," in *CCS*, 2018.

[155] S. Saha, L. Sarker, M. Shafiuzzaman, C. Shou, A. Li, G. Sankaran, and T. Bultan, "Rare Path Guided Fuzzing," in *ISSTA*, 2023.

[156] J. Choi, D. Kim, S. Kim, G. Grieco, A. Groce, and S. K. Cha, "SMARTIAN: Enhancing Smart Contract Fuzzing with Static and Dynamic Data-Flow Analyses," in *ASE*, 2021.

[157] T. E. Kim, J. Choi, K. Heo, and S. K. Cha, "DAFL: Directed Grey-box Fuzzing Guided by Data Dependency," in *USENIX Security*, 2023.

[158] C. Wen, H. Wang, Y. Li, S. Qin, Y. Liu, Z. Xu, H. Chen, X. Xie, G. Pu, and T. Liu, "MemLock: Memory Usage Guided Fuzzing," in *ICSE*, 2020.

[159] H. Wang, X. Xie, Y. Li, C. Wen, Y. Li, Y. Liu, S. Qin, H. Chen, and Y. Sui, "Typestate-guided Fuzzer for Discovering Use-After-Free Vulnerabilities," in *ICSE*, 2020.

[160] B. Mathis, R. Gopinath, and A. Zeller, "Learning Input Tokens for Effective Fuzzing," in *ISSTA*, 2020.

[161] S. Li, Y. Lin, X. Xie, Y. Li, X. Li, W. Ge, Y. Liu, and J. S. Dong, "A First Look at the Effect of Deep Learning in Coverage-guided Fuzzing," in *ASE*, 2021.

[162] G. Ye, T. Hu, Z. Tang, Z. Fan, S. H. Tan, B. Zhang, W. Qian, and Z. Wang, "A Generative and Mutational Approach for Synthesizing Bug-Exposing Test Cases to Guide Compiler Fuzzing," in *ESEC/FSE*, 2023.

[163] R. K. Medicherla, R. Komondoor, and A. Roychoudhury, "Fitness Guided Vulnerability Detection with Greybox Fuzzing," in *ICSE*, 2020.

[164] R. Nooyens, J. Besjes, T. Bardakci, M. Beyazit, and S. Demeyer, "Test Amplification for REST APIs via Single and Multi-Agent LLM Systems," *arXiv:2504.08113*, 2025.

[165] J. Besjes, R. Nooyens, T. Bardakci, M. Beyazit, and S. Demeyer, "Agentic LLMs for REST API Test Amplification: A Comparative Study Across Cloud Applications," *arXiv:2510.27417*, 2025.

[166] C. Pacheco and M. D. Ernst, "Randoop: Feedback-Directed Random Testing for Java," in *OOPSLA*, 2007.

[167] G. Fraser and A. Arcuri, "EvoSuite: Automatic Test Suite Generation for Object-Oriented Software," in *ESEC/FSE*, 2011.

[168] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," *arXiv:2002.08155*, 2020.

[169] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, "CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation," *arXiv:2109.00859*, 2021.

[170] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, W.-t. Yih, L. Zettlemoyer, and M. Lewis, "InCoder: A Generative Model for Code Infilling and Synthesis," *arXiv:2204.05999*, 2023.

[171] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, T. Hubert, P. Choy, C. de Masson d'Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Gowal, A. Cherepanov, J. Molloy, D. J. Mankowitz, E. Sutherland Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, "Competition-Level Code Generation with AlphaCode," *Science*, 2022.

[172] M. Harman, J. Ritchey, I. Harper, S. Sengupta, K. Mao, A. Gulati, C. Foster, and H. Robert, "Mutation-Guided LLM-Based Test Generation at Meta," in *FSE Companion*, 2025.